Contents

Can every consistent time loop live in a reversible world? 1
How to read this . . . 1
Part I. Stories, with no symbols . . . 2
1. The letter nobody wrote . . . 2
2. The grandfather . . . 2
3. Lewis and Novikov: consistency . . . 2
4. A better rule: exactly one history, whatever anyone does . . . 3
5. A coin flip inside a loop . . . 3
6. Reversibility . . . 4
7. The question . . . 4
Part II. The first symbols: loops as functions . . . 5
8. Parties, inputs and outputs . . . 5
9. The environment as a map w: outputs → inputs . . . 5
10. Interventions: what the agents do . . . 6
11. Fixed points: consistent histories . . . 6
12. Process functions: exactly one fixed point, for every intervention . . . 7
13. Worked example: two parties, two bits . . . 7
14. Worked example: three parties and the Lugano loop . . . 7
15. The key lemma: someone can tell any two histories apart . . . 8
Part III. Making the loop reversible . . . 9
16. Bijections: the arithmetic of not forgetting . . . 9
17. Source and sink . . . 9
18. The permutation U . . . 9
19. Worked examples of U . . . 10
20. Open versus closed . . . 10
21. A first look at the arrow of time . . . 10
Part IV. Quantum vocabulary, one word at a time . . . 11
22. Kets: $|o\rangle$ as a perfectly distinguishable configuration . . . 11
23. Tensor products: independent agents . . . 11
24. Unitaries: reversible, nothing lost . . . 11
25. Ancillas: private equipment . . . 12
26. What "the lift works" means, and what failure looks like . . . 12
Part V. The main theorem, step by step . . . 13
27. Statement . . . 13
28. The amplitude formula, read in words . . . 13
29. The test: rows must be orthonormal . . . 14
30. Step 1: the sum splits into one factor per party . . . 14
31. Step 2: a party that receives the same input contributes a clean 0 or 1 . . . 14
32. Step 3: equal records give length 1 . . . 14
33. Step 4: different records give 0, thanks to the lemma . . . 15
34. Step 5: square, hence unitary . . . 15
35. From "unitary under unitaries" to "valid under everything" (proved; machine-checked) 15
36. The converse: inconsistency breaks the lift (proved; note Theorem 1.2) . . . 16
Part VI. The incorruptible referee: why machine-checked proof matters . . . 17
37. A story: the referee who cannot be flattered . . . 17
38. What exactly she has checked . . . 18

39. What a referee cannot check . . . 18
Part VII. Infinity: Hilbert's hotel inside a time loop . . . 19
40. A story: the hotel with infinitely many rooms . . . 19
41. The setting and the easy half (proved) . . . 19
42. The second half: count the consistent histories by chance (proved) . . . 20
43. What the referee has checked about infinity . . . 20
Part VIII. Chance in loops: mixtures and purification . . . 21
44. Classical processes: p(i | o) . . . 21
45. The polytope: a map of all consistent loops . . . 22
46. Purification: randomness as ignorance of a hidden pure state . . . 22
47. Mixtures of process functions are purifiable (proved) . . . 23
48. The classical version settles itself (proved) . . . 23
Part IX. The random corners . . . 24
49. E_ex1, formally . . . 24
50. Why E_ex1 cannot be purified: isotropic subspaces, gently . . . 24
51. The other random corners . . . 25
Part X. The conjecture, the evidence, and the refutation . . . 26
52. The conjecture . . . 26
53. A story: two workshops . . . 26
54. The hidden-label idea (Conjecture D.3) . . . 26
55. The counterexample: contextual randomness (proved; note D.4) . . . 27
56. Two things that cannot be the criterion . . . 28
57. The facet evidence . . . 28
58. Walking from Lugano to E_ex1 . . . 29
58a. Two parties: there is nothing to purify away (proved; note Appendix G) . . . 29
58b. Three parties with a clear order (proved; note Appendix H) . . . 30
58c. A tempting shortcut that fails (proved; note H.5) . . . 30
58d. The shortcut, repaired inside the machine (proved; note H.6) . . . 31
58e. Machines with no first mover (evidence; note H.7) . . . 32
58f. What the Switchboard's rules really say (proved in part; note H.8, C.6, Appendix I) . . . 33
58g. Boards that blur two wirings (evidence; note H.10, H.11) . . . 34
58h. The bet is lost: a reversible loop with genuine chance (proved; note H.12, H.10.6) . . . 35
Part XI. The honest map of dead ends . . . 37
59. A story: the cartographer's other map . . . 37
60. The routes that failed . . . 37
61. Why the dead ends cluster . . . 38
Part XII. Philosophical discussion . . . 39
62. Consistency plus a complete record gives unitarity . . . 39
63. Lewis and Novikov, revisited . . . 39
64. Can chance be fundamental inside a loop? . . . 40
65. Determinism hidden, or created on the fly? . . . 40
66. Reversibility and the arrow of time . . . 41
67. Freedom, bilking and Lewis . . . 41
68. Time symmetry and the reading of the source and sink . . . 41
69. What formal verification adds, epistemically . . . 42
70. What the theorem does not say . . . 42
71. How big is the purifiable set? (H.13) . . . 42
72. Where the line falls (H.14) . . . 42

73. Putting it in spacetime: three time travellers and a quantum loop (H.15) 43
74. Smooth dials instead of switches, and the full facet tables (F.5, Appendix J) 44
What is proved, what is evidence, what is open 44
75. Plain-language note: what the computer now checks about smooth dials (Oct 23 2026) . . 48
76. Plain-language note: helpers of unlimited size (Oct 23 2026) 48
77. Plain-language note: the helper machine is fully reversible, and smooth dials work too (Oct 23 2026) 49
78. Plain-language note: attacking the two open puzzles (Oct 23 2026) 49
79. Plain-language note: what is new, and who got there first (Oct 23 2026) 50
80. Plain-language note: the Switchboard fails a causal test too, and why symmetry saves work (Oct 23 2026) 51
81. Plain-language note: is the Switchboard secretly a famous machine? (Oct 23 2026) 52
82. Plain-language note: the three-dial search is finished, and a proved floor for the Switchboard (Oct 24 2026) 53
83. Plain-language note: what "four terms" looks like, and is four really the minimum? (Oct 24 2026) 54
84. Plain-language note: running a time loop on an ordinary clock, and the four-dial search continues (Oct 24 2026) 55
85. Plain-language note: every dial is stiff in one direction, so there are at most four dials (Oct 24 2026) 56
86. Plain-language note: a glossary for the technical note (Oct 24 2026) 57
87. Plain-language note: the counterexample in the standard language, and infinitely large side systems (Oct 24 2026) 57
88. Plain-language note: closing the formal gaps (Oct 24 2026) 58
89. Plain-language note: the second counterexample on an ordinary clock (Oct 24 2026) . . . 58
90. Plain-language note: a path between the counterexamples, classical stand-ins, and continuous dials (Oct 24 2026) 59
91. Plain-language note: the score gap for the first counterexample shrinks (Oct 24 2026) . . 59
92. Plain-language note: how many times must a classical machine run each box? (Oct 24 2026) 60
93. Plain-language note: the H.12 score range narrows again (Oct 24 2026) 60
94. Plain-language note: ten "never-crossed" constraints fall (Oct 24 2026) 60
95. Plain-language note: a missing tool was not missing (Oct 24 2026) 61
96. Plain-language note: the last gap in the machine-checked infinite-dimensional story is closed (Oct 24 2026) 61
97. Plain-language note: the twelve broken families are broken everywhere; ten remain (Oct 24 2026) 61
98. Plain-language note: reversible loops with continuous dials, one step further (Oct 24 2026) 61
99. Plain-language note: a better floor for the first counterexample (Oct 24 2026) 62
100. Plain-language note: continuous dependence in the infinite-dimensional lift (Oct 24 2026) 62
101. Plain-language note: why the ten stubborn orbits resist one more family (Oct 24 2026) . 62
102. Plain-language note: the best strategy for the first counterexample looks final (Oct 24 2026) 63
103. Plain-language note: the continuous Lugano loop is reversible (Oct 24 2026) 63
104. Plain-language note: the best H.12 strategy is a certified local optimum (Oct 24 2026) . . 63
105. Plain-language note: what a proof for the ten stubborn orbits would need (Oct 24 2026) . 63
106. Plain-language note: four terms really is the minimum (Oct 24 2026) 63
107. Plain-language note: a slightly better floor for the first counterexample (Oct 24 2026) . . 64
108. Plain-language note: double-checking the "four terms" search (Oct 24 2026) 64
109. Plain-language note: the best H.12 strategy is the best of its kind (Oct 24 2026) 64

110. Plain-language note: how a computer can prove “there is no counterexample” (Oct 25 2026) 64
111. Plain-language note: the ten stubborn inequalities, half-closed (Oct 25 2026) 65
112. Plain-language note: the five-class search finished, and found nothing, as it should (Oct 25 2026) 66
113. Plain-language note: three of the ten stubborn inequality families are broken; seven remain (Oct 26 2026) 66
114. Plain-language note: why the broad search missed, and a full map of phase loops; the seven still stand (Oct 26 2026) 67
115. Plain-language note: a first proof for the seven, away from the Lugano loops (Oct 26 2026) 67
116. Plain-language note: two of the seven are now proved on the Lugano loops too (Oct 26 2026) 68
117. Plain-language note: a sturdier proof for orbit 66, two near-misses, and a second search (Oct 27 2026) 69
118. Plain-language note: a near-proof for orbit 57 (Oct 27 2026) 69
119. Plain-language note: orbit 57 proved on every family studied (Oct 27 2026) 70
120. Plain-language note: orbit 75 proved on every family studied (Oct 27 2026) 70
121. Plain-language note: orbits 69 and 81 proved on every family studied (Oct 27 2026) . . . 71
122. Plain-language note: orbit 28, still open; the corner is not the problem (Oct 28 2026) . . 71
123. Plain-language note: orbit 28, half of the deciding quantity proved (Oct 28 2026) 72
124. Plain-language note: an audit, a recheck, and two more converses (Oct 31 2026) 72
125. Plain-language note: alphabets of any size (Nov 2 2026) 73
126. Plain-language note: swapping out addition, and infinitely many parties (Nov 2 2026) . . 74
(a) Addition is not needed: any “free shuffle” will do 74
(b) Infinitely many parties, the well-behaved case: proved 75
(c) The badly-behaved case: it can fail (a consistency result, now machine-checked) . . 76
(d) Settled in ordinary set theory (Nov 3) 77
127. Four stories for the Nov 2 results 77
Story 1: the guest list that closes itself (the countable closure trick) 78
Story 2: the cloakroom (the free shuffle) 78
Story 3: the infinite village of gossips (compactness) 78
Story 4: Hilbert’s hotel of time travellers (the counterexample in ordinary set theory) . . 79
128. Three stories for finite prefixes, one-sided ledgers, and a hidden perfect set 80
Story 1: the night that is almost consistent (finite prefixes) 80
Story 2: ledgers that only count one way (Σ^0_2 and Π^0_2) 81
Story 3: the even seats (a bad map hidden on a perfect dust) 81
129. Two stories for countable lists, finite corners, and a dense dust 82
Story 1: the boxes a countable closed list cannot hold 82
Story 2: finite corners, and why a dense dust cannot be countable 83
130. A finite palette of whole lists, and a witness that runs away 84
Story 1: the hotel that only ever prints finitely many guest lists 84
Story 2: the witness that runs off the end of the hotel 84
Appendix: superseded readings (written assuming the conjecture) 85

Can every consistent time loop live in a reversible world?

A gentle, graded version of “Every process function lifts to a unitary process”

Draft. Not refereed. This is a slow, explanatory retelling of the technical note paper/lift.md (web version: the technical note). The technical note is the source of truth. If anything here seems to say

more than the note does, the note wins, and the mismatch is a mistake in this version.

How to read this

The note has two results. First, every consistent deterministic loop (a process function) can be made quantum-reversible. Second, and this is now the note's main result, the natural guess that the *only* chancy loops that can be made quantum-reversible are mixtures of deterministic ones is **false** (Section 58h). It also draws some philosophical lessons. This version takes the same path, but slowly. It begins with stories and no symbols at all. Then it brings in notation one piece at a time. Each time a new symbol appears, I try to say three things:

1. **What it stands for.** The plain meaning.
2. **Why this choice.** Why this piece of mathematics captures the idea, rather than some other piece.
3. **What it commits us to.** Every formalism takes sides on something. Choosing a symbol is often choosing a philosophy.

One thing this is not about: spacetime. The stories below use time machines and wormholes as motivation, but the mathematics is about abstract loops between laboratories. A unitary process on a finite alphabet is not a solution of Einstein's equations, and not a wormhole. The theorem says that a consistent loop's list of events has a unitary dilation. It does not say that the dilation is a spacetime (see paper/EMBEDDING.md).

Claims are marked by status, following Section 9 of the note:

- **Proved** means there is a proof. Where the proof is also machine-checked in Lean, the Lean name is followed by a label such as [L-123]. It links to the Lean results index (Appendix M of the technical note), which lists every checked theorem with its file, its exact assumptions and the axioms it uses. Those are always only Lean's three standard ones. **Proved, refereed** means a paper proof that an independent referee pass has checked.
- **Evidence** means computer experiments or partial results that point one way but do not prove it.
- **Open** means nobody knows yet.
- **Interpretation** means philosophy. It is a reading of the mathematics, not a result.

Looking up a word. The technical note has a glossary (Appendix K of paper/lift.md). It defines every main technical term, in alphabetical order, says where in the note it is defined and whom it comes from, and flags words this note uses differently from the literature.

The path runs from easy to hard. Stories first (Part I). Then the symbols, one at a time (Parts II–IV). Then the main theorem and its proof (Part V), and why a machine-checked proof matters (Part VI). Then infinity, with Hilbert's hotel smuggled into a time loop (Part VII). The second half is about chance: mixtures and purification (Part VIII), the random corners that cannot be purified (Part IX), the conjecture, the evidence that seemed to support it, and the counterexample that refutes it (Part X, ending in Section 58h), and an honest map of the routes that failed (Part XI). Part XII is all philosophy, clearly marked as interpretation. A summary box at the very end says what is proved, what is evidence, and what is open.

Part I. Stories, with no symbols

1. The letter nobody wrote

A woman finds a letter in her attic. It contains the plans for a machine. She builds the machine, and it is a time machine. She goes back a few decades and leaves the letter in the attic, where she will find it.

Nothing in this story is contradictory. Every event has a cause, and every event fits with every other event. Yet something is off. Who designed the machine? The plans were copied from the letter, and the letter was copied from the plans. The information came from nowhere.

Philosophers call this a *bootstrap* (or "ontological paradox"). It is not a contradiction. The trouble is the opposite: the story is *underdetermined*. Physics and the past together do not tell us what the letter says. A letter with completely different plans would have been just as consistent. So would a blank letter, which she never acts on. The loop permits many histories and does nothing to choose between them.

2. The grandfather

The more famous story runs the other way. A man travels back and prevents his grandfather from meeting his grandmother. Then he is never born, so he never travels back, so the grandparents meet, so he is born, so he travels back…

Here the trouble is the reverse of the bootstrap. There is no consistent history at all. Every candidate history contradicts itself.

The two stories mark the two ways a time loop can fail:

- **Too many** consistent histories (the bootstrap: underdetermination).
- **Too few**, namely none (the grandfather: paradox).

Keep this pair in mind. The whole note turns on asking for *exactly one*.

3. Lewis and Novikov: consistency

David Lewis, in "The paradoxes of time travel" (1976), dealt with the grandfather by splitting the word "can". Can the traveller kill his grandfather? He *can*, relative to his own abilities, his gun, and the local facts. He *cannot*, relative to the whole past, which includes the fact that the grandfather survived. Something will stop him: a slip on a banana peel, a jammed gun. There is no contradiction in the world. There is only a contradiction in the wish.

Physicists reached a similar view. Around 1990, Friedman, Morris, Novikov, Thorne and others proposed what is now called the *Novikov self-consistency principle*: only self-consistent histories occur. Echeverria, Klinkhammer and Thorne (1991) then studied a billiard ball fired through a wormhole time machine. They chose "dangerous" starting conditions, where the ball would emerge in the past and knock its younger self off course: a billiard-table grandfather. One expects no consistent history. They found the opposite. A wide class of dangerous starts has *infinitely many* consistent continuations. Some quiet starts, such as a ball at rest far from the wormhole, have exactly one. They searched for a start with *no* consistent continuation and found none.

So the billiard balls illustrate the bootstrap, not the grandfather. The real trouble was too many histories, not too few.

So consistency, by itself, has two gaps. It throws out the grandfather histories, but it does not pick among the bootstrap ones, and the billiard balls show that the bootstrap is the live danger. And, as John Earman warned, a consistency condition stuck on after the laws of motion is not itself a law of motion. It tells you which films are allowed. It does not tell you how the film runs.

4. A better rule: exactly one history, whatever anyone does

Here is a stronger demand. Imagine several people, each in a closed laboratory. Each receives something from outside (a message, a reading, a ball) and sends something back out. Somewhere outside the labs, the world may route their outputs back into their own past inputs. That is the time loop.

Now demand this:

However each person chooses to behave, there is exactly one consistent history.

Put plainly, "exactly one" rules out both failures at once. It rules out the grandfather (no consistent history), because there is always at least one. It rules out the bootstrap (more than one), because there is never more than one. Two old ideas have the same shape. On Russell's analysis, a phrase like "the author of Waverley" succeeds only if exactly one thing fits it. With none, or with two, sentences about "the author" come out false. The process-function demand makes "the history, given these strategies" a description that succeeds for every strategy. Newton's mechanics is prized because a well-posed initial-value problem has exactly one solution, given the starting conditions. And the demand does both *without taking anybody's freedom away*. The people in the labs may adopt any strategy whatsoever: "if I receive a 0, send a 1", "always send 0", anything. The loop is not allowed to forbid an action. It may only settle how everything fits together once the actions are chosen.

The note calls a loop with this property a **process function**. The idea comes from Baumeler and Wolf, and the "you may still do anything you could do in an ordinary lab" clause is the *no-new-physics* principle of Baumeler, Costa, Ralph, Wolf and Zych.

Lewis would recognise the move. The local "can" is untouched: every strategy is available. The global facts adapt, and they always adapt in exactly one way.

5. A coin flip inside a loop

Now a stranger story, which will come back near the end.

Three people sit in a ring of labs. Each holds one bit, a 0 or a 1. Before anything happens, a fair coin is tossed out of everyone's sight.

- **Heads**, the loop is a *copier*: each person receives exactly the bit sent by the previous person in the ring.
- **Tails**, the loop is a *negator*: each person receives the *opposite* of the bit sent by the previous person.

Neither loop is consistent on its own. Suppose everyone simply passes on what they receive. In the copier world, "all 0" and "all 1" both work: two histories, a bootstrap. In the negator world, a bit flipped three times around the ring must equal its own opposite: no history, a grandfather.

But look at the pair. Whenever the copier has two consistent histories, the negator has none. Whenever the negator has two, the copier has none. When someone ignores their input and sends a fixed bit, each loop has exactly one. Averaged over the coin, the number of consistent histories is always exactly one. The probabilities always add up to exactly 1, whatever anyone does.

So this is a *consistent* loop in the probabilistic sense. Every strategy yields a well-defined probability distribution over what happens. But its randomness is strange. It is not the randomness of ignorance about which of several consistent loops we live in, since neither the copier nor the negator is a consistent loop. The chance seems to be woven into the loop itself.

This loop is called **E_ex1** in the literature (Baumeler and Wolf). It is the central example of the second half of the note.

6. Reversibility

One more idea before the question. A process is *reversible* if from the way things end up you could, in principle, reconstruct exactly how they began. Nothing is forgotten. The fundamental laws of physics, as currently understood, are reversible in this sense. When information seems lost, as when an egg breaks or a message is burned, it has in fact been scattered into the environment.

Many consistent loops are not reversible. In the most famous three-person loop (the "Lugano" loop, Section 14 below), two different sets of actions can lead to the same set of received messages. Looking only at what was received, you cannot tell which happened.

But we can make any loop reversible by adding two things:

- a **source**: a place at the beginning that sends things in but receives nothing, and
- a **sink**: a place at the end that receives a record but sends nothing on.

The source is like an unknown initial condition. The sink is like the rest of the universe, which keeps a record of what was done. Baumeler, Costa, Ralph, Wolf and Zych showed that every process function becomes a reversible bijection once a source and a sink are added.

7. The question

Now the question of the note can be put without symbols:

> **Can every consistent loop live inside a reversible world?**

More precisely: take a consistent classical loop, made reversible with a source and a sink. Now let the people in the labs act in fully *quantum* ways. They may put their bits into superpositions, apply operations that have no classical counterpart, and entangle their labs with private equipment. Does the whole arrangement still behave sensibly, with probabilities that always add up to exactly one? Or does quantum freedom bring paradoxes back?

The note's first answer, the **main theorem**, is: yes, for every process function. The second question is about loops with chance in them, like the coin-flip loop. Can they be realised reversibly? The note proves that the coin-flip loop E_ex1 cannot be. It had conjectured that the only chancy loops that can be are those whose chance is mere ignorance about which deterministic consistent loop obtains. **That conjecture is false** (Section 58h). A machine with three labs and one bit each produces chancy statistics that are not any mixture of deterministic loops. This has been checked exactly, re-checked independently, and proved in a proof assistant, with two stated limits.

Two further questions ride along. What if the alphabets are infinite, so that a party can send any whole number? In a hotel with infinitely many rooms, a perfectly orderly shuffle can still leave a room empty, and we must check that a time loop cannot lose probability the same way (Part VII). And how sure can we be of a short proof about something this strange? Part VI tells the story of a referee who cannot be flattered.

The rest of this document builds the symbols needed to say all that exactly.

Part II. The first symbols: loops as functions

8. Parties, inputs and outputs

We have **parties** numbered $k = 1, 2, \ldots, n$. Think of each party as a closed laboratory with a person inside. Party k has:

- a set I_k of possible **inputs**, the things it might receive, and
- a set O_k of possible **outputs**, the things it might send out.

In the small examples both sets are $\{0, 1\}$: one bit in, one bit out.

Why this choice. We describe agents only by what crosses the walls of their labs. What happens inside is the agent's own business. This is a deliberately thin picture of agency: a lab is a black box with one door in and one door out, each used once.

What it commits us to. Two things. First, *locality of agency*: each party acts only on what it receives. It cannot reach out and alter another lab directly. Second, for now, *finiteness*: the sets are finite. Countably infinite alphabets are handled in Part VII. Continuous ones remain open.

A word about "agents" and "freedom", which recur throughout. Nothing in the mathematics models anyone *choosing*. As Section 10 explains, "freedom" will mean only the range of a quantifier: a condition must hold *for every* strategy a lab could follow.

When we write a whole list of outputs, one per party, we write $o = (o_1, \ldots, o_n)$. The set of all such lists is the product $O_1 \times \ldots \times O_n$, written $\prod O_k$. Likewise $i = (i_1, \ldots, i_n)$ is a list of inputs.

9. The environment as a map w: outputs → inputs

Here is the central object. The world outside the labs is described by a function

$$w : \prod O_k \to \prod I_k$$

which takes the list of everything the parties sent out and returns the list of what each party receives. Its k-th component $w_k(o)$ is what party k receives when the parties' outputs are o.

Why a map from outputs to inputs? This direction is the heart of the formalism, so it deserves a slow explanation.

In ordinary physics we write the world as a map from earlier states to later states. Here we have deliberately *not* said which party is earlier. Perhaps A acts before B. Perhaps B before A. Perhaps, in a loop, each is in some sense before the other. So we cannot write "A's output causes B's input" as a fixed arrangement. Instead we take the agents' actions as the free variables. The world's job is then: *given everything the agents did, say what each of them saw.*

The map w is this answer. It does not mention time at all. A party's input may depend on its "future" outputs as easily as on its "past" ones. That is exactly what allows loops.

What it commits us to.

- *No global time order is assumed.* Order, if there is one, has to be read off from the shape of w (who depends on whom). It is not built in.
- *The environment is deterministic.* Given the outputs, the inputs are fixed. (Part VIII relaxes this, allowing randomness.)
- *Agents are the source of variation.* Everything "free" lives in the outputs. The world responds; it does not choose. This matches the no-new-physics stance of Section 4.

Notice one more thing. The map w alone is not yet a history. It says what the parties *would* receive for each possible list of outputs. A history is a list of outputs together with the inputs that w assigns to it, and it is only *actual* if it also fits with what the parties do on receiving those inputs. That fit is the next idea.

10. Interventions: what the agents do

Each party follows a **local intervention**, also called a strategy:

$f_k : I_k \to O_k$

"If I receive this, I send that." The list $f = (f_1, \ldots, f_n)$ is a choice of strategy for everybody.

Why a function from input to output? It is the most general deterministic behaviour of a closed lab with one door in and one door out. Whatever goes on inside, from the outside it is a rule that turns what came in into what goes out. Randomised strategies come later. They turn out to be averages of deterministic ones.

What it commits us to. We will quantify over *all* such f. That universal quantifier *is* the formal expression of freedom, and it is all that "freedom" means here. Freedom is the range of the quantifier "for every f", not an act of anyone choosing. No agent, will or decision appears in the mathematics. There is only the demand that the conclusion hold whichever f is plugged in. We are not allowed to say "the loop is fine, provided nobody adopts strategy so-and-so." That would be the loop forbidding an action, which is the move Section 4 refused. (Whether the agents are free in some deeper metaphysical sense is another matter. The note says explicitly that it does not settle this. See Section 67.)

11. Fixed points: consistent histories

Put the two together. The agents send o. The world turns o into inputs w(o). Each agent turns its input into an output, $f_k(w_k(o))$. For the story to hang together, these outputs must be the very outputs o we started with:

$o_k = f_k(w_k(o))$ for every party k.

A list o that satisfies this is a **fixed point** of the composite map $o \mapsto f(w(o))$. It goes all the way round the loop and comes back unchanged.

Why a fixed point is the right notion. A fixed point is precisely a *consistent history*. Every event in it is explained by the others, and nothing contradicts anything. The grandfather story is a loop with no fixed point. The letter-nobody-wrote story is a loop with more than one.

12. Process functions: exactly one fixed point, for every intervention

Definition (process function). A map w is a *process function* if, for every choice of local interventions $f = (f_1, \ldots, f_n)$, the map $o \mapsto f(w(o))$ has **exactly one** fixed point.

Unpack the logical shape: *for every* f, *there exists exactly one* o.

- "For every f" is freedom: no strategy is forbidden.
- "At least one o" is **no paradox**: no grandfather.
- "At most one o" is **no underdetermination**: no letter that nobody wrote.

What it commits us to. The definition encodes Lewis's "can" in its first quantifier, Novikov's consistency in its second, and adds a demand Novikov's principle lacked: uniqueness. The world is never left to choose between equally consistent pasts. Given the agents' choices, the history is settled.

13. Worked example: two parties, two bits

Let Alice (A) and Bob (B) each send and receive one bit. Write a for Alice's output and b for Bob's.

A process function. Let w(a, b) = (0, a). Alice always receives 0. Bob receives whatever Alice sent. Pick any strategies f_A and f_B. Consistency demands a = f_A(0) and b = f_B(a). The first equation fixes a. The second then fixes b. Exactly one history, whatever they do. This is ordinary one-way communication, Alice before Bob.

Not a process function. Let w(a, b) = (b, a). Each receives what the other sent: a two-way loop.

- Suppose both use the identity strategy, "send what you receive". Then a = b is all we need, so (0, 0) and (1, 1) are both consistent. *Two histories: a bootstrap.*
- Suppose Alice flips and Bob copies: a = not b and b = a. No bit equals its own negation. *No history: a grandfather.*

The same loop produces both failures, depending on what the agents do. That is typical.

A fact about two parties. There are $4^4 = 256$ maps from two bits to two bits. Exactly 12 are process functions (the note's code/check.py confirms this by brute force), and all 12 are causally ordered. In each of them, at most one direction of influence is used. With two parties and bits, consistency forces a time order.

14. Worked example: three parties and the Lugano loop

With three parties, something new happens. The **Lugano process function** (found by Araújo and Feix, and studied by Baumeler and Wolf) is

```
w(a, b, c) = ( (not b) and c,   (not c) and a,   (not a) and b ).
```

Alice's input depends on Bob's and Charlie's outputs. Bob's depends on Charlie's and Alice's. Charlie's depends on Alice's and Bob's. If you draw an arrow from each party to the parties whose input it influences, you get a cycle. There is no first party.

Yet every one of the $4^3 = 64$ deterministic strategy choices has exactly one fixed point. For instance:

- Everyone copies their input: the unique history is a = b = c = 0.
- Everyone negates their input: the unique history is a = b = c = 1.
- Alice negates and the others copy: the unique history is (a, b, c) = (1, 1, 0). Check: w(1, 1, 0) = (0, 1, 0). Alice receives 0 and sends not 0 = 1. Bob receives 1 and sends 1. Charlie receives 0 and sends 0. ✓

So Lugano is a genuine causal loop with no paradox and no bootstrap. It is the standard example of a loop that violates a *causal inequality*, a statistical test passed by every arrangement with a definite time order.

Note also that Lugano is two-to-one: its eight output lists go to only four input lists. Looking at the inputs alone, you cannot recover the outputs. Lugano is consistent but not reversible. Section 17 fixes this.

15. The key lemma: someone can tell any two histories apart

Here is the one combinatorial fact the main theorem needs. It deserves the most philosophical attention, because it is where consistency turns into something a quantum world can use.

Lemma (exclusivity). Let w be a process function. Let s and s′ be two different lists of outputs. Then there is some party k such that

- $s_k \neq s'_k$ (party k sends different things in the two lists), *but*
- $w_k(s) = w_k(s')$ (party k receives the same thing in both).

In words. Take any two different global scenarios. Some agent receives the same input in both, yet acts differently. That agent's own behaviour tells the two scenarios apart, using no information the agent did not have.

Proof, in words, then symbols. Suppose not. Then for every party, *receiving the same input in s and s′ means sending the same output*. Now build a strategy for each party:

- if you receive $w_k(s)$, send s_k;
- if you receive $w_k(s')$, send s'_k;
- otherwise do anything.

This is a legitimate strategy. The only possible clash is if $w_k(s) = w_k(s')$, and then, by our supposition, $s_k = s'_k$, so the two instructions agree. But with these strategies, s is a consistent history (each party receives $w_k(s)$ and sends s_k) and so is s′. Two distinct fixed points for one intervention: a bootstrap. That contradicts w being a process function. ∎

Worked check. For w(a, b) = (0, a), take s = (0, 0) and s′ = (1, 0). They differ at Alice, and Alice receives 0 in both. Alice is the witness. For the swap w(a, b) = (b, a), take s = (0, 0) and s′ = (1, 1). Alice receives 0 in one and 1 in the other. So does Bob. Nobody receives the same thing while acting differently. The lemma fails, and the swap duly has a bootstrap.

Where this comes from, and what it commits us to. This is the forward direction of a recent theorem of Dourdent, Leitherer, Boghiu, Simonov, Kunjwal and Acín (arXiv:2609.39735, dated 30 September 2026). They show that a list of events comes from a process function exactly when it is complete and *pairwise exclusive*. The note connects this to a principle of Specker: if any two of several questions can be answered jointly, then all of them can. Philosophically, the lemma says that consistency is a constraint on *distinguishability by local agents*. Two global histories are never so alike that no agent placed identically in both could tell them apart by its own actions. The theorem will turn this into orthogonality. In quantum mechanics, orthogonal means perfectly distinguishable.

Part III. Making the loop reversible

16. Bijections: the arithmetic of not forgetting

A function is a **bijection** if it is one-to-one and onto: every possible result comes from exactly one starting point. A bijection can be run backwards. From the result you recover the start. That is the mathematical form of "no information is lost".

Lugano is not a bijection, as Section 14 showed. Of the 256 two-bit two-party maps, twelve are process functions and none of them is its own input–output reverse (the note, Section 6). Consistency does not give reversibility for free on the original alphabet.

17. Source and sink

The fix, due to Baumeler, Costa, Ralph, Wolf and Zych, is to add two new regions.

- The **source** P is a region that only *sends*. It has no input. Its value $e = (e_1, \ldots, e_n)$ holds one symbol for each party, drawn from that party's input alphabet, so $P = \prod I_k$.
- The **sink** F is a region that only *receives*. It has no output. It records one symbol for each party, drawn from that party's output alphabet, so $F = \prod O_k$.

What they represent. Metaphysically, the source is the *initial condition*: the part of the world that is simply given and depends on nothing the agents do. The sink is the *final record*: the rest of the universe, into which every action leaves a trace. Physicists' reversible laws always come with this pair, a state you start from and a state you end in. Adding them to the loop is asking what the loop looks like as part of a whole world with a beginning and an end, rather than as an isolated knot.

18. The permutation U

The reversible version of w is the map

```
U(o, e) = ( w(o) + e,  o ).
```

Read it slot by slot. It takes the parties' outputs o and the source value e. It returns two things:

- the parties' **inputs**, w(o) + e: what the loop would deliver, shifted by the source;
- the **sink record**, o: a copy of what the parties did.

Why the "+". We label each input alphabet I_k by the numbers 0, 1, ..., $|I_k| - 1$ and add modulo $|I_k|$ (so for bits, + is exclusive-or: $1 + 1 = 0$). In mathematical terms I_k is treated as the cyclic group $\mathbb{Z}_n$, with $n = |I_k|$. All we need is that adding e can be *undone* by subtracting e. Any group would do; the cyclic one is simply the most convenient. The output alphabets O_k need no structure at all.

Why this is reversible. Given the result (i, o), recover the start: the outputs are o (the sink kept them), and the source value is $e = i - w(o)$. Nothing is lost. So U is a bijection, and it is one *for every map w*, process function or not. The source shift and the sink copy have absorbed all the forgetfulness.

What it commits us to. The sink keeps a perfect copy of the outputs. This is the classical ancestor of a principle we will meet again. In a reversible world, whatever is done leaves a record somewhere. Lemma D.1 of the note (Section 56 below) shows that something like this is *forced* in any reversible realisation, not just this one.

19. Worked examples of U

One party, constant loop. With a single party, the only process functions are constant maps (a lone party's input cannot depend on its own output, or it could contradict itself). Take w = 0. Then U(o, e) = (e, o). The source feeds the party's input, and the party's output goes to the sink. This is just an ordinary lab between a past and a future. Nothing loops.

Two parties, one-way. Take w(a, b) = (0, a). Then

```
U(a, b, e_A, e_B) = ( e_A,  a + e_B,  a, b ).
```

Alice receives the source bit e_A. Bob receives Alice's output, shifted by the source bit e_B. The sink records (a, b). This is a causal chain from the source to Alice, from Alice to Bob, and from both to the sink.

Lugano.

```
U(a, b, c, e) = ( ((not b) and c) + e₁,  ((not c) and a) + e₂,  ((not a) and b) + e₃,   a, b, c
```

The two-to-one loop has become one-to-one.

20. Open versus closed

There is a subtlety worth stating clearly, because the whole note depends on it. U is a bijection of the *open* system: outputs and source in, inputs and sink out. But the parties are still inside the loop. They take their inputs and turn them into their outputs. When we plug the parties in, the loop *closes*, and what remains is a map from source to sink.

Closing the loop is where consistency matters. For the swap w(a, b) = (b, a), U is still a perfectly good bijection. Close the loop with both parties copying, and source value e = (0, 0) has two consistent continuations, while e = (1, 0) has none (Section 26 computes this). The open bijection says nothing about the closed loop. The note's slogan: **U is a bijection for every w; closing the loop is well behaved exactly when w is consistent.** (Both halves are now proved, for local unitaries with private equipment: the note's Theorem 1.2, told in Section 36. One version without equipment is still open.)

21. A first look at the arrow of time

A story. A film shows a glass falling from a table and shattering. Run it backwards and you see shards leap from the floor and fuse into a glass on the table. Every frame of the reversed film obeys the laws of mechanics. We still know at once which way is "real".

Physicists usually explain the asymmetry by boundary conditions, not by the laws. The laws are reversible. The world happens to have started in a very special state, and records pile up toward the future. The glass leaves shards, sound waves and warmth, a sink full of traces. The source-and-sink picture of Section 17 has exactly this shape: a reversible U in the middle, a given starting state at one end, a record at the other.

Interpretation (mine, not the note's). In the lift, nothing irreversible happens inside U. Irreversibility appears only when we *fix* the source to one state and *discard* the sink, when we stop tracking the beginning and the end. Part VIII comes back to this. When a chancy loop can be realised reversibly at all, its randomness is the shadow of exactly that ignoring: a fixed source state and a discarded sink. Which chancy loops can be realised this way is the subject of Part X. The coin-flip loop of Section 5 cannot, and Section 58h shows that some that can are not mixtures of deterministic loops.

Part IV. Quantum vocabulary, one word at a time

22. Kets: |o⟩ as a perfectly distinguishable configuration

Quantum mechanics describes a system by a vector in a space of possibilities. The notation |o⟩, read "ket o", is the vector that stands for "the system is definitely in configuration o". Here o is one of our classical labels. The angle bracket marks it as a vector, not a number.

The kets for different labels are *orthogonal*: at right angles to one another. Orthogonal means *perfectly distinguishable*. There is a measurement that tells |0⟩ and |1⟩ apart with certainty.

What is new is that vectors can be added. For example,

$(1/\sqrt{2})(|0\rangle + |1\rangle)$

is a legitimate state, a *superposition*. It is neither 0 nor 1, and it is not a coin flip between them either. The numbers in front (here $1/\sqrt{2}$ each) are **amplitudes**. They can be negative or complex. When you finally look, the probability of each label is the squared magnitude of its amplitude: here ½ and ½.

Why this notation. The ket keeps the classical label in view while placing it inside a larger space. The classical world is not thrown away. It survives as a special basis, the "computational basis", of the quantum one.

What it commits us to. A wider space of possibilities than the classical one. Between "definitely 0" and "definitely 1" there are not just probabilities but superpositions with phases, and phases can *interfere*. Two routes to the same result may cancel. Any consistency condition that holds only for the classical labels might fail once we allow superpositions. That is the danger the note has to rule out.

We also write ⟨o| ("bra o") for the mirror image of |o⟩, and ⟨u|v⟩ for the inner product of two vectors. It is 1 when they are the same unit vector, 0 when they are orthogonal, and something in between otherwise.

23. Tensor products: independent agents

When two systems sit side by side, the space for both together is the **tensor product**, written ⊗. If Alice's bit lives in a two-dimensional space and Bob's in another, the pair lives in a $2 \times 2 = 4$-dimensional space with basis |0⟩⊗|0⟩, |0⟩⊗|1⟩, |1⟩⊗|0⟩, |1⟩⊗|1⟩. We usually write these as |00⟩, |01⟩, and so on.

When Alice acts with V_A on her system and Bob acts with V_B on his, the joint operation is V_A ⊗ V_B. It acts on each factor separately.

Why this captures independence. The tensor product is built so that local operations compose without interfering. Alice's action does not depend on which basis Bob uses, and the amplitude of a joint event is the product of the local amplitudes. Dimensions multiply, just as the number of joint classical configurations does.

What it commits us to. Each lab is a separate factor, and each agent acts only on its own factor. This is the quantum version of the locality of agency from Section 8. It also allows *entanglement*, joint states that are not products. The note's validity argument has to handle entangled equipment too (Section 35).

24. Unitaries: reversible, nothing lost

A **unitary** is a linear map U on a space of states that preserves every inner product. Equivalently,

$$U\,U^\dagger = U^\dagger\,U = 1.$$

Here U† ("U dagger") is the conjugate transpose: flip the matrix across its diagonal and take complex conjugates. The 1 is the identity map, which does nothing.

What this says. U† undoes U. Every unitary is reversible, and its reverse is U†. Because inner products are preserved, perfectly distinguishable states stay perfectly distinguishable. Total probability is conserved: a normalised state stays normalised.

Why unitary = reversible = no information lost. These are three faces of one property. "Reversible": U† runs the process backwards. "No information lost": distinguishable inputs remain distinguishable outputs, since orthogonal stays orthogonal. "Probability conserved": lengths are preserved, so probabilities still sum to 1. In quantum theory the fundamental, closed-system dynamics is unitary. Anything else arises by ignoring part of a larger unitary story (Section 35).

Classical bijections are unitaries. A bijection of labels, extended to superpositions by linearity, is a *permutation matrix*. That is a matrix of 0s and 1s with exactly one 1 in each row and each column, and it is unitary. So U from Section 18 becomes the operator

$$U\,|o\rangle_O\,|e\rangle_P = |w(o) + e\rangle_I\,|o\rangle_F.$$

The subscripts say which system each ket lives in: the parties' outputs O, the source P, the parties' inputs I, and the sink F. This is exactly the classical table, written in kets. It is the starting point of the note (Section 1 there).

The Hadamard: a test with no classical analogue. The Hadamard gate is

$$H|0\rangle = (|0\rangle + |1\rangle)/\sqrt{2}, \quad H|1\rangle = (|0\rangle - |1\rangle)/\sqrt{2}.$$

It is unitary, but it is not a permutation. It turns definite bits into superpositions, with a minus sign that can cause cancellation. If a party inside the loop applies H, the loop sees something no classical strategy could produce. The note calls this "the test case". If U only behaved well for classical strategies, a Hadamard inside the loop is where it would break.

25. Ancillas: private equipment

A real laboratory has more than one bit's worth of equipment. We allow party k a private input system X_k and a private output system Y_k (an **ancilla**, from the Latin for "servant"). The party's most general reversible action is then a unitary

$$V_k : I_k \otimes X_k \to O_k \otimes Y_k,$$

which needs the dimensions to match: $|I_k|\cdot|X_k| = |O_k|\cdot|Y_k|$. This also lets input and output alphabets have different sizes (Appendix B of the note).

What it commits us to. The agents are as powerful as quantum theory allows inside their labs. The theorem quantifies over *all* such V_k, with ancillas of any finite size. Again, the universal quantifier is the formal shape of freedom.

26. What "the lift works" means, and what failure looks like

Plug every party's V_k into U. Feed each input system I_k into V_k, and feed each output O_k back into U. What remains is a map from the source plus the parties' input ancillas to the sink plus their output ancillas:

$M : P \otimes X \to F \otimes Y.$

The lift works if M is unitary for every choice of local unitaries V_k. Araújo, Feix, Navascués and Brukner (2017, Definition 1) call such a process **pure**: unitaries in, unitary out.

Why this is the right test. If M were not unitary, some source state would come out with total probability more than 1 or less than 1. That would be a bootstrap or a grandfather appearing *in the quantum world*, even though the classical loop was fine.

Failure, worked out. Take the bad loop w(a, b) = (b, a), and let both parties do nothing (each V_k is the identity, "send what you receive"). The amplitude M[s, e], for sink record s given source value e, is 1 if s is consistent and 0 otherwise. Consistent means $s_k = w_k(s) + e_k$ for both parties. Notice that the source shift turns "do nothing" into the strategy "subtract e_k". So each column of M counts the consistent histories of one classical intervention. Computed:

- e = (0, 0): s = (0, 0) and s = (1, 1) both work. The column has squared length 2.
- e = (1, 0): s_A = s_B + 1 and s_B = s_A. No solution. The column has length 0.
- e = (0, 1): length 0. e = (1, 1): length 2.

So M is not unitary. Probability is doubled for some source states and annihilated for others. The bootstrap and the grandfather reappear, as an excess and a deficit of probability.

For a process function and identity V_k, every column has exactly one 1, which is the classical definition again. The real question is what happens when the V_k are Hadamards, or arbitrary unitaries, so that many histories interfere.

Part V. The main theorem, step by step

27. Statement

> **Theorem (proved).** Let w be a process function with finite alphabets. Let each party k apply any unitary $V_k : I_k \otimes X_k \to O_k \otimes Y_k$, with private ancillas of any finite size. Then the induced source-to-sink operator $M : P \otimes X \to F \otimes Y$ is unitary.

In plain words: **every consistent deterministic loop, made reversible with a source and a sink, stays reversible however quantum the agents inside it act.** Quantum freedom does not bring the paradoxes back.

28. The amplitude formula, read in words

The proof starts by writing down M explicitly. Its entries are

$$M[(s, y), (e, x)] = \prod_k V_k[(s_k, y_k), (w_k(s) + e_k, x_k)].$$

Here is how to read this.

- **Left side.** The amplitude for the world to go from source value e and input ancillas x to sink record s and output ancillas y.
- **Right side.** The sink records s, and the sink holds the parties' outputs. So s *is* the list of what the parties sent. Given that, the loop delivers to party k the input $w_k(s)$, shifted by the source to $w_k(s) + e_k$. Party k then has to turn that input (with its ancilla x_k) into its output s_k (with ancilla y_k). The amplitude for that is the matrix entry $V_k[(s_k, y_k), (w_k(s) + e_k, x_k)]$.

- **The product $\prod_k$.** The parties act independently, so the joint amplitude is the product of the local amplitudes. This is the tensor product of Section 23 doing its job.

Philosophical point. Because the sink records the outputs, each entry of M corresponds to *a single* consistent assignment of inputs and outputs. There is no sum over alternative histories inside an entry. The sink's record has made the histories distinguishable, so they cannot interfere with each other on the way to the same final state. Whatever interference there is happens *between* entries, and that is what the next step controls.

29. The test: rows must be orthonormal

A square matrix is unitary when its rows are orthonormal. Each row must have length 1, and any two different rows must be orthogonal. In symbols, $M\,M^\dagger = 1$, which written out is

$$(M\,M^\dagger)[(s, y), (s', y')] = \Sigma \text{ over } e, x \text{ of } M[(s, y), (e, x)] \cdot \text{conj } M[(s', y'), (e, x)].$$

We need this to be 1 when $(s, y) = (s', y')$ and 0 otherwise.

Meaning. The rows are labelled by final records. So the test asks whether *different final records are perfectly distinguishable*, and whether each one carries total weight 1. Reversibility, in this form, is a statement about the records the world keeps.

30. Step 1: the sum splits into one factor per party

Plug the product formula into the sum. The sum is over all source values $e = (e_1, \ldots, e_n)$ and all ancillas $x = (x_1, \ldots, x_n)$. Each party's factor involves only that party's own e_k and x_k. A sum of products of independent pieces is a product of sums:

$$(M\,M^\dagger)[(s, y), (s', y')] = \prod_k \left(\Sigma \text{ over } e_k, x_k \text{ of } V_k[(s_k, y_k), (w_k(s) + e_k, x_k)] \cdot \text{conj } V_k[(s'_k, y\right.$$

Meaning. The global question breaks into one local question per lab. This is where the source does its real work. Because each party has its own independent source symbol e_k, summing over it lets each lab's input range over everything.

31. Step 2: a party that receives the same input contributes a clean 0 or 1

Look at party k's factor when $w_k(s) = w_k(s')$. Then both V_k entries have the *same* input slot, $w_k(s) + e_k$. As e_k runs through all of I_k, so does that slot. Rename it i. The factor becomes

$$\Sigma \text{ over } i, x_k \text{ of } V_k[(s_k, y_k), (i, x_k)] \cdot \text{conj } V_k[(s'_k, y'_k), (i, x_k)],$$

which is just the inner product of two rows of V_k. Since V_k is unitary, it is 1 if $(s_k, y_k) = (s'_k, y'_k)$ and 0 otherwise.

Meaning. For a party in the same situation in both scenarios, local reversibility settles the question. Its factor is 1 if it did the same thing in both, and 0 if it acted differently.

32. Step 3: equal records give length 1

If $s = s'$, then $w_k(s) = w_k(s')$ for every k trivially, so every factor is of the kind in Step 2. The product is 1 exactly when $y = y'$. So every row has length 1, and rows with the same s but different ancilla records are orthogonal.

33. Step 4: different records give 0, thanks to the lemma

If $s \neq s'$, the factors of parties with $w_k(s) \neq w_k(s')$ could be anything. Here is where consistency enters. The exclusivity lemma (Section 15) guarantees a party k with $w_k(s) = w_k(s')$ and $s_k \neq s'_k$. That party's factor is of the Step 2 kind, and since $s_k \neq s'_k$, it is 0. One zero factor makes the whole product 0.

Meaning. Two different final records are orthogonal because some agent, placed identically in both scenarios, acted differently. Its local unitarity forces the zero. Without a process function, there may be no such witness, and the other factors can conspire to make $M\,M^\dagger \neq 1$. The swap of Section 26 shows them doing it.

34. Step 5: square, hence unitary

So $M\,M^\dagger = 1$. M is also square: the source-plus-input-ancilla side has dimension $\prod |I_k||X_k|$, and the sink-plus-output-ancilla side has dimension $\prod |O_k||Y_k|$, which are equal because each $|I_k||X_k| = |O_k||Y_k|$. For square matrices, $M\,M^\dagger = 1$ implies $M^\dagger\,M = 1$. So M is unitary. ∎

The whole proof in one sentence. Each final record picks out a single consistent history (the sink), each lab contributes independently (the tensor product), a lab in the same situation in two histories contributes a clean 0 or 1 (local unitarity), and consistency guarantees that any two different histories have such a lab that acted differently (exclusivity).

Checked by machine. The lemma and both halves of unitarity are machine-checked in the Lean proof assistant, for complex amplitudes, any finite number of parties, any finite alphabets and any finite ancillas. Part VI explains what that means and why it matters.

A warning for later. Step 5 is the only step that uses counting: a *square* matrix with orthonormal rows is unitary. In infinite dimensions that step fails, and Part VII has to replace it with a different idea.

35. From "unitary under unitaries" to "valid under everything" (proved; machine-checked)

Agents in labs do not only apply unitaries. They measure, add noise, throw things away, and use equipment entangled with other labs. Does the result survive all that? The note's Appendix A shows that it does, and every step is now machine-checked in Lean (Part VI says why that matters).

The key tool is **Stinespring dilation**. Any physically allowed operation (a "CPTP map", in the jargon: completely positive and trace preserving) can be written in three steps. Add a fresh ancilla in a fixed state, apply a unitary to everything, then throw part of the result away. Throwing away is a *partial trace*: we average over what we can no longer see.

Philosophical gloss. Every noisy or irreversible operation is a reversible one *seen partially*. Irreversibility is ignorance of a larger story in which nothing is lost. This is the first appearance of the idea of **purification**, which drives the second half of this document.

With Stinespring in hand, the argument has four steps. Each comes with the Lean theorem that checks it.

1. **Local unitaries give a unitary M.** This is the theorem of Part V. Lean also proves the version where the local maps are isometries, not necessarily square (`PFUL.induced_isometry` L-149). Packaged as a validity statement, `PFUL.valid` L-231 says the following. For any finite ancillas and any joint quantum state of the source and the labs' equipment, entangled or shared, both

the global output and the state left in the sink are genuine density matrices: positive, with total probability 1.

2. **General local operations.** Any operation given by Kraus operators has an isometric Stinespring dilation that reproduces it (`PFUL.krausDilP_isometry` L-027, `PFUL.ptrace_krausDil` L-028). Validity for arbitrary local CPTP maps is `PFUL.valid_kraus` L-030.
3. **Entangled and shared equipment.** This is already covered by step 1. The joint starting state is allowed to be anything.
4. **Measurements with outcomes (instruments).** Each outcome's conditional sink state is positive, and they add up to a density matrix, so outcome probabilities are non-negative and sum to 1 (`PFUL.valid_instrument` L-029).

The process-matrix form. Physicists usually state validity differently, as a "generalised Born rule". Write the whole process as one matrix $W = |U\rangle\rangle\langle\langle U|$. Each lab's operation is likewise written as its *Choi matrix* C, a positive matrix that encodes the operation completely. The rule then asks that $\mathrm{Tr}[W(\rho \otimes 1 \otimes C_1 \otimes \ldots \otimes C_n)] = 1$ for every source state ρ and every allowed local operation. Lean checks this too:

- W is positive (`PFUL.Choi.procMat_posSemidef` L-037).
- The Born-rule normalisation equals 1 for all local CPTP Choi matrices (`PFUL.Choi.bornC_eq_one` L-031), and also with lab ancillas (`PFUL.Choi.bornC_anc_eq_one` L-035, `PFUL.Choi.bornAnc_eq_one` L-020). The last needs no structure at all on the ancilla types.
- The two ways of stating validity, the Born rule and "outputs are density matrices", are equivalent (`PFUL.Choi.afnb_iff_density` L-034, and `PFUL.Choi.afnb_iff_density_split` L-019 in the exact layout of `PFUL.valid` L-231).
- **Choi's theorem** itself is proved: an operation is completely positive exactly when its Choi matrix is positive (`PFUL.Choi.isCP_iff_choi` L-044). So "completely positive" is a theorem about the matrices, not a definition slipped in by hand.
- One theorem bundles the whole chain: unitary validity, Kraus-channel validity and Born-rule normalisation for every completely positive, trace-preserving local operation (`PFUL.Choi.validity_package` L-022).

Everything in this list is for finite dimensions, with standard axioms only. The one remaining technicality is the Choi form in *infinite* dimensions (Section 43).

Fix the source in $|0\rangle$ and discard the sink, and you get back the classical loop w: each output o yields the input w(o) with certainty. The quantum lift contains the original classical loop as a special case.

36. The converse: inconsistency breaks the lift (proved; note Theorem 1.2)

What if w is *not* a process function? Then the exclusivity property fails (Dourdent et al.). For a long time the note had only numbers here: every non-process function in the two- and three-party binary cases fails unitarity for random local unitaries (code/check.py). Now it has a proof.

Theorem 1.2 (unitary iff consistent; proved). The closed loop is unitary for every choice of local unitaries and finite ancillas *if and only if* w is a process function.

The "if" half is the main theorem. It needed more than columns of length one. It needed distinct rows to be orthogonal, because unit-length columns need not be orthogonal, and orthogonality is what exclusivity supplies.

A story for the "only if" half: the ledger that counts histories. Suppose w is not a process function. Then some intervention f has the wrong number of consistent histories. Call that number N. It is 0 in the

grandfather case and 2 or more in the bootstrap. Give each lab a private notebook (an ancilla, Section 25). The lab's unitary does two things at once. It applies f to what it receives, and it writes a copy of what it received into the notebook. Because the copy is kept, nothing is erased, and the operation is a genuine unitary. Now start the source and the notebooks blank and look at the closed loop's column for that start. Every consistent history of f writes exactly one line, and the copies keep the lines apart so they cannot cancel. So **the squared length of the column counts the fixed points of f.** It equals N. A unitary needs every column to have length 1. N is not 1, so the loop is not unitary. The grandfather case loses probability; the bootstrap counts it twice. (In symbols: V_k|i, x⟩ = |f_k(i) + x, i⟩, and column (0, 0) has squared norm #{s : f(w(s)) = s} = N.) The gadget never needs input and output alphabets of equal size.

Checked by computer too. The criterion "unitary for all copy-gadget interventions" was compared with brute-force process-function status on all 256 two-party binary maps and about 13,000 further maps, with 0 mismatches. In every run the squared column norm equalled the fixed-point count.

Still open. Two things. The converse *without* ancillas, that some equipment-free local unitaries already break unitarity when w is not a process function, is not proved. It is only sampled: one random draw per map broke unitarity for every non-process function tried. And Theorem 1.2 does not cover countably infinite or continuous alphabets. *Update (Nov 2 2026): both of these are now settled for discrete alphabets. The converse without ancillas is proved and machine-checked (Section 124). Both converses hold for alphabets of any size, countable or not (Sections 125 and 127). Only genuinely continuous alphabets remain.*

Machine-checking. Both directions are now machine-checked in Lean: the "if" direction (Part VI) and the copy gadget (`PFUL.CopyGadget.unitary_iff_pf` L-066, Section 124).

The code also checks the theorem by brute force: all 12 two-party and 744 three-party binary process functions, under random local unitaries, with and without ancillas, plus three-symbol two-party examples. The deviation from unitarity stays below 10^{-13}.

What has been shown, philosophically. Daher Ahmed and Kunjwal (2026) had conjectured this result and proved it for a subclass. They noted that the bare permutation unitary might fail to be a valid process. The note argues that it never fails. Araújo et al. (2017) had already written down this construction for Lugano, and Baumeler, Gilani and Rashid (2022) had remarked on unitary extensibility without a separate proof. The note's contribution is a short general proof for the coherent permutation unitary. Its philosophical reading, refined in Section 62, is that **consistency plus a complete record gives unitarity**. No extra quantum filter is needed.

Part VI. The incorruptible referee: why machine-checked proof matters

37. A story: the referee who cannot be flattered

Imagine a referee with three odd traits. She reads every line, never skims, and never gets tired. She has no reputation to protect and no friends to please, so a famous name on the paper means nothing to her. And she accepts nothing she cannot rebuild from first principles. "It is well known that…" earns a polite refusal until you show her why.

Human refereeing has none of these traits reliably. Short proofs in mathematical physics often hide a step that "obviously" generalises: a square-matrix argument quietly reused in infinite dimensions, a sum swapped with a limit, an ancilla assumed to be unentangled. The proof in Part V is short. Part VII

showed that one of its innocent-looking steps, "orthonormal rows plus square means unitary", is exactly the step that fails in Hilbert's hotel.

Lean is a proof assistant: a programming language in which definitions and proofs are written so precisely that a small, trusted program (the *kernel*) can check every inference. **Mathlib** is its large shared library of formalised mathematics. A theorem that compiles has been checked by the referee who cannot be flattered.

38. What exactly she has checked

These results are machine-checked, in Lean 4.34.1 with Mathlib, using only Lean's three standard axioms (`propext`, `Classical.choice`, `Quot.sound`). There is no "sorry", meaning no unproved step, no custom axiom, and in the main results no `native_decide`, a shortcut that trusts the compiler.

- **The definition.** `PFUL.IsProcessFunction` w says that for every strategy f there is exactly one s with $f_k(w_k(s)) = s_k$ for all k. That is the definition of Section 12, transcribed.
- **The lemma is derived, not assumed.** `PFUL.exclusivity` L-226 proves Section 15 from the definition.
- **The main theorem.** $M\ M^\dagger = 1$ (`PFUL.induced_mul_conjTranspose` L-228), $M^\dagger\ M = 1$ (`PFUL.induced_conjTranspose_mul` L-227), and membership in the unitary group (`PFUL.induced_unitary` L-229). These hold for complex amplitudes, any finite number of parties, any finite alphabets, and ancillas of any finite size, with input and output sizes allowed to differ. The row identity even holds over any field with a conjugation.
- **Validity under everything** (Section 35), including Choi's theorem, ending in `PFUL.Choi.validity_package` L-022.
- **Countable alphabets** (Part VII), ending in `PFUL.Countable.induced_unitary_l2` L-002.
- **Both converses, alphabets of any size** (Sections 124–126), ending in `PFUL.General.unitary_l2_iff_pf_any` L-132, `PFUL.General.unitary_l2_noAncilla_iff_pf_any` L-133 and, with addition replaced by a free shuffle, `PFUL.General.unitary_l2_iff_pf_σ` L-017.
- **Infinitely many parties** (Section 126), `PFUL.Infinite.pf_of_exclusive_continuous` L-134 for continuous loops and `PFUL.Infinite.exists_exclusive_not_pf_of_covM` L-119 for the counterexample under an extra set-theoretic assumption.

39. What a referee cannot check

She checks that the proof proves the statement. She cannot check that the statement says what we *mean*. Two episodes from the repository's own history make the point.

- **The weaker theorem that looked like the real one.** Early Lean files proved row cancellation for integer-valued "rows". There, row orthonormality was a *hypothesis*, and so, in one file, was the witness party that the exclusivity lemma is supposed to supply. Those files compile. They are honestly labelled in lean/STATUS.md as "proved only under assumptions (NOT the paper's theorem)". A green tick on the wrong statement is still a green tick.
- **A claim marked as false on purpose.** The Lean files define a placeholder `SpacetimeEmbeddingClaim` and set it equal to False. This is a deliberate flag that nothing here says the loop is a spacetime (Section 70).

So machine-checking moves trust; it does not remove it. Trust moves away from the long chain of inferences and onto three short things: the small kernel, the three axioms, and the *definitions*. Anyone

can read `PFUL.IsProcessFunction` in two lines and judge whether it captures "exactly one consistent history for every intervention". That is a much smaller thing to trust than a long chain of index-juggling.

What remains unchecked. The process-matrix (Choi) form $\mathrm{Tr}[W(\ldots)] = 1$ in infinite dimensions. The trace form of validity there is now checked (see below). Continuous alphabets, which have no settled definition yet. Everything in the stochastic half (Parts VIII–XI): the certificates there are exact computer arithmetic, but they are not Lean proofs. And, of course, every philosophical claim.

Part VII. Infinity: Hilbert's hotel inside a time loop

40. A story: the hotel with infinitely many rooms

David Hilbert liked to describe a hotel with infinitely many rooms, numbered 1, 2, 3, and so on, all of them occupied. A new guest arrives. The manager asks every guest to move from room n to room $n + 1$. Nobody is turned out, nobody shares a room, and room 1 is now free. The move was perfectly one-to-one, and still a room appeared from nowhere. Run it the other way, everyone from $n + 1$ to n, and the guest in room 1 has nowhere to go.

Now smuggle the hotel into a time loop. Suppose a party's alphabet is infinite: it can receive and send any whole number. The source, the sink and the loop shuffle "guests" (basis states) between infinitely many "rooms". In a finite hotel, a shuffle that never doubles guests up must fill every room. That is the counting step at the end of the finite proof (Section 34: a square matrix with orthonormal rows is unitary). In an infinite hotel that step is simply false. The map could be like Hilbert's manager. Different sink records would stay perfectly distinguishable, but some state coming in from the source could fail to arrive anywhere in full. Probability would *leak* into the infinite corridors.

In quantum terms, the operator that moves n to $n + 1$ (the *shift*) has orthonormal rows but is not unitary. So the question for infinite alphabets is sharp. **Does the loop leak?**

41. The setting and the easy half (proved)

The note's Appendix F takes finitely many parties. Each input alphabet I_k is countable and carries a group structure, so the source can still "add" its value. The integers $\mathbb{Z}$ are the model case. Each output alphabet O_k is countable. Quantum states live in ℓ^2, the space of square-summable sequences over these sets: the infinite-dimensional analogue of the finite spaces we have used so far.

- **The open map is still a bijection (proved, F.1).** $U|o, e\rangle = |w(o) + e, o\rangle$ shuffles basis states one-to-one and onto, with inverse $(i, f) \mapsto (f, i - w(f))$. So U is unitary on ℓ^2. Here the hotel is harmless: U is a genuine bijection.
- **No interference between different histories (proved, F.2).** The exclusivity lemma holds for alphabets of any size. Its proof only ever prescribes two values of each strategy. Each matrix entry of M is still a single product with no infinite sum hiding inside it, so no question of convergence arises. The rows of M are orthonormal, $M\,M^\dagger = 1$, exactly as in Part V.

That is the half of unitarity that says *distinct records never blur together*. The other half, that *nothing leaks*, is where the hotel lurks.

42. The second half: count the consistent histories by chance (proved)

The finite proof got the second half for free by counting dimensions. The new proof (Appendix F.3) replaces counting with probability, and the idea is lovely.

Step 1: read each lab as a random rule. Fix a source value e (with ancilla x). Each party's unitary V_k turns an input into a superposition of outputs. The squared amplitudes give, for each input i, a probability distribution $q_k(\cdot \mid i)$ over outputs. Because V_k is unitary, each of these really is a probability distribution, summing to 1.

Step 2: draw a whole strategy at random. For every party k and every possible input i, independently, draw an output $f_k(i)$ with probability $q_k(f_k(i) \mid i)$. The result is a *random deterministic strategy* for everybody: a random choice of the f of Section 10. With infinitely many inputs this is an infinite product of probability spaces. That is perfectly standard measure theory.

Step 3: notice what the column norm is. The squared length of the column of M for source value e works out to

$$\Sigma \text{ over } o \text{ of } \prod_k q_k(o_k \mid w_k(o)).$$

The term for a given o is exactly the probability that o is a fixed point of the random strategy. Summing over all o, with Tonelli's theorem to justify swapping sum and expectation, gives the **expected number of fixed points**.

Step 4: consistency finishes the job. For a process function, *every* deterministic strategy has exactly one fixed point. So the number of fixed points is the constant 1, and its expectation is 1. Every column has length 1. Nothing leaks. Together with the orthonormal rows, M is unitary.

Worked check. For w(a, b) = (0, a), the column norm is Σ over a, b of q_A(a | 0) · q_B(b | a). Sum over b first: each q_B(· | a) sums to 1. Then sum over a: q_A(· | 0) sums to 1. Total 1, as promised. For the swap w(a, b) = (b, a) with both parties copying (q(o | i) = 1 exactly when o = i), the column norm is the number of a, b with a = b, which is 2. That is the bootstrap of Section 26 again: "probability" doubled.

What the story shows. Back in the hotel, a leak would be a source state with column length less than 1, a guest who checks in and never fully arrives. The expected-fixed-points argument shows that the guest is always *exactly* accounted for. The reason is the reason for everything else in this document. Every way the agents could behave yields exactly one consistent history. A random behaviour is just an average over such ways, and an average of ones is one.

Philosophical gloss (interpretation). Consistency does double duty. Through the exclusivity lemma it forbids different histories from interfering. Through the expected count it forbids probability from being created or destroyed. The first is a statement about *distinguishability*, the second about *conservation*. Both come from the single clause "exactly one fixed point, for every intervention."

The same argument also gives a second, *dimension-free* proof in the finite case. It never uses the fact that the matrix is square.

43. What the referee has checked about infinity

- **Finite alphabets, second proof (machine-checked).** The averaging identity `PFUL.Column.sum_prod_eq_one` L-045 holds over any commutative ring. Unit columns are `PFUL.induced_colnorm` L-046. $M^\dagger M = 1$ from orthonormal rows plus unit columns, with no dimension count, is `PFUL.induced_conjTranspose_mul_of_colnorm` L-047.

- **Countable alphabets (machine-checked).** The expected-fixed-points identity is `PFUL.Countable.tsum_prod_eq_one` L-071, built with Mathlib's infinite product measure, and it allows inputs of any cardinality. Unit columns are `PFUL.Countable.induced_colnorm` L-070. Orthonormal rows are `PFUL.Countable.induced_rows` L-201, using the factorisation lemma `PFUL.Countable.hasSum_pi_prod` L-200. The assembly is `PFUL.Countable.induced_unitary_l2` L-002. It gives a linear isometric *equivalence* between the two ℓ^2 spaces whose matrix is M: a genuine unitary on infinite-dimensional Hilbert space. Its hypotheses are finitely many parties, countable alphabets and ancillas, and each V_k a unitary kernel. Standard axioms only.

Still open or not formalised.

- **Continuous alphabets (mostly settled, F.5).** Inputs in the real line or the circle. The closed loop is now defined for local operations that move points around (shifts, flips, squeezes, rearrangements, phase kicks), and for every local operation when the loop is causally ordered. In both cases it is proved unitary in the paper, for every loop, causal or not. The key lemma is machine-checked (`PFUL.Continuous`). The measure-theory step is now machine-checked too (Oct 23 2026): the loop map never squeezes or stretches volume, for any loop, causal or not (`PFUL.Continuous.tau_measurePreserving` L-060, proved for the real line, the circle and any similar group). The loop built from shifts plus phase kicks is checked to be unitary (`closedLoop_unitary` L-063). Round 11 (Oct 23 2026): the volume-change factor version is machine-checked for maps that are smooth except on a negligible set (`closedLoop_jacobian_unitary_ae` L-057), and the infinite-dimensional loops are shown to be automatically well-behaved bounded operators, for helper systems of any size (`isUnitaryKer_Vsb` L-081, `closedLoop_l2_unitary` L-170). For causally ordered loops, chopping the line into finer and finer cells gives the same answer in the limit (Proposition F.5.8, paper proof). Still open: a canonical definition for non-causal loops with general local operations, such as a Fourier transform.
- **The process-matrix (Choi) form in infinite dimensions (formalised, small gaps).** The Born-rule pairing with W converges to 1 for countable alphabets, for any local channel given by Kraus operators or by a normal completely positive trace-preserving map (`valid_born_l2` L-024, `valid_kraus_l2` L-157, `valid_normalCPTP_l2` L-160). The H.12 counterexample's own definitions are now tied to the general ones (`PFUL.Counterexample.isPF_iff` L-107, `valid_of_mem_convexHull` L-109). Left: purity of the counterexample with infinite-dimensional ancillas, and automatic continuity of positive trace-preserving maps (assumed as "normal").

Part VIII. Chance in loops: mixtures and purification

44. Classical processes: p(i | o)

Now let the environment be random. Instead of a function w, we have conditional probabilities

```
p(i | o) = the probability that the parties receive the inputs i, given that they send the outp
```

The "|" here is the ordinary probability bar, read "given". It is not part of a ket.

A **classical process** is such a p that is **logically consistent**. For every choice of local interventions, the total probability of all consistent histories must be exactly 1. (The full definition also covers randomised interventions, but deterministic ones are the heart of it.) For a deterministic p, which is just a w, this is the process-function condition again. Exactly one consistent history, with probability 1.

What it commits us to. Chance is put in the *environment*, not in the agents. And consistency is demanded only "on average". A random loop may be inconsistent in some of its deterministic pieces, as long as the probabilities always add up correctly. The coin-flip loop of Section 5 was exactly this.

45. The polytope: a map of all consistent loops

Collect all the numbers p(i | o) into one long list. Each classical process is then a point in a high-dimensional space. Logical consistency is a set of linear equations and inequalities (probabilities are non-negative, and certain sums equal 1). A region cut out by finitely many linear conditions like that is a **polytope**, a many-dimensional cousin of a polygon or a cut gemstone. This one is the **Baumeler–Wolf polytope**.

Three features of a polytope carry philosophical meaning.

- **Points inside** can be written as *mixtures*, that is, weighted averages, of other points. A mixture with weights p_j is the situation "with probability p_j, we are in process number j". Mixing is the geometry of *ignorance*. You know the menu of possibilities, but not which one obtains.
- **Vertices** (corners) are the points that are *not* mixtures of any other points. A vertex is a "pure option". There is no way of reading it as ignorance over something more definite inside the polytope.
- **Facets** are the flat outer walls. Each one is a linear inequality ("this weighted sum of probabilities is at most that one"), and the polytope is exactly the set of points satisfying all of them. Inequalities are how one *tests* membership. A facet violation is a certificate that a point is outside.

Every process function is a vertex: a deterministic point cannot be a mixture of other points. The question is whether there are other vertices.

- **Two binary parties.** The polytope has 12 vertices, and all of them are deterministic. They are the 12 process functions. Every two-party binary classical process is a mixture of process functions, so all its chance is ignorance about which deterministic consistent loop obtains.
- **Three binary parties.** The polytope has 710,760 vertices. Only 744 of them are deterministic, that is, process functions. All the rest are *random corners*: genuinely chancy consistent loops that are not mixtures of anything else (Baumeler and Wolf 2016).

Inside the big polytope sits a smaller one, the **process-function hull**, conv(PF). It is the set of all mixtures of the 744 process functions. For three binary parties, the random corners stick out beyond it.

46. Purification: randomness as ignorance of a hidden pure state

Can a chancy loop be realised in a reversible quantum world? The precise notion is **purifiability** (Araújo, Feix, Navascués and Brukner 2017, Definition 3).

A classical process W is *purifiable* if there is

- a unitary V, with a source and a sink, that is **pure**: unitary to unitary for every choice of local unitaries, as in Section 26; and
- a fixed **source state** $|\psi\rangle$ (ψ is Greek "psi"),

such that preparing the source in $|\psi\rangle$ and discarding the sink gives back exactly the probabilities of W.

A word on how the probabilities are read off (the basis convention). Each lab reads and writes its wire in one fixed basis, its *frame*. Unless another frame is named, that is the ordinary 0/1 basis, the

computational basis of Section 22, and "W is purifiable" means purifiable in that frame. The care is needed because one and the same pure machine can look like a mixture of process functions when its wires are read in one frame and not when they are read in another (note H.15.8: the computational frame gives a mixture, the Hadamard frame does not). Think of a coin whose face depends on which way it is turned to the light. So any claim about the classical shadow of a particular machine must say which frame it uses. The counterexample of Section 58h uses the computational frame; its Hadamards sit inside the machine.

Why the word "purification". In quantum theory a "pure" state is one of maximal knowledge, a single vector. A "mixed" state expresses uncertainty. Every mixed state can be seen as part of a larger pure state, with the uncertainty arising from not looking at the rest. To *purify* a random process is to exhibit it as a fragment of a deterministic, reversible, pure whole, with the randomness coming entirely from our lack of access to the source and the sink.

What it commits us to. That chance can be *explained away* as ignorance about a hidden but perfectly definite state of the world, as long as the world, once fully described, is reversible. Purifiability asks whether a given chancy loop admits such an explanation.

(The W here is the process written as a matrix, the "process matrix". For classical processes it is diagonal, with the numbers p(i | o) on the diagonal. Nothing below needs more than that.)

47. Mixtures of process functions are purifiable (proved)

Proposition (proved; note, C.1). Every mixture of process functions is purifiable.

Construction, in words. Suppose W is "process function w_j with probability p_j". Add to the source a *label register* that says which j. Build one big unitary that reads the label and runs the lift of w_j:

$U\ |o,\ e,\ j\rangle = |w_j(o) + e\rangle_I\ \ |o,\ j\rangle_F.$

Under local unitaries this acts as M_j on the j-th block, and each M_j is unitary by the main theorem. So the whole thing is pure. Now prepare the label in the superposition

$\Sigma_j\ \sqrt{p_j}\ |j\rangle.$

Why the square roots. Probabilities are squared amplitudes, so amplitude $\sqrt{p_j}$ gives probability p_j.

Why the sink must keep a copy of j. If the sink records which j, the different branches are perfectly distinguishable at the end. When we discard the sink, the cross terms between branches vanish, and what remains is exactly $\Sigma\ p_j\ W_{w_j} = W$. If the label were *not* handed to the sink, the branches would still interfere, and the result would no longer be the classical mixture.

Meaning. This is randomness as ignorance, in its purest form. There is a definite fact about which loop we are in. It sits coherently in the source, and its trace is kept in the sink. We just do not see it.

The same trick mixes *any* purifiable processes, so the purifiable classical processes form a convex set containing the whole process-function hull. In particular, every two-party binary classical process is purifiable.

48. The classical version settles itself (proved)

Before going quantum, ask the classical question. Call W **classically purifiable** if it is what you see, averaging over a random source value λ, of a classical reversible process. That is, a bijection that stays a bijection from source to sink under every deterministic local intervention.

Proposition (proved, sketch; note D.6). W is classically purifiable if and only if it is a mixture of process functions.

Why. Fix the source value λ. You get a deterministic loop w_λ. For the source-to-sink map to stay a bijection under every intervention, each intervention must have exactly one consistent run. With zero runs, that source value would have no image. With two, two runs would collide in the sink, by counting. So each w_λ is a process function, and W is the λ-average of them. The converse is the random-source construction of Section 47.

Meaning. In a *classical* reversible world, chance in a consistent loop is always ignorance about which deterministic consistent loop obtains. This is now a theorem.

So the big question narrows sharply:

> **Does quantum reversibility allow anything that classical reversibility does not, for classical loops?**

Part IX. The random corners

49. E_ex1, formally

Recall the coin-flip loop of Section 5. Let C be the *circular identity*, in which each party receives the previous party's output:

$$x_1 = a_3, \quad x_2 = a_1, \quad x_3 = a_2$$

(here x is an input and a an output). Let $\bar{C}$ be its bitwise negation: each party receives the opposite of the previous party's output. Then

$$E_ex1 = \tfrac{1}{2}\, C + \tfrac{1}{2}\, \bar{C}\ .$$

Facts (proved; note, Appendix C).

- E_ex1 is logically consistent. Section 5 gave the idea: C's surplus histories match $\bar{C}$'s missing ones.
- E_ex1 is a *vertex* of the three-party binary polytope (the active constraints have full rank, 64 of 64). So it is not a mixture of process functions, nor of anything else.

So E_ex1 is a consistent loop whose chance is not ignorance over consistent deterministic loops. It averages two *inconsistent* deterministic loops. In the geometry of the polytope it is a corner: there is nothing more definite inside the polytope for its chance to be ignorance about.

50. Why E_ex1 cannot be purified: isotropic subspaces, gently

Proposition (proved, exact certificate; note C.2). E_ex1 is not purifiable.

The proof uses an exact criterion due to Araújo et al. (their Theorem 4). Here is its shape, without the full machinery.

Step A: turn the question into linear algebra. A purification would supply a set of vectors, one for each "piece" of W. There are r of them, where r is the **rank** of W (for E_ex1, r = 16). Some constraints on these vectors are linear. They force the vectors to live in a certain subspace V_W, whose dimension the note computes exactly: 27 for E_ex1. Araújo et al. give a necessary condition: dim V_W ≥ r. Intuitively, there must be room for 16 independent vectors. Here 27 ≥ 16, so this test does not rule anything out.

Step B: the quadratic constraints. The remaining constraints are *quadratic*. Certain quadratic forms must vanish identically on the whole r-dimensional span of the vectors. A subspace on which a quadratic form vanishes identically is called **totally isotropic** for that form.

A toy picture. Take the form $q(x, y) = x^2 - y^2$ on the plane. It vanishes on the line $x = y$ and on the line $x = -y$. But there is no *plane* on which it vanishes, since the only plane available is the whole space, and $q(1, 0) = 1$. One plus direction and one minus direction allow at most a one-dimensional isotropic subspace. In general, a form with P positive directions, N negative directions and Z zero directions has totally isotropic subspaces of dimension at most $Z + \min(P, N)$. Each "null" direction has to pair a plus with a minus, and the zeros come for free.

Step C: the certificate. The note exhibits one explicit integer combination of the constraint forms. Its symmetric part, on the 27-dimensional space V_W, has exactly 13 positive directions, 13 negative and 1 zero. This was computed in exact rational arithmetic. So any totally isotropic subspace has dimension at most $1 + \min(13, 13) = 14$. But a purification needs a 16-dimensional one. Since $14 < 16$, no purification exists. ∎

What isotropy encodes, philosophically. The linear conditions (Step A) say a purification has *enough room*. The quadratic conditions (Step B) say the room is *shaped right*. They ask that the hidden vectors fit together with no leftover interference: whatever cross terms appear between them must cancel. The certificate is a counting argument. There are not enough balanced plus/minus directions to host sixteen mutually cancelling directions. A coin flip inside a loop is too tightly wound to be unwound into a reversible whole.

A numerical cross-check. A gradient search for purifications drives the residual to about 10^{-13} for Lugano, which is purifiable, but stalls at 79.7 for E_ex1. The search agrees with the certificate.

51. The other random corners

How were these corners found? Not by listing all 710,760 vertices of the three-party binary polytope. The note's code (code/stochastic/sample.py) solved 40,000 linear programs. Each one minimised a random linear objective, with independent Gaussian coefficients, over the Baumeler–Wolf polytope, that is, over all logically consistent $p(i \mid o)$. Each optimum is a vertex. Of the 40,000 vertices found, 39,354 were deterministic (process functions) and only 646 were non-deterministic. Yet only 744 of the 710,760 vertices are deterministic. So the method is far from uniform: random objectives land on corners in proportion to how "pointy" they are, and the process functions are evidently very pointy. **The 51 orbits are therefore only as representative as this sampling.** Indeed most random corners have certainly never been seen. A symmetry orbit has at most 384 members, so the 51 orbits contain at most $51 \times 384 = 19{,}584$ vertices, under 3% of the 710,016 non-deterministic ones.

Up to symmetry (swapping parties and relabelling inputs and outputs), the 646 non-deterministic samples fall into **51 orbits**. For all 51, the simple room test $\dim V_W \geq \operatorname{rank} W$ passes, so it decides nothing.

- **Proved (exact certificates):** orbits 7 and 50, the two orbits with $\dim V_W = 27$. One of them contains E_ex1.
- **Evidence only:** for the other 49 orbits ($\dim V_W$ from 82 to 256), the certificate method was inconclusive. A GPU search, with 64 random restarts per orbit, found no purification for any of them. Best residuals ranged from 12.97 to 59.35, far from zero, while the same code finds Lugano's purification and stalls on E_ex1 as expected. A local search can miss solutions, so this is not a proof.

Part X. The conjecture, the evidence, and the refutation

52. The conjecture

> **Conjecture (now refuted; see Section 58h).** A classical process is purifiable if and only if it is a mixture of process functions.

The "if" direction is proved (Section 47). The "only if" direction is false (Section 58h). Sections 52–58g are kept as they were written, while it was open. By Section 48, the conjecture is equivalent to:

> **A classical process that is quantum-purifiable is already classically purifiable.**

In words: for loops with classical statistics, quantum reversibility buys nothing that classical reversibility did not already give. If the conjecture holds, every reversible quantum realisation of a chancy classical loop is, at the level of its statistics, a coherent ignorance-mixture of deterministic consistent loops.

53. A story: two workshops

Two workshops build time loops to order. The first, the *clockmaker's*, uses only classical parts: gears, levers, a shuffled deck sealed in a drawer at the start, and a ledger at the end that records everything. Its machines are reversible: from the ledger and the final positions you could reconstruct the deck. The second, the *quantum workshop*, may use anything quantum mechanics allows: superposed gears, entangled drawers, interference. Its machines must also be reversible, in the quantum sense.

A customer orders a loop with given classical statistics: "under every strategy, these inputs with these probabilities".

- The clockmaker can build exactly the **mixtures of process functions**. The shuffled deck picks a deterministic consistent loop, and the ledger remembers which (Section 48, proved).
- The quantum workshop can build at least those (Section 47, proved). It provably *cannot* build the coin-flip loop E_ex1 (Section 50, proved).

The conjecture said the two order books are identical (Section 58h shows they are not). **Quantum reversibility buys loops nothing that classical reversibility does not.** If true, all the extra power of the quantum workshop, its superpositions and interference, is invisible in the classical statistics of a loop. Whatever it builds, the clockmaker could have built with a shuffled deck.

54. The hidden-label idea (Conjecture D.3)

The most natural strategy for proving the conjecture goes like this. Take any purification. Look inside its source. Hope to find a classical label λ, "which loop are we in?", such that for each value of λ the purification just runs one deterministic loop w_λ. Then W would be the λ-average of the w_λ. Each w_λ would have to be a process function, because the purification stays unitary on each label alone. And the conjecture would follow.

Stated precisely, the note's **Conjecture D.3** is: if V purifies a classical process, then there is an orthonormal basis $\{|\lambda\rangle\}$ of the source such that

$$V\,|\lambda, o\rangle = (\text{phase}) \cdot |w_\lambda(o)\rangle \otimes |g_\lambda(o)\rangle,$$

for functions w_λ (the loop for label λ) and g_λ, with g_λ one-to-one on outputs (the sink's record). The note shows that D.3 would imply the conjecture.

What D.3 encodes. It is a *hidden-variable* thesis. Underneath the quantum description there is a single classical fact, fixed in advance, about which deterministic loop is running. The fact is fixed independently of what the agents do.

55. The counterexample: contextual randomness (proved; note D.4)

A story first: the coin whose face is decided by how you look. A sealed token sits in a drawer. Alice, in the first lab, makes a choice. Then Bob, in the second lab, receives a bit. The whole thing behaves exactly as if a fair coin had been tossed in advance. Heads, Bob's bit is a copy of Alice's. Tails, Bob always receives 0. Naturally you assume the coin's face is written somewhere in the drawer, waiting.

It is not. If Alice chooses 0, the token is consulted with one question, and the face does not matter, since both "loops" give Bob a 0. If Alice chooses 1, the token is consulted with a *different*, incompatible question, and only then does a face appear, at random. There is no single way of reading the token that would serve as the coin for both of Alice's choices. The statistics are those of a coin tossed in advance. The mechanism is a coin whose face is decided by how you look.

Now the details.

D.3 is false. The counterexample is not even a time loop. It is an ordinary, causally ordered circuit.

The circuit. Alice has no input; she just chooses a bit oA. Then Bob receives a bit and sends one. The source holds two qubits, m and a, both starting in $|0\rangle$. The circuit:

1. If oA = 1, apply a Hadamard to m. If oA = 0, leave m alone.
2. Copy m into a, using a controlled-NOT, which flips a exactly when m is 1.
3. Send m to Bob as his input.
4. Route oA, Bob's output oB, and a to the sink.

This is a circuit of unitaries in a fixed causal order, so it is a valid unitary process.

What the parties see.

- If oA = 0: m stays $|0\rangle$, so Bob always receives 0.
- If oA = 1: m becomes $(|0\rangle + |1\rangle)/\sqrt{2}$, the copy into a makes the branches distinguishable, and Bob receives 0 or 1 with probability ½ each.

Is it a mixture of process functions? Yes. Let λ be a fair coin and w_λ(oA) = λ·oA. With λ = 0, Bob always gets 0. With λ = 1, Bob gets Alice's bit. Average over λ: if oA = 0 Bob gets 0; if oA = 1, Bob gets 0 or 1 equally. That matches. The conjecture is not threatened.

But there is no hidden label in the source. Where, in the source, does the coin λ live? If Alice sends 0, the "coin" is read off the source in the ordinary 0/1 basis (the **Z basis**). If Alice sends 1, it is read off in the Hadamard-rotated basis (the **X basis**). These two bases are incompatible. Formally, for each input and output the note forms the operator

```
G_{i,o} = V_o† (Π_i ⊗ 1_F) V_o
```

on the source. Here V_o is the purification with the parties' outputs fixed at o, and Π_i projects onto "the parties received i". This operator asks: "which source states lead to input i, given outputs o?" D.3 holds exactly when all the G's are projectors that commute with one another, since their common eigenbasis

would be the label basis. Here the G's are Z-projectors when oA = 0 and X-projectors when oA = 1. They do not commute (commutator norm ½). So no single basis of the source works. Restricting attention to the actual source state |00⟩ does not help either.

Philosophical meaning. The randomness here *is* ignorance about which deterministic loop obtains, since W is a mixture. But the "which loop" variable is **contextual**. Which question the source answers depends on what Alice did. There is no fact sitting in the source, fixed in advance and independent of the agents' choices, that says "λ = 0" or "λ = 1". The ignorance reading survives as a claim about the *statistics* W. It fails as a claim about a single hidden label. (The resemblance to quantum contextuality in the Kochen–Specker sense is suggestive. That is an interpretive remark, not a result of the note.)

Consequence for proofs. No argument that tries to extract one classical label from the source can work. A proof of the conjecture must show *directly* that W lies in the process-function hull, for example by showing that purifiability implies every facet inequality of that hull.

56. Two things that cannot be the criterion

The note also proves two negative results. They say where the answer is *not*.

Quantum-controlled interventions do not help (proved; note D.5). One might hope that E_ex1 fails some weaker quantum test, such as letting the parties apply instruments coherently controlled by shared entangled equipment, and checking that probabilities still add to 1. They always do, for *every* classical process. Because W is classical (diagonal), it strips away all quantum coherence at the parties' doors. What remains is a classical joint behaviour that cannot be used to signal. Every such behaviour is an *affine* combination of local deterministic ones (signed weights allowed). The normalisation is linear and equals 1 on each deterministic strategy, so it equals 1 for all of them. So only the *unitarity* (reversibility) requirement can separate E_ex1 from mixtures. Consistency under quantum interventions is not enough.

The actual source state is not enough (proved; notes D.2 and D.8). Any test that looks only at what happens when the source is in its actual state |ψ⟩ is passed by every classical process, including E_ex1. This holds even when the parties use arbitrary local unitaries and superpositions of strategies. So any proof must use how the purification would act on source states *orthogonal* to |ψ⟩, the ones that never actually occur. In Araújo et al.'s terms, that is exactly the non-convex isotropic-subspace condition of Section 50.

Philosophical gloss (interpretation). What rules out E_ex1 is not anything that happens. It is the *modal* structure of the reversible world: how it would have behaved had it started differently. Reversibility is a constraint on counterfactuals, not only on actual histories.

A related structural fact (**proved; note D.1**): in any purification, the sink keeps a perfect record of the pair (outputs, inputs). The vectors it holds for different pairs are orthogonal, with squared lengths p(i | o). Reversibility forces a complete record, as Section 18 anticipated.

57. The facet evidence

If the conjecture is true, purifiability must imply every facet inequality of the process-function hull. So it helps to know those inequalities.

Computed exactly (note D.7). The 744 three-party binary process functions span a 37-dimensional space. Their hull has exactly **29,344 facets**, falling into **93 orbits** under the 384 symmetries (permuting

parties and relabelling inputs and outputs). This was computed with the normaliz software on a rented 32-core machine in about 1.9 hours.

One inequality separates everything sampled. Facet orbit 82 contains the inequality

$$p(000 \mid 000) \le p(000 \mid 001) + p(010 \mid 001) + p(000 \mid 100) + p(010 \mid 110) + p(011 \mid 111),$$

where each string lists the three parties' bits, inputs before the bar and outputs after it.

Reading it. The left side is the probability that, when everyone sends 0, everyone receives 0. The inequality says that this cannot be likely unless certain neighbouring events are at least as likely in total. For a deterministic process function it says: if w(000) = 000, then w(001) is 000 or 010, or w(100) = 000, or w(110) = 010, or w(111) = 011. It is a piece of combinatorial bookkeeping that every consistent deterministic loop obeys, and that mixtures therefore obey too. E_ex1 violates it: the left side is ½ and the right side is 0. Here is why. When everyone sends 0, the copier branch delivers 000, and that branch has probability ½. In each of the four situations on the right, neither the copier nor the negator delivers the listed inputs. For instance, when the outputs are 111 the copier delivers 111 and the negator 000, never 011.

Members of facet orbit 82 are violated by E_ex1 and by **all 51** sampled non-deterministic vertex orbits. So a single class of inequalities separates every sampled random corner from the hull. **Deriving it from purifiability is the natural next target. It has not been done.**

58. Walking from Lugano to E_ex1

The note also tests the boundary numerically (evidence; note D.8). Walk along the straight line from Lugano (purifiable) toward E_ex1 (not):

$$x_t = (1 - t) \cdot \text{Lugano} + t \cdot \text{E_ex1}.$$

The process-function hull is left almost at once, at about $t \approx 2 \times 10^{-7}$, because Lugano is a corner and this direction points straight out. The purification search finds nothing at t = 0.02 or t = 0.2, with best residuals 52.2 and 60.1. The same code reaches about 10^{-13} on Lugano itself. So along this line, purifiability appears to stop exactly where the hull does. That is consistent with the conjecture, but it is not proof.

58a. Two parties: there is nothing to purify away (proved; note Appendix G)

A story first: the two-room house. Two people live in a house with two rooms. Each can leave a note for the other, but the house has a rule: no note may ever create a paradox, whatever either of them decides to write. With only two rooms, it turns out the rule is very strict. In every run, one of the two must in effect read before the other writes, perhaps with a coin deciding who goes first. A genuine loop, where each reads what the other has not yet written, needs at least three rooms.

The statement. Write a two-party classical process as p(a, b | x, y). Here a is what Alice receives (her *input*), x is what she sends (her *output*), and b, y are the same for Bob. "Valid" means two things. Every probability is non-negative. And for every pair of strategies, Alice replying x = f(a) and Bob replying y = g(b) with *any* functions f and g, the probabilities of all consistent histories add up to exactly 1. The note proves (**Theorem G.1**) that every such p, with alphabets of any finite size, is a mixture of *causally ordered* process functions. In each of these, either Alice's input is fixed and Bob's is a function of what Alice sent, or the other way round.

How the proof goes, in words. Split p into pieces according to which of the four variables a, b, x, y each piece really depends on. Validity kills every piece that would let Alice's choice and Bob's choice interact inside the same input. What survives is a constant part, an "Alice-to-Bob" part and a "Bob-to-Alice" part. The constant part is large enough to cover the negative dips of the other two, so p splits into two non-negative halves: one where Alice goes first and one where Bob goes first. Each half is a mixture of deterministic one-way processes. This is the classical shadow of an argument by Oreshkov, Costa and Brukner (2012).

Consequence. For two parties, every valid classical process is a mixture of process functions. By Section 47 it is therefore purifiable. Both sides of the conjecture are true, so the conjecture holds, but trivially. The interesting question starts at three parties, which is where E_ex1 lives.

58b. Three parties with a clear order (proved; note Appendix H)

With three parties, Alice, Bob and Carol, there can be genuine loops (Lugano). But many three-party processes still have an order hidden in them. The note proves that whenever they do, the two-party result does the rest.

- **Someone is last (Theorem H.1).** Suppose Carol's output z changes nobody's input; she only listens. Then forget Carol for a moment. Alice and Bob form a valid two-party process, which by 58a is a mixture of ordered process functions. Carol's input is then just some random function of what Alice and Bob did. Every piece is a deterministic consistent loop, so the whole is a mixture of process functions.
- **Someone is first, properly (Theorem H.2).** Suppose Alice's input a does not depend on anyone's output (she is *marginally first*). Suppose also that, once a and Alice's output x are fixed, what remains between Bob and Carol is itself a valid two-party process (the *conditionals are valid*). Then each remainder is a mixture of ordered two-party process functions (58a). Gluing Alice in front gives a mixture of three-party process functions.

A process that is, in any run, a mixture of such "someone goes first, and the rest is consistent" pieces is called *causally separable*. So: **every causally separable three-party classical process is a mixture of process functions, and the conjecture holds for it** (trivially again: both sides are true).

Where the real question is. Lugano is not causally separable: it violates a causal inequality (Baumeler, Feix and Wolf, 2014), and each party's input depends on both others. So the open part of the conjecture lives exactly among the loops with no hidden order. That region contains Lugano-type process functions, their mixtures, and every candidate counterexample.

A subtlety that matters. "Marginally first" alone, without valid conditionals, is not enough. E_ex1, the coin-flip loop, gives each party an input that looks like a fair coin whatever anyone does. So every party is marginally first. Yet E_ex1 is not a mixture of process functions. Once you fix, say, Alice's input, the remaining Bob-Carol process can have no consistent history, or two.

58c. A tempting shortcut that fails (proved; note H.5)

Since E_ex1 and the hard orbits 7, 8 and 50 are all marginally first in some party, a tempting plan was: show that *purifiability* forces valid conditionals for a marginally-first party. Then 58b would finish them off.

A story: the referee's coin. A referee flips two coins, β and γ, and hands them to Bob and Carol as their inputs $b = \beta$ and $c = \gamma$. Bob and Carol then each send a bit, y and z. Alice, last of all, is told $a = \beta \oplus \gamma \oplus$

$y \oplus z$, where $\oplus$ means addition mod 2 ("the parity"). In formula form,

$p(a, b, c \mid x, y, z) = \frac{1}{4}$ if $a = b \oplus c \oplus y \oplus z$, and 0 otherwise.

- This is a mixture of four deterministic consistent loops, one for each value of the coins (β, γ). In each, Bob and Carol go first and Alice last. So it is purifiable (Section 47).
- Looking only at the statistics, Alice's input is a fair coin whatever anyone does. She is *marginally first*, even though in the machine she is actually last.
- Fix Alice's input to $a = 0$ and look at what remains for Bob and Carol: (b, c) uniform among the pairs with $b \oplus c \oplus y \oplus z = 0$. If Bob and Carol simply copy their inputs to their outputs ($y = b$, $z = c$), the "probabilities" of consistent histories add up to 2. For $a = 1$ they add up to 0. The conditional is not a valid process.

So the shortcut is false, already for a classically purifiable process. Every claim above is checked in exact fraction arithmetic (code/stochastic/structural/margfirst_cex.py). **Why the shortcut fails:** looking at a's statistics cannot tell you whether a is *computed first* in the machine. Fixing a is then post-selection, keeping only the runs where something downstream turned out a certain way, and post-selection does not respect reversibility. A version that might still be true would have to talk about the purifying unitary itself, not about W. That version is true; see 58d.

Philosophical gloss (interpretation). "Comes first" is a fact about mechanism, not about statistics. A variable can look exactly like a fresh coin flip while being the last thing the machine computes. This is the same lesson as Section 55, from the other side. There, the statistics looked like a coin tossed in advance, but no coin was in the drawer. Here, a party looks like it acts first and in fact acts last.

58d. The shortcut, repaired inside the machine (proved; note H.6)

58c showed that statistics cannot tell you who goes first. The machine can. Say Alice is *first in the machine* when her input wire is filled from the source alone, before anyone's output, her own included, can touch it. In the machine's own terms, the unitary splits into two stages. First a step that prepares Alice's input from the source. Then everything else.

A story: the sealed envelope. Before the game starts, the referee seals Alice's instruction in an envelope, using only the coins in her own pocket. Nothing Bob or Carol later does can reach back into that envelope. Opening it early is then just reading a note that was already written. It is not the post-selection of 58c, where we kept only the runs in which a later computation came out a certain way. So we may open the envelope first, write down a and Alice's reply x, and look at what is left between Bob and Carol. What is left is again a reversible machine of the same kind: a pure two-party process. Its statistics are exactly the conditional $p(\ldots)/p(a)$. So the conditionals are valid, 58b applies, and the whole process is a mixture of process functions.

Checks on the two running examples (code/stochastic/structural/opfirst.py): - In the referee's-coin machine of 58c, Alice is *not* first in the machine; she is computed last, and her conditionals are invalid. Bob and Carol *are* first in the machine, and their conditionals are valid. - In the Lugano machine nobody is first, and nobody can be. Lugano is not even marginally first in any party, so no machine for it puts anyone first.

What this leaves. A purifiable process outside the mixtures, if one exists, must have only machines in which *every* party's input is reached by somebody's output: a genuine loop, like Lugano's. Each of E_ex1 and the open orbits fails 58b's test in every party, so any machine for one of them would have to be loop-shaped all the way through. The conjecture now lives entirely there. The lemma is short, and

close to known results on causal structure in unitary processes (Barrett, Lorenz and Oreshkov 2021). We did not find it stated in this form.

Philosophical gloss (interpretation). 58c said "first" is a fact about mechanism. 58d says that mechanism-level "first" is enough. Wherever the machine has a genuine first mover, chance inside the loop really is ignorance of which deterministic loop is running. The open question, whether chance can be basic inside a loop, is a question only about machines with no first mover at all.

58e. Machines with no first mover (evidence; note H.7)

58d leaves one place where a counterexample could hide: machines in which *nobody* is first. In such a machine, every party's input is touched by somebody's output. Here "the machine" means a purification: a reversible (unitary) device V that takes a source state and the parties' outputs, and returns the parties' inputs plus a sink record. We went looking for such machines with three parties sending and receiving single bits. For each one we asked whether the classical statistics it produces, p(i|o), can fall outside the mixtures of process functions (the "hull"). The answer from this search was **no, every time**. The search was small and local, so this is evidence, not proof.

Each machine found was given the same checks: - *Is it pure?* Close each party's slot with random local unitaries and check that the result is still unitary. - *Is anyone first?* For each party k, does any output reach k's input? - *Who influences whom?* The full influence graph. - *Is it in the hull?* A linear program measures the distance from p to the hull of the 744 process functions. - *How much Lugano does it need?* The smallest weight that any decomposition of p must put on Lugano-type process functions. Weight 0 means p is an ordinary mixture of *ordered* process functions.

Four kinds of machine turned up. They have informal names here; the technical note gives the formal descriptions.

The Ring (single-function lifts). Lift one process function w as in Section 1. Party k's input is then w_k(o) plus a source bit, so k is first in the machine exactly when w_k ignores everyone's outputs. Of the 744 binary process functions, exactly 64 have no such constant part. They form a single family under relabelling: Lugano's orbit, in which each party's input is set by the other two parties' outputs. So, for lifts of a single function, "no first mover" means exactly "Lugano-type". Adding phases on the source alone or the sink alone keeps the machine pure. Phases that mix outputs and source destroy purity. Either way, the statistics are just w itself.

The Chorus (coherent blend of three causal orders; proved). Put three ordered process functions in coherent superposition, as in Section 47: one with Alice first, one with Bob first, one with Carol first. Inside this machine no party is first and no party is last. Yet the classical statistics are an ordinary mixture of ordered process functions.

A story: the ledger with no clock. Three clerks share an office with no clock. Ask any of them who signed the ledger first, and each will truthfully say that someone else's entry was already there. Yet each morning the ledger reads, page by page, as one of three tidy orders. The disorder lived inside the room. The record is ordinary.

The Chorus refutes a tempting guess: "every machine with no first mover is Lugano in disguise." That guess is false.

The Switchboard (a new family). The machine reads all three outputs o and picks one 8×8 unitary C_o for that joint choice. It applies C_o to a source register s, which produces the three input bits e. It also writes o, and e XOR'ed with a second source register t, into the sink:

```
V|o, s, t> = sum_e C_o[e, s] |e> |o> |e xor t>.
```

Here |...> is a basis state, and the sum runs over the eight possible input strings e. The machine is always reversible, and the sink always keeps a copy of (outputs, inputs), so the statistics are classical: p(i|o) = |C_o[i, 0]|². Lugano is the special case where each C_o just flips bits according to w(o).

A story: the switchboard. Every caller's choice rewires the board before any call connects. Nothing is ever lost, because each board is unitary. The question is whether the boards for different choices agree well enough for the whole exchange to stay coherent. **Lemma H.7.1 (proved)** says exactly how much agreement is needed when the parties use no ancillas: a list of linear conditions on the products C_o C_o'†, one for each pair of different choices. With ancillas these conditions are still necessary, and Section 58f shows they are also enough. Whether the agreement forces an ordered story is open. It is a bet, not a proof.

The Shadows (three Switchboard machines found near Lugano). Starting the search close to Lugano produced three new pure machines (lug_21 to lug_23). Their C_o are generic unitaries, not relabelled permutations. Every party influences every other one. Their statistics need Lugano weight 0.87 to 0.93, so they are genuinely not causally separable, yet they lie inside the hull. One machine from a random start (rand_15) also has every party influencing every other, but its statistics need Lugano weight 0: like the Chorus, it is disordered inside and ordinary on the record.

The numbers (evidence). - *What was run:* 11 random starts, 3 starts near Lugano, then polishing to high precision. That gave 13 pure machines in total. One random start never became pure. One was polished only to a residual of 1.6e-6. - *Who is first or last:* 9 of the 10 pure random-start machines have a party that is first or last inside the machine, so 58d (first) or 58b (last) already covers them. - *Hull:* every pure machine's statistics lie in the hull, at distance 0. - *Pushing for a facet violation:* runs that also tried to push the statistics across a hull facet reached violations near the maximum, but only at purity residuals of 0.03 to 0.05. None was a pure machine.

What this leaves open. The conjecture for machines with neither a first nor a last mover. The Switchboard is one structured slice of them, and the Lemma is a foothold there. A general machine can have a bigger source and need not have this form.

Philosophical gloss (interpretation). The Chorus and the Shadows pull apart two things that are easy to run together: disorder *inside* the machine, and disorder *in the record*. A machine can be thoroughly cyclic inside while its record is an ordinary mixture of ordered stories. So far, every reversible loop we have found leaves a record that is a mixture of consistent deterministic loops.

58f. What the Switchboard's rules really say (proved in part; note H.8, C.6, Appendix I)

Section 58e left the Switchboard with a list of linear rules: the conditions under which its eight boards agree well enough to stay coherent. We now have a cleaner reading of those rules, and they turn out to be all the rules there are.

The rules in plain words (proved). Compare the boards for two different choices of outputs. The "difference" between them is the unitary K = C_o C_o'†. Like any operation on three bits, K can be written as a sum of simple pieces, each of which flips and/or phase-flips some of the three bits. (These pieces are the *Pauli strings*: X flips a bit, Z flips its sign, and a string says what happens to each of the three bits.) The rule is this: every piece of K must leave alone at least one of the parties whose output changed. For a process function, K is a single piece, and the rule becomes the exclusivity lemma of Section 1: two different output strings must agree at some party whose input they do not change. So the

Switchboard's rule is exclusivity with a twist. A quantum board may be a *sum* of pieces that each leave a different party alone, where a process function must pick one. A computer check in exact fractions confirms that the old list and the new reading say the same thing.

Ancillas do not matter (proved). Section 58e said that private scratch qubits (ancillas) might impose extra conditions. They do not: in the Switchboard, purity with ancillas and purity without them are the same condition. So the 13 pure machines of 58e are exactly pure, not merely pure in the tests.

Who is first, who is last (proved). The rule also tells us, by inspection, who influences whom. A party is first exactly when no board difference ever touches its bit. A party is last exactly when changing its output only multiplies the board by a phase.

The easy case is settled (proved). If all the boards are just relabellings with phases (a common basis that each board only shuffles), then for *every* source state the statistics are a mixture of process functions. This covers the Ring of 58e. The hard case is boards that genuinely superpose, like a board that half-flips Alice's bit and half-flips Bob's at once. The Shadows are of this kind, and their statistics are still in the hull.

How hard is it to cheat? (evidence). We asked the computer to push a Switchboard's statistics across the most important wall of the hull (facet 82) by a set amount v, while staying as close to pure as it could. In 108 tries, the "impurity" R it had to accept was always at least about $0.31 \cdot v^2$. In words: to get across the wall by v, you must be impure by roughly $0.56 \cdot v$ (measuring impurity as $\sqrt{R}$). Nothing found came closer. *(Superseded: Section 58h exhibits an exact machine that crosses a hull wall at zero impurity; the toll was an artefact of the search.)* If some machine could cross the wall at zero cost, the search would be closing in on a true counterexample, and it never did. This is evidence, not proof.

A story: the toll on the wall. The hull is a walled town. Every honest machine lives inside it. A traveller can always step up to the wall, and can even climb over, but each metre over the wall costs a fixed toll in impurity, and the toll never drops to zero. Whether the toll is a law of the land or only the price at the gates we have tried is the open question.

Close to a mixture, close to liftable (proved, but easy). If a process is close to a mixture of process functions, it is just as close to a liftable one, because the mixture itself lifts. The reverse, "close to liftable implies close to a mixture", is *exactly* the conjecture in disguise, so it cannot be proved separately. The meaningful version is the toll above: nearly pure machines give statistics nearly inside the hull.

A dead end, checked before spending (evidence). For orbit 8, a stronger kind of certificate (degree 4) was proposed: find a large slice of the space that contains no "forbidden" direction. Before renting any computer, we tested whether such a slice can exist at random. It cannot: random slices of the proposed size, and even smaller ones, all contain forbidden directions. So that certificate cannot be built that way, and nothing was spent.

What is still open. Whether the Switchboard's rule forces its statistics into the hull, and whether every machine with no first mover can be put in Switchboard form. Both are stated as conjectures in the technical note (H.9).

58g. Boards that blur two wirings (evidence; note H.10, H.11)

The hard Switchboards are the ones whose boards *blur* two flip patterns at once: partly "flip Alice's bit", partly "flip Bob's". We tried three ways to tame them.

Shuffling does not help (proved). One hope was to average the machine over harmless relabellings

until the boards stop blurring. The only relabellings that leave all the statistics alone are phase changes on the input strings, and those leave the blur exactly as it was. So any proof has to face the blur directly.

The cheapest kind of proof cannot work (proved). A "degree-2" argument, which treats each board's entries only through averages of their products, cannot tell a real machine from a fake one that never relates different boards at all. That fake can produce any statistics whatever. So a proof has to be at least degree 4 in the board entries.

A whole family, checked exactly, then tested (evidence). The simplest blurring boards mix two process functions with a fixed angle: "mostly w1, a little of w2", coherently. The computer listed *every* way to choose w1 and w2 so that the Switchboard's rule holds. (Along the way it confirmed that, for single bits, the exclusivity rule singles out exactly the 744 process functions.) There are 39,512 such pairs, and 34,304 of them genuinely blur. For every one of their 237 symmetry types, at seven angles, and against *all* 29,344 walls of the hull, the statistics stayed inside the hull for every source state. The worst value was -0.00000000000002, which is zero up to rounding. This is evidence, not proof. No counterexample.

Is every no-first-mover machine a Switchboard? (partly proved). Any machine can be given an extra "carbon copy" of the inputs in its sink without spoiling its purity. A slightly bigger Switchboard, with extra sink space, obeys the same rule, piece by piece. What is missing is one fact: that the machine's sink can always be made to remember the parties' outputs, whatever the source holds, and not just for the source state actually used. If that fact holds, every no-first-mover machine is a Switchboard, and the conjecture comes down to the Switchboard rule. We could neither prove that fact nor break it.

A story: the two-tone bell. A bell-founder casts bells that ring two notes at once. Each note on its own is a tune the town knows. The worry is that the two notes, ringing together, could beat into a tune no one has heard. Every bell of the simplest two-note kind has now been rung against every wall of the town, and none has carried a new tune out. Whether some three- or four-note bell could do it is still unknown.

58h. The bet is lost: a reversible loop with genuine chance (proved; note H.12, H.10.6)

First, a win (proved). The two-tone bells of 58g are now settled: *every* two-pattern Switchboard gives statistics that are a mixture of process functions, for every angle and every source state. The proof is a bookkeeping trick. For each board, the machine only ever moves probability around inside small blocks of input strings, and it moves the total in each block nowhere. So one can "ship" the source's probabilities to the board's probabilities block by block. Each way of shipping, combined over the different boards, turns out to be a process function. A computer checked, exactly, that this works for all 39,512 two-pattern boards.

Then a loss (proved). Allowing all eight flip patterns at once changes the picture. The computer found a Switchboard whose eight boards are built only from the phases 1, i, -1, $-\mathrm{i}$ in the Hadamard basis. Every one of its rules checks out exactly, in exact arithmetic with no rounding. For one simple source state, its statistics are a *valid* classical process: every intervention gives total probability exactly 1. Yet that process lies **outside** the hull of process functions. There is a linear inequality that every process function satisfies, and this process breaks it, by exactly $3\sqrt{2}/16 - 1/4$ (about 0.015, for the inequality's stated normalisation). Measured as a distance, the counterexample sits a definite way outside. Adding up the absolute differences over all 64 entries of p, a linear program finds the distance to the hull to be about **0.0607**. That figure is *computed* (in floating point), not certified. What Lean certifies is smaller and safer: the separating inequality guarantees a margin of at least about **0.0076** on the same measure, one eighth of the computed value. Lean does not certify the 0.0607, the count 744, or that the inequality

is a facet. In the note this inequality is a facet (a wall) of the hull, facet 1741 of the complete facet list (note D.7, computed with normaliz). The Lean proof below establishes the inequality on every process function, but not that it is a facet. So the conjecture this note has been chasing is **false**: a reversible quantum machine can produce classical statistics in a time loop that are *not* a disguised mixture of consistent deterministic loops.

Why the earlier search missed it. The big search over all boards kept settling into local valleys (Section 58f's "toll" was one of them). Restricting to boards built from flips alone made the landscape simple enough that exact solutions appeared at once. The toll was the price at the gates we happened to try, not a law of the land.

How sure are we? Three separate checks now agree. 1. *The note's own proof.* The facts about the classical statistics (valid, outside the hull) are exact. Purity follows from the switchboard lemma and its two companions (H.7.1, H.8.1, H.8.2), proved on paper, together with an exact check of the rule. 2. *An independent referee.* Someone else wrote a fresh verification program working only from the note's description of the machine. It checks purity directly, without using those lemmas. It recounts the 744 process functions from scratch and finds its own wall that the process crosses. It agrees on every point (note H.12.2). 3. *A proof assistant.* The whole statement, "there is a valid, purifiable classical process outside the hull of process functions", has been proved in Lean (note H.12.3).

What "checked by a proof assistant" means here. Lean is a program that accepts a proof only if every step follows from the rules of logic and from earlier proved facts. It does not trust the author or take a step on faith, and it is the incorruptible referee of Part VI. Its verdict is only as good as the definitions it was given. The first proof used definitions of "pure", "valid" and "process function" written for it in a small self-contained file. A careful reader should still check that they mean what this guide says. The two limits stated in earlier versions are now closed in Lean. The theorem is restated with the standard process-matrix definitions (`purification_conjecture_false_choi` L-092), and the scratch equipment may be of any size (note H.19, H.20; notes 87–88 and 96 here). In one sentence: it is machine-checked in Lean 4 with Mathlib, using only the standard axioms and definitions that match this guide. The proof works through a separating inequality proved on every process function. That the inequality is a facet of the hull is not formalised. The result has not been peer reviewed.

What changes. The "if" half stands: every mixture of process functions is purifiable. Two parties, and three parties with an order or with a first mover, still give nothing new. The new statement is: *quantum reversibility is strictly more permissive than classical reversibility for classical loops, already with three parties and one bit each.* Classically (D.6), reversibility does force the hull.

A story: the three-note bell. The founder finally cast a bell with eight notes. Each pair of notes was in tune with the town's rules, and every rule held to the last digit. When it rang, it played a tune that no deterministic loop could play, yet a tune no one could catch in a contradiction. Chance, it seems, can live inside a consistent loop, provided the loop is quantum.

Where the machine comes from (note H.21.2). It is not exotic. It is a frame-rotated time reverse of the $\sqrt{\text{X}}$-Lugano process: the standard purification of the Lugano loop with Bob's bit flip replaced by a square root of a flip ($\sqrt{\text{X}}$), run backwards and read in rotated frames. That process is a member of a known family of Lugano circuits, but it is not a purification of a process function, which fits the note's earlier finding that the counterexample is not a time-reversed purification of a process function.

Philosophical gloss (interpretation). Consistent time loops are not just mixtures of deterministic consistent worlds. Section 64 asked whether chance inside a loop can be fundamental, or only ignorance about which deterministic loop obtains. For classical reversible machines the answer is still "only

ignorance". For quantum reversible machines it is now "not always".

Part XI. The honest map of dead ends

59. A story: the cartographer's other map

Explorers publish the route that reached the summit. The best of them also publish a second map, of the gullies that ended in cliffs, the passes that were snowed in, and the bridges that looked sound and were not. That second map is less glamorous, but it is the one the next party needs most.

The "only if" half of the conjecture, open until Section 58h, had such a map. Here it is, route by route. Every entry is either **proved** to be a dead end, or **evidence** that it stalls.

60. The routes that failed

1. **The hidden-label route. Dead end, proved (D.4).** Find a classical "which loop" variable in the source. The causal counterexample of Section 55 shows that no such variable need exist, even when the conclusion is true.
2. **The weaker quantum test. Dead end, proved (D.5).** Demand only that probabilities be normalised under quantum-controlled interventions. Every classical process passes, including E_ex1 (Section 56).
3. **The actual source state. Dead end, proved (D.2, D.8).** Look only at what the purification does to the source state that is really prepared. Every classical process passes there too, so no certificate built from that data alone can rule out a purification (Section 56).
4. **Single-form certificates. Partial success, then a stall.** Combine the constraint forms into one and count its plus and minus directions (Section 50). This *works* for the two smallest orbits, giving E_ex1's exact certificate. For the 49 larger orbits it was inconclusive.
5. **Block forms and semidefinite relaxations. Stalled.** Matrix-valued forms and semidefinite programming generalise the counting test. The note's Section 9 lists them, with single-form inertia, as the attempts that "stalled for the larger orbits".
6. **Symmetry and representation theory. Negative result (Appendix E).** The 384 symmetries (permuting parties, relabelling inputs and outputs) could shrink the problem, so the note computed each process's *stabiliser*, the symmetries that leave it unchanged. E_ex1 and orbit 50 have 48, orbit 7 has 16, Lugano 6, and 34 of the sampled orbits have only the trivial one. Symmetry helps most exactly where certificates already exist, and not at all for most open orbits. Worse, the natural convex relaxation that symmetry would have sped up, a moment relaxation over pairs of copies of the projector, turned out to be *feasible* for E_ex1 at $r = 16$, with strict margins. It cannot even detect the one case we know is non-purifiable. It is strictly weaker than the plain counting test, so symmetry-reducing it is pointless. Stronger convex certificates would need higher moment levels, or would have to encode rank.
7. **Local numerical search. Evidence, not proof.** Gradient searches find Lugano's purification and stall on every random corner tried (Sections 51 and 58). A local search can always miss a solution.

8. **Algebraic geometry. Assessed: infeasible by size** (code/stochastic/alggeom/SIZE_ESTIMATES.md). Why not just solve the equations directly? Purifiability asks for a 16-dimensional subspace on which every constraint form vanishes, which is a system of polynomial equations. The forms are sesquilinear, so the system lives over the real numbers. Written out in coordinates, E_ex1 has 352 real unknowns and 23,552 real quadratic equations. Orbit 8 has about 2,112 unknowns and 100,352 equations. Homotopy continuation, the standard numerical method, would have to track about 2^{352} solution paths for E_ex1 and 2^{2112} for orbit 8. And certifying that *no* solution exists is exactly the hard case for such methods. Gröbner bases, the exact method, are far out of reach: practical limits are tens of variables, not thousands. Symmetry does not reduce the number of unknowns, because the subspace sought need not itself be symmetric. Small sub-problems are feasible but uninformative: isotropic subspaces exist for small r, and non-existence only starts near r = 15–16, where the full size applies. So the obstruction will not yield to brute-force elimination. It needs a structural argument, or a stronger convex hierarchy (route 6).
9. **Rank (determinant) bound for orbit 8. Fails, numerically (note H, route B).** If u is in a subspace that makes every form vanish, the whole subspace sits inside the vectors "orthogonal" to u under all forms. Counting those gives a dimension bound. But vectors of the isotropic set exist for which the bound allows about 28–32 dimensions, more than the 16 needed. So one vector is not enough.
10. **Form certificates for orbits 8, 10, 20. Proved too weak (note C.5).** Every linear or block combination of the forms tried stalls. Adding the normalisation provably adds nothing. Numerically, subspaces making all forms vanish exist up to dimension about 10, short of 16, but the gap is non-convex and invisible to such certificates. The earlier estimate that about half of the open orbits could be certified this way is withdrawn.
11. **Marginally first implies valid conditionals. Dead end, proved (H.5; Section 58c).** (The machine-level version is true: Section 58d.)
12. **Counterexample hunt from the quantum side. No hit (note H, route C).** Searching directly over unitaries for a purifiable process outside the hull found none. Starting from Lugano, every step outside the hull cost purity.

61. Why the dead ends cluster

The map has a shape, and the shape is the lesson.

- **The easy information is useless.** Everything readable from the actual source state is satisfied by every classical process (routes 2 and 3). Any proof must use how the purification would treat source states orthogonal to the one actually prepared.
- **That information is non-convex.** Demanding that a whole 16-dimensional (or larger) subspace make many quadratic forms vanish at once is not a convex condition. Convex questions can usually be settled by duality: when no solution exists, there is a checkable certificate saying so. Non-convex questions offer no such guarantee. Every convex surrogate tried, routes 4 to 6, either needs luck in small cases or is provably too weak. Attacking the non-convex system head-on (route 8) is hopeless by sheer size.
- **No shortcut through a hidden variable** (route 1). The argument has to work at the level of the statistics W, presumably facet by facet (Section 57).

When this map was drawn the conjecture was open. D.3 is refuted, and the conjecture itself was later refuted by Section 58h, through a route this map did not list: restricting the search to flip-built

Switchboards.

Philosophical gloss (interpretation). Negative results are knowledge. "This route provably cannot work" narrows the space of possible proofs as surely as a positive lemma narrows the space of possible worlds. Here the negative results also have content of their own. They say that what separates basic chance from ignorance, if anything does, lies in the *counterfactual* behaviour of a reversible world, how it would have treated a source state that was never prepared. Nothing in what actually happens shows it.

Part XII. Philosophical discussion

> **Everything in this part is interpretation.** It is a reading of the results above, following the note's own philosophical sections (3–8 and the end of 9). None of it is a theorem. Where a claim goes beyond the note's own wording, I say so.

62. Consistency plus a complete record gives unitarity

The classical demand "exactly one consistent history, whatever anyone does" was invented to dissolve the grandfather and the bootstrap. Nothing in it mentions amplitudes, phases or interference. Yet the main theorem shows that, together with one more ingredient, it is enough to keep the quantum lift unitary. The extra ingredient is the sink's *complete record*: its perfect copy of every party's output (Sections 18 and 28). Because the sink records the outputs, each final state corresponds to a single history, and histories cannot interfere on the way to the same record. Consistency then guarantees that any two global histories are told apart by some agent who was in the same situation in both. Quantum mechanics turns "told apart by a local agent" into "orthogonal", and that orthogonality, between records that already separate the histories, is what unitarity needs.

The note's own slogan is that fixed-point consistency is "already a quantization condition". The more careful statement is: **consistency plus a complete record gives unitarity.** Consistency alone does not, since the bare loop w (Lugano, say) is not even reversible; the source and sink must be added first. The note surveys other quantum treatments of time loops. Deutsch's fixed-point model pays for its consistency with nonlinearity. Post-selection discards inconsistent branches, which the note calls a filter on samples, "of the same shape as a measure-zero axiom". Neither, the note argues, supplies the criterion. For process functions, the theorem shows that no extra quantum filter is needed. Once the world keeps a complete record, the classical consistency condition, imposed for all interventions, carries over.

63. Lewis and Novikov, revisited

Lewis showed that time travel stories need not contradict themselves. Novikov proposed that physics admits only the stories that do not. Both left a gap. Lewis's "can and cannot" explains why the grandfather survives, but not *which* history obtains. Novikov's principle throws out inconsistent histories but does not choose among the consistent ones. The billiard balls of Section 3 often had infinitely many.

The process-function clause closes both gaps at once, at the price of a strong demand. *Every* intervention must have exactly one consistent history. The main theorem then adds something neither Lewis nor Novikov could have asked for. When the agents are given every power quantum mechanics allows, the demand still holds, with probabilities exactly normalised, for finite and countably infinite alphabets alike. On this reading, the consistency principle is not a filter bolted onto the dynamics, which was

Earman's worry. For process functions, it *guarantees a property of a reversible dynamics* that contains the loop. (The converse, that every inconsistent loop breaks unitarity, is supported numerically but not proved; Section 36.)

64. Can chance be fundamental inside a loop?

Two pictures of chance are in play.

- **Chance as ignorance.** The world is in a definite, deterministic state. Probabilities express what we do not know. In a loop, this means there is a definite consistent deterministic loop, and we are unsure which.
- **Chance as basic.** The world itself is chancy. No more definite story underlies the probabilities.

The polytope separates these pictures geometrically. Mixtures of process functions are ignorance. The random corners, such as E_ex1, are consistent loops whose chance cannot be read as ignorance over consistent deterministic loops.

The results so far:

- In a *classical* reversible world, only ignorance-chance is possible (D.6, proved).
- In a *quantum* reversible world, ignorance-chance is possible (C.1, proved), and at least some basic-chance loops are impossible (E_ex1, and every vertex in orbits 7 and 50, one of which contains E_ex1; proved).
- *Not* every basic-chance loop is impossible in a quantum reversible world: the conjecture is false (Section 58h, proved). Some are possible, and some (E_ex1) are not.

What this teaches (interpretation). Purity does not single out the mixtures of deterministic loops. Quantum reversibility is strictly more permissive than classical reversibility, which does single them out (D.6). The extra loops need three labs, since with two every consistent chancy loop is already a mixture (Section 58a). In them no lab is last, and no lab goes first inside the machine: by Sections 58b and 58d, a machine with a last party or a first mover only produces mixtures. Whether every machine without one can produce chance of this kind is not shown. Whether the chance counts as "basic" also depends on how the loop is read. The machine itself evolves unitarily, and chance enters only when its record is read. Read in the ordinary (computational) basis, the H.13.2 machine, which is the time-reversed Lugano process, gives a plain mixture of deterministic loops. Read in the Hadamard basis, it gives statistics outside the hull (note H.15.7e, H.15.8). So the claim the results support is narrower: *some readings of a reversible loop give statistics that no mixture of deterministic loops can reproduce.*

A reading written assuming the conjecture, now superseded, has moved to the appendix "Superseded readings" at the end.

65. Determinism hidden, or created on the fly?

There are two ways to imagine the chance in a mixture of loops.

- **Hidden determinism.** The world was always one particular deterministic loop. The coin was tossed at the beginning and its face is written somewhere. We just have not looked.
- **Determinism created on the fly.** No face is written anywhere. A definite loop *emerges* as the agents act, and which one emerges depends on what they do.

The clockmaker's workshop (Section 53) is pure hidden determinism: the shuffled deck in the drawer. The D.4 counterexample (Section 55) shows that a quantum reversible world can produce the very same

statistics by the second route. The token answers "which loop?" only when asked, and the question it is asked depends on Alice's choice.

An interpretation written assuming the conjecture, now superseded by Section 58h, has moved to the appendix "Superseded readings" at the end.

66. Reversibility and the arrow of time

Section 21 gave the standard picture: reversible laws, an arrow supplied by a special beginning and an accumulating record. The lift realises that picture precisely, for loops.

- In the middle is U, reversible, with M unitary however the agents act.
- At the beginning is the source, set to a fixed state.
- At the end is the sink, which keeps a perfect record of what the parties sent and received (D.1, Section 56), and which we then ignore.

Randomness appears only at the edges. A mixture of loops arises from a coherent source and a discarded record (Section 47). If the label is not handed to the sink, the branches interfere, and the result is no longer a classical mixture. **Interpretation.** In this toy world, classical chance and the arrow of time have a common origin: what we do not track at the boundaries. The conjecture, read this way, says that boundary-ignorance is the *only* source of chance a reversible loop can have. That is why basic chance, as in E_ex1, would be excluded not by consistency (E_ex1 is consistent) but by reversibility. Section 58h refutes that reading. Interference among the board operations inside the machine, not only ignorance at the boundaries, can produce classical chance in a reversible loop. Reversibility still excludes some basic chance, such as E_ex1, but not all of it.

67. Freedom, bilking and Lewis

Black's *bilking* argument against backward causation goes like this. If a later event causes an earlier one, an agent who sees the earlier event can act to prevent the later one. Either the agent cannot so act, which wants explaining, or the later event was not needed for the earlier one. Lewis answered that the agent can and cannot, in two senses of "can".

The process-function framework takes a definite side. It quantifies over *all* interventions, so no action is forbidden. The loop only constrains which global history results. The note is explicit that this does not settle whether the agent's choice was genuinely *open*. The fixed-point clause says the choice, if made, completes uniquely. Whether that answers Black depends on whether a consistency condition may rule out interventions, or only rule out histories given the interventions. The note takes the second reading, and the formalism is built for it: consistency never forbids an intervention. As the note's Section 5 puts it, "Consistency is not allowed to forbid the operation. It is allowed only to constrain the global solution."

68. Time symmetry and the reading of the source and sink

Read forwards, the source is an input and the sink is an output. Read backwards, using U†, they swap roles, and the reversed operator is the time-reversed process. In a loop like Lugano, an earlier input already depends on a later setting. Some authors, notably Price, argue that time symmetry pushes toward retrocausality. Leifer and Pusey made that argument precise under an assumption ("λ-mediation"), and Maudlin has objected to it. The note's position is modest. The mathematics does not choose between a retrocausal reading and a merely operational time-reverse. It does remove one cheap reply: one cannot say that the cyclic classical model dies as soon as the agents act quantumly.

69. What formal verification adds, epistemically

Interpretation. A proof does two jobs. It *convinces* (this is true), and it *explains* (this is why). Machine-checking strengthens the first job enormously and leaves the second untouched. The Lean files guarantee that the stated theorem follows from the stated definitions. They do not tell you why exclusivity is the heart of the matter. That is what Parts II–V are for.

Machine-checking also changes the *social* character of trust. Without it, a reader trusts the author, the referees, and the hope that someone would have noticed a gap. With it, a reader can trust a small public kernel and two lines of definition, and needs no one's reputation. That suits a result about consistency well. Its claim is that a loop is safe when every agent's every choice completes uniquely, and its warrant is a check that every inference completes.

Two cautions. First, a checked theorem is only as good as its statement (Section 39). Second, the parts that matter most philosophically, the readings in this Part, are exactly the parts no referee, human or mechanical, can check.

70. What the theorem does not say

- **It is not about spacetime.** See "How to read this" at the top: a unitary dilation is not a spacetime.
- **It does not classify all quantum processes.** It places every process function, including ones that violate causal inequalities, among the unitarily extendible processes. It says nothing about processes that are not deterministic classical functions, beyond the stochastic results of Parts VIII–XI.
- **It does not cover continuous alphabets.** Discrete ones of any size are done, in both directions (Part VII; Sections 124–126). Continuous ones are settled for point-moving local operations and for causal loops (F.5); the general non-causal case still needs a definition.
- **It does not settle freedom** (Section 67).
- **The weak point is the lemma.** The combinatorial lemma is the piece an attacker has to break. Everything after it is row orthogonality. In this case the lemma is also machine-checked, and it holds for alphabets of any size.

71. How big is the purifiable set? (H.13)

The H.12 counterexample can be made much plainer. A board that uses only signs (±1) gives a counterexample in which every number is a simple fraction such as 1/16 or 9/16. The source state is just (|000>+|001>+|010>-|101>)/2. Each step between boards mixes 4 classical patterns instead of 8. This was checked in exact fraction arithmetic. Mixing two purifiable processes always gives a purifiable process: a hidden coin decides which purification runs. So the purifiable set is convex. It sits strictly between the mixtures of process functions and the valid processes. A complete search of one natural family of boards (13,105 boards) shows which facets of the hull can be crossed and roughly by how much. 31 of the 93 facet types were never crossed. These are numerical leads, not theorems.

72. Where the line falls (H.14)

A board whose steps each mix at most three classical flip patterns of the plain (X-only) kind can never leave the hull. This is now proved, so four patterns is exactly where X-only counterexamples start. Allowing eighth roots of unity instead of fourth roots crosses 8 more facet types of the hull. Those crossings are checked in exact arithmetic. 23 facet types have still never been crossed. The largest

crossings found have exact values such as $(\sqrt{17}-3)/8$. The pattern suggests, without proving it, that the purifiable set has a curved boundary and is not a polytope.

73. Putting it in spacetime: three time travellers and a quantum loop (H.15)

The stage. General relativity allows spacetimes with closed timelike curves, for example wormholes whose mouths are out of step in time. Baumeler, Costa, Ralph, Wolf and Zych showed that three agents, each in an ordinary small lab, can sit around such a loop. Each can be both in the past and in the future of the others, and each stays free to do whatever they like. No grandfather paradox ever arises. The loop's job is to return to each lab, in its past, a bit computed from what all three sent out. The allowed rules are exactly the "process functions" of this note. Every such rule can be built from reversible machinery, for example billiard balls bouncing inside the wormhole region.

Story 1: the classical loop. Ana, Ben and Cai each find a bit on their doorstep, think, and post a bit into the wormhole. A hidden billiard-ball computer decides what lands on the doorsteps, so that whatever they do, history stays consistent. If the computer's starting position is random, they see a random mixture of consistent histories. These mixtures are the "hull".

Story 2: the quantum loop. Now let the machinery inside the loop be quantum and perfectly reversible: nothing is ever erased, and everything that goes in comes out somewhere. This is what "purifiable" means. Our counterexamples show that the statistics Ana, Ben and Cai record can then fall outside every mixture of consistent classical histories. They can test it: run many rounds, histogram the bits, and add them up with the integer weights of one facet. Classical loops always give zero or more. The simplest quantum loop we found gives minus one-eighth, using only plus and minus signs. That number is a fingerprint of quantum time travel, not just of time travel.

Story 3: what the loop must look like. - Nobody can be first *inside the quantum machinery*. If any one lab's incoming carrier could bypass the loop, the fingerprint would vanish (H.6). (Correction, Oct 23 2026: in what the labs actually observe, one lab can come first. In one version of the rational example, lab B's input ignores everyone's outputs, so B could sit in everyone's past. B's input must still be correlated with the loop (H.15.6).) - The loop obeys a strict traffic rule. When some of the travellers change what they send, the loop's response must leave at least one of them untouched (the Pauli rule, H.8). - Simple loops are not enough. A loop that mixes at most three plain bit-flip patterns at each step can always be faked by a random classical history. Four are needed (H.14). With phase flips allowed too, the same is now proved for most three-pattern loops (H.17); the rest is still open.

Story 4: no wormholes. In ordinary flat spacetime, with no time travel, a similar experiment would need the three "labs" to be smeared out in time, and it would have a hidden step-by-step explanation (Vilasini and Renner). There, the fingerprint would not show time travel. (Correction, Oct 23 2026: we earlier said the hidden explanation "must itself be coherently quantum". That is not proved. The classical Lugano loop also needs smeared-out labs, so smearing alone does not single out our loops. Those flat-spacetime theorems are also proved only for finite-dimensional systems. A 2026 paper by Grothus and Vilasini shows that in 3+1 dimensions, causal loops between point-like events cannot hide behind "no faster-than-light signalling".)

Story 5: could a hidden ordinary schedule explain it? (H.15.7) Suppose each lab acts exactly once, at one hidden moment. The order of moments may be fixed, chosen on the fly, or even quantum-controlled. Then the labs' results must obey "causal inequalities". For the rational loop, we found an exact inequality it breaks (value -5/4 or -4/3, where any ordinary schedule gives at least 0). So no such schedule explains it. A flat-spacetime version would need labs whose single action is spread over several places. For the

first counterexample (H.12), the simple test we ran does not detect this; it is open. Whether our loops can be drawn as the "routed circuits" of Vanrietvelde, Ormrod, Kristjánsson and Barrett is also open. Their conjecture predicts yes, and our loops are a concrete test of it.

Story 5, continued. We wrote a checker for the formal rules of Vanrietvelde et al. and tried the most natural circuit: a station that applies an output-dependent phase in the Fourier basis. It passes the first rule (bi-univocality) but breaks the second (weak loops). The same circuit breaks the same rule for the textbook Lugano process too, and Lugano does have a valid routed circuit, so this failure proves nothing about our loops. For the first counterexample (H.12) and a Z8 loop, a search over many measurement choices found no causal-inequality violation. That narrows the open question but does not settle it.

What is solid and what is not. The mathematics is proved. The papers cited were checked on arXiv (re-audited Oct 23 2026, H.15.6; one wording about the Baumeler-Wolf polytope was corrected: deterministic processes are only 744 of its 710,760 corners). The reading as time travel is an interpretation within one model of quantum time travel; other models (Deutsch's) differ. No actual spacetime or device realising our loop has been constructed.

74. Smooth dials instead of switches, and the full facet tables (F.5, Appendix J)

The question. So far every party's setting was a whole number, like a switch with finitely or countably many positions. What if it is a dial, a real number or an angle?

Story. Ana, Ben and Cai each turn a dial. A loop returns to each of them a dial setting that depends on the others. Before, we counted possibilities: "on average, exactly one consistent history" was the heart of the proof. With dials there are uncountably many histories, so counting has to become measuring volume.

What we proved. - Take any consistent loop and suppose each person only shifts, flips or stretches their dial (and may add a phase). Then the closed loop squeezes no volume and loses none. So it is a perfectly reversible quantum operation, even when nobody is first. - The trick is to peel off one person at a time. Once everyone else's part of the history is fixed, the loop has to hand the remaining person a fixed setting; otherwise two different histories would both be consistent. Peeling the people off in turn turns "one history on average" into "volume preserved". - If the loop is causally ordered, any quantum operation at all works. - What is left open: for loops with no order, and for operations that mix positions completely (like a Fourier transform), there is not yet a natural definition of what closing the loop even means.

Machine checks. The peeling lemma is checked in Lean for dials of any kind. The counterexample of Part VIII now uses the same definitions as the rest of the Lean proofs.

The tables. Appendix J of the technical note lists every one of the 93 kinds of facet, the walls of the classical region. For each kind it gives an explicit integer inequality, how many walls are of that kind, and how far our quantum loops get past it. The 23 kinds that no loop we tried ever crossed are listed separately; that is a numerical finding, not a proof.

What is proved, what is evidence, what is open

Proved

- **A purifiable process outside the hull** (H.12; Section 58h). Exact counterexample to the purification conjecture. It is the note's main result. Purity follows from the paper lemmas H.7.1, H.8.1 and H.8.2. The whole statement was re-derived independently (H.12.2). *Machine-checked in Lean as* `purification_conjecture_false` *L-078, standard axioms only.* The two limits first stated, finite ancillas and self-contained definitions, are now closed: the theorem is restated in the process-matrix framework, and the machine is pure with ancillas in arbitrary Hilbert spaces (H.12.3, H.19; note 87).
- **Two-tone Switchboards are mixtures** (H.10.6; Section 58h). Coset-transport theorem plus an exhaustive exact check of all 39,512 two-pattern boards.
- **Unitary iff consistent** (Theorem 1.2; Section 36). With finite ancillas, the closed loop is unitary for all local unitaries exactly when w is a process function. The converse is a paper proof by the copy gadget (a column's squared norm counts fixed points), with 0 mismatches against brute force. *Update (Nov 2 2026):* the converse is now machine-checked (`PFUL.CopyGadget.unitary_iff_pf` L-066), also without ancillas (`PFUL.NoAncilla.unitary_noAncilla_iff_pf` L-174), and both versions hold for discrete alphabets of any size (`PFUL.General.unitary_l2_iff_pf_any` L-132, `PFUL.General.unitary_l2_noAncilla_iff_pf_any` L-133) and with addition replaced by any free shuffle (`PFUL.General.unitary_l2_iff_pf_σ` L-017; Sections 124–127).
- **The conjecture is refuted** (H.12; Section 58h). "Purifiable iff a mixture of process functions" is false. Lean certifies a margin of about 0.0076 outside the hull; the computed LP distance is 0.0607 (not certified). H.21.2 identifies the machine as a frame-rotated time reverse of the $\sqrt{\text{X}}$-Lugano process.
- **The main theorem.** Every process function, with any finite alphabets, lifts to a pure unitary process: under any local unitaries with any finite ancillas, the source-to-sink map is unitary. *Machine-checked in Lean 4 (exclusivity lemma; $M\,M^\dagger = 1$ and $M^\dagger\,M = 1$; unitary-group membership), standard axioms only.*
- **Validity under everything.** Validity under arbitrary local operations, entangled or shared ancillas, and instruments follows by Stinespring, and so does the process-matrix normalisation $\mathrm{Tr}[W(\ldots)] = 1$. *Machine-checked in Lean, finite dimensions:* `PFUL.valid` *L-231,* `PFUL.valid_kraus` *L-030,* `PFUL.valid_instrument` *L-029,* `PFUL.Choi.bornC_anc_eq_one` *L-035 and the group-free* `PFUL.Choi.bornAnc_eq_one` *L-020, the equivalence* `PFUL.Choi.afnb_iff_density` *L-034 (and* `_split`*), Choi's theorem* `PFUL.Choi.isCP_iff_choi` *L-044, all bundled in* `PFUL.Choi.validity_package` *L-022.*
- **Countable alphabets** (Appendix F). With finitely many parties and countable alphabets, the inputs carrying a group structure, the lift is unitary on ℓ^2. Rows come from exclusivity, and unit columns from the expected-number-of-fixed-points argument, which also gives a dimension-free finite proof. *Machine-checked in Lean, ending in* `PFUL.Countable.induced_unitary_l2` *L-002. The finite-alphabet column proof is checked too. Update (Nov 2 2026): discrete alphabets of any size,* `PFUL.General.induced_unitary_l2_any` *L-130.*
- **Infinitely many parties** (Sections 126–127). For a continuous loop, the "two histories always disagree somewhere their inputs agree" property still gives exactly one consistent history (`PFUL.Infinite.pf_of_exclusive_continuous` L-134). Without continuity it fails, in ordinary set theory: an explicit Hilbert's-hotel loop (`PFUL.Infinite.exists_exclusive_not_pf` L-233; Story 4 of Section 127). An earlier version needed the extra axiom $\mathrm{cov}(M) = \mathfrak{c}$, which

CH implies (`PFUL.Infinite.exists_exclusive_not_pf_of_covM` L-119, `PFUL.Infinite.exists_exclusive_not_pf_of_CH` L-118; paper proof refereed).

- **Mixtures.** Mixtures of process functions are purifiable: a coherent random source, with the label recorded in the sink.
- **The classical version.** Classical reversible purifiability holds exactly for mixtures of process functions (D.6). By the counterexample, quantum reversibility does add something.
- **Certificates.** E_ex1 and the vertex orbits 7 and 50 (one of which contains E_ex1) are not purifiable. *Exact rational certificates, not Lean proofs.*
- **Structure.** In any purification, the sink holds an orthogonal record of (outputs, inputs) (D.1).
- **The dead ends.** Constraints on the fixed source state alone are vacuous (D.2). Validity under quantum-controlled interventions holds for every classical process (D.5). The hidden-label strengthening D.3 is false, with a causal circuit as counterexample (D.4). No certificate that uses only the fixed source state can rule out purification of any classical process (D.8).
- **Two parties** (Appendix G; Section 58a). Every two-party valid classical process is a mixture of ordered process functions, so for two parties purifiable means mixture; the counterexample needs three.
- **Ordered three-party processes** (Appendix H; Section 58b). If some party is last, or first with valid conditionals (causally separable), the process is a mixture of process functions.
- **A shortcut closed** (H.5; Section 58c). Purifiable plus marginally first does not force valid conditionals; exact counterexample.
- **The Chorus** (H.7; Section 58e). A machine with no first or last mover can still produce an ordinary mixture of ordered process functions, so "no first mover" does not mean "Lugano in disguise". The Switchboard's no-ancilla purity condition (Lemma H.7.1) is proved, and it is the full purity condition: it reads as operator-valued exclusivity, and ancillas add nothing (H.8; Section 58f). Relabelling-type Switchboards give mixtures of process functions.
- **The shortcut repaired at machine level** (H.6; Section 58d). If some purification has a party first *in the machine* (its input prepared from the source alone), the conditionals are valid and the process is a mixture of process functions.
- **Facets.** The three-party binary process-function hull has exactly 29,344 facets in 93 orbits (exact computation with normaliz).

Negative results (Appendix E)

- Symmetry stabilisers are large only where certificates already exist. Most open orbits have trivial or tiny stabilisers.
- The moment relaxation is feasible for E_ex1 at $r = 16$, so it is strictly weaker than single-form inertia, and symmetry-reducing it is pointless.
- Algebraic geometry on the full system is infeasible by size: 352 real unknowns and 23,552 equations for E_ex1, and about 2,112 and 100,352 for orbit 8, with about 2^{2112} homotopy paths (code/stochastic/alggeom/SIZE_ESTIMATES.md).

Evidence (not proof)

- A GPU search found no purification for any of the 49 remaining sampled non-deterministic vertex orbits. Best residuals were 12.97–59.35.

- Machines with no first mover (H.7; Section 58e): a small, local search found 13 pure machines, including the Shadows near Lugano. All of their statistics lie inside the hull.
- Two-tone Switchboards (H.10; Section 58g): all 39,512 two-pattern boards allowed by the rule, checked against every hull wall at seven angles, stay inside the hull for every source state. Float64 evidence, no counterexample. Proved: no relabelling trick and no degree-2 argument can settle the general case.
- ~~Switchboard toll (I.4; Section 58f)~~: withdrawn. It was an artefact of local search; H.12 crosses the hull at zero impurity.
- Degree-4 certificates for orbit 8 of the proposed shape cannot exist for random slices (C.6). No paid compute was used.
- A single facet orbit (82) is violated by E_ex1 and by all 51 sampled non-deterministic vertex orbits.
- Along the segment from Lugano to E_ex1, purifiability appears to stop exactly where the hull does.
- Orbits 8, 10, 20: isotropic subspaces appear to stop near dimension 10, short of the 16 needed; form certificates provably cannot capture this (C.5). A unitary-side counterexample search found nothing (H.3).
- Without ancillas: non-process functions fail unitarity under random local unitaries in every sampled case. *(Superseded: the ancilla-free converse is now proved and machine-checked, Section 124.)*

Open

- **After the refutation.** The conjecture itself is settled (see Proved). Open now: what the purifiable set actually is (a polytope?), the smallest counterexamples, and other scenarios.
- ~~**The converse without ancillas, and infinite alphabets**~~ *Settled for discrete alphabets of any size (Sections 124–125).*
- **Infinitely many parties, in ordinary set theory.** *Settled Nov 3:* yes, ZFC alone gives such a loop, and it is even Borel, in fact exactly Baire class 2 [proved, Lean: `PFUL.Infinite.zfcF_borel_measurable` L-247, `zfcF_coord_baireTwo` L-248, `zfcF_not_baireOne` L-249] (Section 126(d)). Still open: which kinds of mild discontinuity are enough to force a consistent history. Section 128: finite prefixes settle the bounded-defect case; one-sided Σ^0_2 and Π^0_2 maps can fail to have a consistent history; continuity off a closed perfect nowhere-dense set can fail too; and for Baire class 1 that geometric restriction is equivalent to the full question. Section 129: the countable-closed conjecture is now a theorem, and so is the countable G-delta strengthening. Section 130: a finite palette of whole output lists forces a consistent history, and so does an output range trapped near one list. The prefix-elimination limit and the Yablo approximants do not settle Baire class 1. An arbitrary countable exceptional set together with Baire class 1, and the unrestricted Baire-class-1 question, remain open. No exclusive Baire-class-1 counterexample is known proved, Lean: L-269–L-321.
- **Loops without a first mover.** Which processes are purifiable among those whose purifications all have no party first in the machine (H.6; Lugano-type loops; the Ring, Chorus, Switchboard and Shadows of Section 58e), and the exact status of orbits 8, 10, 20 and the other 46 sampled open orbits.
- **Facets from purifiability.** No facet inequality, for example orbit 82, has been derived from purifiability.

- **Where any non-purifiability proof must look.** Any proof must use the purifying unitary on source states orthogonal to the actual one. That is the non-convex isotropy condition of Araújo et al.'s Theorem 4, where every certificate attempt has stalled for the larger orbits: single-form inertia, block forms, SDP and moment relaxations.
- **Continuous alphabets.** Settled for point-moving local operations and for causal loops (F.5, paper proof; core lemma in Lean). Open for non-causal loops with general local operations.
- **Not formalised.** The literal Choi form $\mathrm{Tr}[W(\ldots)] = 1$ in infinite dimensions, where W is not trace class. The trace form is checked: the project's own trace-class and partial-trace library (`lean/PFUL/TraceClass.lean`) gives `PFUL.Countable.valid_density_l2` L-225, $\mathrm{Tr} = 1$ for every input density operator on ℓ^2.
- **Not addressed.** Spacetime embedding. (Lifts built from shuffles other than addition are now treated: Section 126(a).)

For definitions, full proofs, references and code pointers, see the technical note, paper/lift.md, and lean/STATUS.md.

75. Plain-language note: what the computer now checks about smooth dials (Oct 23 2026)

Section 74 described inputs that are smooth dials (a position on a line or an angle on a circle) rather than finite menus. Two of its key facts are now checked line by line by the Lean proof assistant, using only the standard axioms.

- **No volume is lost or gained.** Wire the parties into any loop, causal or not. The map from "what everyone outputs" to "the net shift around the loop" moves regions of outcome space around without changing their total size. This is the continuous replacement for "exactly one consistent history" in the finite case. The computer proves it for any number of parties on lines, circles, or any group with a translation-invariant notion of size.
- **Probabilities still add to one.** If every party just shifts its dial and adds a phase, the whole loop is a unitary. Total probability is conserved for every input state.
- **The counterexample is tied to the standard formalism.** The Switchboard's purity, the key ingredient of the counterexample to the purification conjecture, is now shown to be a property of its process matrix $W = |V\rangle\rangle\langle\langle V|$. That is the object the rest of the theory uses.

Not yet checked by computer: local operations that stretch the dial (these need a Jacobian change of variables), memories (ancillas) that are infinite-dimensional, and the automatic boundedness of physical maps.

76. Plain-language note: helpers of unlimited size (Oct 23 2026)

Before this round, the computer checked the Switchboard counterexample only when each outside "helper" system had finitely many settings. Helpers may now have countably infinitely many settings, such as an infinite ladder of levels. The computer checks that the combined process still loses no information: it is an isometry, meaning it preserves lengths and angles. It also checks that the new definition gives the same answer as the old one whenever the helpers are finite, and the counterexample to the purification conjecture holds in this stronger form. Two things are still open. The computer has not shown the stronger statement that the process is fully reversible (unitary) for infinite helpers. The links to the general validity definition and to general smooth reparametrisations are also still open.

77. Plain-language note: the helper machine is fully reversible, and smooth dials work too (Oct 23 2026)

Last round, the computer checked that our helper construction never loses information when the helper is unlimited. Now it also checks that the construction can be run backwards, so it is fully reversible. This means the counterexample survives the strictest reading of "purifiable".

The computer also checks the "smooth dial" result for any smooth, one-to-one rearrangement of space. Stretching and squeezing are fine, provided the rearrangement never crushes a region flat.

Two puzzles are still open: - the boards that are not of the X type; - the 23 inequalities that no test has broken yet. (Note 78 shows one of them is just "probabilities are never negative", so 22 are genuine.)

78. Plain-language note: attacking the two open puzzles (Oct 23 2026)

This round attacked the two open puzzles: switchboards that are not of the X type, and the 23 inequalities no test had broken, of which 22 turn out to be genuine (paper Appendix H.17).

- **Non-X switchboards exist.** These are reversible switchboards with up to three terms per setting that are not of the X type. The computer built more than 20,000 of them, and every one stayed inside the region spanned by ordinary process functions. This is a computer check, not a proof.
- **A small theorem.** One way such a board could escape is through a "sign flip" between two settings of the same dial. An exhaustive exact check proves this can never fit into a full three-party board, as long as the dial settings are generic.
- **Validity alone is not enough.** For 22 of the 23 inequalities we have exact counterexamples that satisfy every consistency rule but break the inequality. So any proof of those 22 must use reversibility (purifiability), not just consistency. The last one (number 92) turned out to be simply "probabilities are never negative". An exact certificate now proves this, so it holds for every process. It should never have been on the open list.
- **Wider searches.** Finer phase dials, continuous dials and local rotations of each party broke none of the 22 genuine inequalities.
- **Half a proof for non-X boards.** The old argument for X-type boards carries over to every board whose "reachable flips" form a closed group. This part is proved. For three-term dials the reachable flips are {none, A, B}, and the flip "A then B" is not covered automatically. That last finite check is what remains.
- **The last check, done.** A complete computer enumeration of every way two dials can sit side by side found 329,280 local trouble spots. All but 288 of them provably cannot fit into a full three-party board. An exact search over every possible full board then showed that the last 288 cannot fit either. So every such board with generic dial settings is proved to be a blend of fixed-point machines. Boards with finely tuned dial settings, where terms cancel, are different. Of the 37 tuned boards the search found, 35 fail the test, so the proof does not reach them, although all 37 still sit inside the blend region numerically. A second trick handles them. First fold one dial's setting into the starting state, then run the same test on what is left. That variant is proved to work whenever it passes. We rebuilt 18 tuned boards, plus one saved earlier, and all 19 pass it, so each of them is proved to be a blend. A computer also found the blend recipe explicitly for each. We then listed every way the terms of two dials can cancel. Dials built on different term sets cannot cancel at all. Dials sharing all their terms cancel in only a few ways, and that list is proved. Dials sharing

half their terms can lose exactly one term, never two, and that list is proved too. A further 32 tuned boards also pass, 51 in all. Most of them turned out not to be really tuned at all: the search had given the same dial two names. Every genuinely tuned board found shrinks to the simple one-switch type once one dial is folded into the starting state. We then ran that exhaustive search for every board built from just two dials, tuned or not: all 59,836 ways the two dials can combine. None of these boards escapes, so every two-dial board is proved to be a blend. Next came boards with three dials where one of them is the trivial do-nothing dial: about 244,000 cases. A first pass flagged 144 suspects. A sharper version of the same test, also proved, cleared all of them, so these boards are proved blends too. Still unproved: tuned boards with three real dials, or four or more. Only 2 of the 51 tuned boards found are like that, and both pass.

- **Three real dials, done (update, note 82).** Boards with three non-trivial dials, with at least one pair of them tuned together, were next. Swapping the parties around and applying certain phase gates that change nothing observable cuts the 1,071 possible first dials down to 61 essentially different ones. The search has now checked every combination of second and third dial for all 61. No board escapes, once a simple consistency rule is applied: the dial-to-dial differences must multiply correctly. Every suspect the search raised broke that rule, so it could not be a real board [proved, H.17.15]. Folding one dial into the starting state does not reduce these boards to the easier case, so they had to be checked by search.

Still open: boards with four or more dials (all found so far pass), and the 22 genuine inequalities in general.

79. Plain-language note: what is new, and who got there first (Oct 23 2026)

We checked the literature on Oct 23 2026, by web search and by reading the primary papers. We did not do a systematic citation search, and very recent preprints may be missing. Paper Section 0 ("Novelty and prior work"), Section 6, C.3 and H.13.5 give the details.

- **A reversible quantum loop with genuine chance beyond process-function mixtures (H.12, H.13): new, as far as we can tell.** Baumeler, Gilani and Rashid (2022) had already said that mixtures of process functions are purifiable. Nobody we found had stated, proved or refuted the converse.
- **The closest open question is not answered.** Kunjwal and Oreshkov ask whether reversible quantum loops can produce *antinomic* correlations: statistics no classical deterministic loop could produce, even with clever local processing. They also note that classical processes outside the process-function hull cannot be made classically reversible. Our boards are quantum-reversible versions of exactly such processes. But the statistics of the H.13.2 example, read in the plain way, can be imitated classically. An exact decomposition into 23 classical strategies shows this (code/stochastic/cyclic/nomic/). So their question stays open. Other measurements on our boards have not been tested.
- **Every process function lifts to a unitary process: partly known.** It had been asserted without a full proof (Baumeler, Gilani and Rashid). It had been proved for Lugano (Araújo et al. 2017), for a family of loops (Araújo, Guérin and Baumeler 2017), and for a subclass (Daher Ahmed and Kunjwal 2026). Daher Ahmed and Kunjwal state the general case as Conjecture VII.1, which our proof appears to settle. The extension to countably infinite alphabets is, as far as we found, new.
- **The three-term threshold (H.14): new.** It is phrased in this note's own Switchboard terms.
- **No single-use fine-grained model (H.15.7b): an application.** The underlying principle is known (Wechs et al. 2021). Only the exact certificate for this example is ours.

- **Update after H.15.7e: the H.13.2 machine was already known (H.15.8).** Guérin and Brukner (2018) studied "the Lugano process run backwards in time". They showed it is a legitimate reversible quantum loop and that it fails a causal test. Our H.13.2 machine is that same loop, viewed through Hadamard-rotated glasses on every agent's wires.
  - What we found no one saying is this. If you read that known loop's wires in the rotated way (the Hadamard basis), the classical statistics it produces are *not* a mixture of classical deterministic loops. If you read it the ordinary way (the computational basis), they are, trivially: each agent just gets a copy of the starting bits.
  - So our "beyond process-function mixtures" claim survives. For H.13.2, though, it is a new reading of a known machine, not a new machine.
  - The Switchboard (H.12) is not of this rotated form [proved in that frame], so the main example is unaffected.
  - Among all 744 three-party deterministic classical loops, only the Lugano family gives a non-mixture when rotated this way [numerical].

80. Plain-language note: the Switchboard fails a causal test too, and why symmetry saves work (Oct 23 2026)

- **The Switchboard (H.12) now fails a causal test, with exact proof (H.15.7d).** A "causal inequality" is a scorecard that any story with a fixed or dynamically chosen order of the three agents must pass. In H.15.7b only the H.13.2 board was shown to fail one. For the Switchboard, the earlier search tried 13 measurement directions and found nothing. It was also steering by a score that sat flat at its best value ("t = 1"), so there was no slope to follow. This time we let each agent use any measurement on its qubit, and we steered by one fixed scorecard at a time. That gave the search a slope.
  - The best strategy turns out to be simple: each agent measures along a tilted direction and sends out its own setting.
  - We then rounded the six directions to points with rational coordinates, so every number in the calculation is exact (fractions and $\sqrt{2}$).
  - The score comes out to a definite negative number, about -0.706, where every causal story scores at least 0.
  - So the Switchboard, like H.13.2, cannot be explained by agents who each act once in some (possibly dynamically chosen) order [proved].
- **The H.13.2 board, turned sideways (H.15.7d) [proved].** Apply a Hadamard rotation to every agent wire and the board becomes very simple. Each agent receives a bit straight from the past. The machine then flips agent k's outgoing bit according to the *other two* agents' incoming bits. For example, Alice's bit is flipped exactly when Bob's and Charlie's are both 1. No agent's flip depends on its own input. This echoes the Lugano rule that a majority for Alice never depends on Alice's own vote [INTERPRETIVE].
- **Routed graphs, still open.** We checked that our validity checker accepts the published valid diagram for Lugano. We then tried two natural diagrams for the sideways H.13.2 board, and both fail the same way. In each, the agents can "choose" both what they send and what they received, and those two kinds of choice chase each other around a loop. Whether some other diagram works is still open.
  - *Update (H.15.7e): solved, and the answer is surprising [proved; one step cited].* The sideways H.13.2 board is the famous Lugano process *run backwards in time*, up to a few local bit flips. Its flipping rule "flip Alice when Bob and Charlie are both 1, ..." is exactly the Lugano majority rule after renaming $0 \leftrightarrow 1$ on some wires. Running the published Lugano diagram

backwards gives a valid diagram for H.13.2: every arrow is reversed, and the "forward" and "backward" kinds of choice swap roles, so the loop rule still holds. Our two failed diagrams had pointed time the wrong way. They put the flip-controlling bits on what the agents *receive*, but they belong on what the agents *send*.

    - *Credit (H.15.8):* the backwards-in-time Lugano process itself, and the fact that it is a valid reversible loop failing a causal test, are due to Guérin and Brukner (2018). What is ours is noticing that H.13.2 *is* that process, and the diagram that follows.

- **How small can the Switchboard's measurements be? (H.15.7e).**
  - With only the six "textbook" directions (±x, ±y, ±z), none of our 2,432 scorecards is failed [numerical].
  - Add the cube's edge and corner directions and it fails, with a short exact proof. The score is $131/48 - (13\sqrt{6} + 19\sqrt{2} + 7\sqrt{3})/24 \approx -0.222$. You can check that this is negative using three decimal places of each square root [proved].
  - The best score we found with any measurements is about −0.72 [numerical]. Whether that is the true limit is open.
- **Symmetry and orbits, gently (H.17.15).** Suppose you have to check 1,071 tile designs for a flaw, and you know that rotating or mirroring a tile can never create or remove the flaw. Then you only need to check one tile from each family of designs that are rotations or mirror images of one another. Here, literally:
  - A *class* is the list of Pauli strings that appear in one of the board's dials. There are 63 one-string lists and 1,008 two-string lists, which gives 1,071 classes.
  - The "rotations" are the 48 moves of the group G_{48}. A move renames the three qubits (6 ways) and, on any subset of qubits (8 ways), conjugates by the phase gate S, which swaps X and Y there.
  - None of these moves changes the outcome probabilities (up to renaming the parties), whether the board is a legal process, or anything the computer search tests. So a counterexample exists for some class exactly when one exists for every class in its family.
  - A family, called an *orbit*, is everything one class can be turned into by the 48 moves. The 1,071 classes fall into 61 orbits, and the search checks one representative of each.
  - An orbit is "proved" once its search has finished and every leftover candidate has been ruled out by the triple-consistency rule. "Still being searched" means it is not finished yet. (Update: the search has finished and all 61 orbits are now proved; see note 82.) "Genuine" belongs to a different kind of orbit: families of scorecards (Appendix J), where it means "not just positivity".

81. Plain-language note: is the Switchboard secretly a famous machine? (Oct 23 2026)

- **Credit check on the routed diagram (H.15.9(A)) [cited].** The authors of the routed-circuit framework (Vanrietvelde, Ormrod, Kristjánsson and Barrett) define the "time-reversed" diagram themselves, and their two legality rules are symmetric under it. So "a legal diagram run backwards is still legal" follows straight from their definitions. Our backwards-Lugano diagram is a direct application of their work. What we add is only the explicit check and the gate-by-gate assembly.
- **The Switchboard is not Lugano in disguise (H.15.9(B)).**
  - Two machines count as "the same up to local relabelling" if you can turn one into the other with a separate rotation on each wire and by renaming the parties.
  - Some fingerprints survive every such relabelling. Here we use the spectra (eigenvalue lists) of the machine's reduced states on one or two wires.
  - The Switchboard's fingerprints match none of the 744 classical-loop machines (Lugano

among them), run forwards or backwards [numerical].
  - It also cannot be a quantum switch or any other "orders in superposition" machine, since those never fail a causal test and the Switchboard does [proved].
- **Running the Switchboard backwards tames it [proved in the plain basis; numerical in three rotated bases].** Backwards, each agent simply receives a copy of the past. In every basis we tried, the classical machine you read off lies inside the hull of classical loops. Backwards Lugano behaves differently: it escapes the hull in the rotated basis. That escape is the H.13.2 board.
- **How badly can the Switchboard fail its test? (H.15.9(C)).**
  - Best score found so far: about -0.72 [numerical].
  - A relaxed problem that allows more than real measurements can achieve proves the score can never go below about -1.59 for the main test [numerical, solver accuracy].
  - The true limit lies somewhere in between, and it is open.
  - *Update (note 82):* this floor (-1.5856) is now proved exactly.

82. Plain-language note: the three-dial search is finished, and a proved floor for the Switchboard (Oct 24 2026)

- **Three dials: closed (H.17.15) [proved].** The long computer search over boards with three non-trivial dials, at least two of them tuned together, has finished all 61 families ("orbits", note 80): 5,659,539 combinations in all.
  - It raised 188 suspects, and 1,274 cases where it ran out of its step budget.
  - Every one of those 1,462 breaks the simple consistency rule: the difference between dials a and c has to be built from the a-to-b and b-to-c differences. So none of them can be a real board.
  - That the rule really filters is checked too. On ordinary combinations it lets most through (for example, 16,715 of 17,855 in one family), and the known tuned boards pass it.
  - Conclusion: every board with at most three terms and at most three distinct dials is a blend of classical loops. Only boards with four or more dials remain open.
- **A proved floor for the Switchboard's score (H.15.9(C′)) [proved].**
  - Note 81 gave a floor of about -1.59, but only to the computer's accuracy. Now that floor is a theorem.
  - The relaxed problem has a "dual": a set of matrices that certifies a floor whenever certain combinations of them have no eigenvalue below a stated number. We rounded the computer's dual to exact fractions. Then we checked every eigenvalue claim (512 blocks, plus 1,536 for the extra "partial transpose" constraints) with exact fraction arithmetic, with no rounding anywhere.
  - Result: whatever measurements the agents make, the score is at least -1.5856. A simpler version without the extra constraints gives -1.7243. A useful feature is that the rounded dual does not need to fit the constraints exactly. Any misfit just lowers the floor a little, here by about 0.00004.
  - So the true best score lies between -1.5856 and -0.7058, both ends proved. This relaxation cannot do better than about -1.5855, so narrowing further needs a stronger method. Where in this range the true value sits is open.

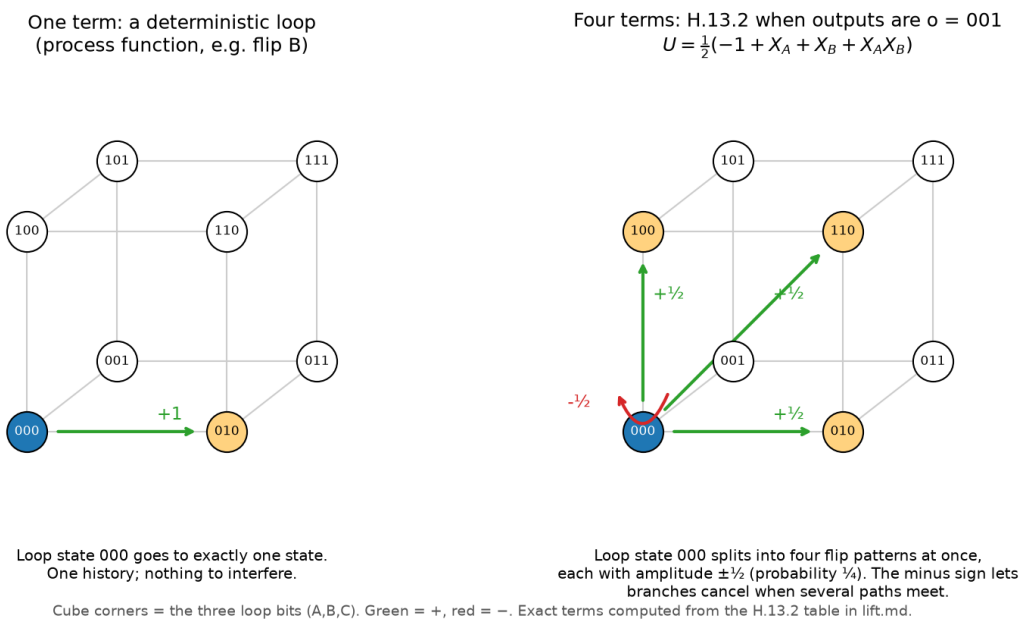


Figure 1: One term versus four terms on the loop cube. Left: a deterministic loop sends loop state 000 to exactly one state. Right: the four-term H.13.2 operator at outputs o = 001 sends 000 to four states at once, with amplitudes −½, +½, +½, +½.

83. Plain-language note: what "four terms" looks like, and is four really the minimum? (Oct 24 2026)

Figure: one term versus four terms. The figure is drawn by docs/figs/fourterms.py, and its amplitudes are checked against the H.13.2 table.

- **What a "term" is.** At each setting of the agents' outputs, the machine applies one operator to the three loop bits A, B, C (the corners of the cube). Write that operator as a sum of "flip patterns": leave alone, flip A, flip B, flip both, and so on. Each pattern that appears is one *term*.
  - **One term (left).** One flip pattern, so a definite loop state goes to exactly one next state, here 000 to 010 ("flip B"). That is one history, an ordinary deterministic loop: a process function. Nothing can interfere with it.
  - **Four terms (right).** This is the rational counterexample of H.13.2 at outputs o = 001, where the operator is ½(−1 + X_A + X_B + X_A X_B). State 000 goes to four states at once (000, 100, 010, 110), each a *signed branch* of amplitude ±½, so each has probability ¼ on its own. One branch carries a minus sign.
  - **Interference.** When several branches reach the same state by different routes, their amplitudes add, and the signs can make them cancel. This cancellation lets the four-term machine produce statistics that no mixture of ordinary loops can (H.13.2) [proved].
- **Is four really the minimum?** Two terms are never enough (H.10.6) [proved]. For three terms, notes 80 and 82 showed that a three-term machine whose operators come in at most three genuinely different kinds ("dials", or classes) is always a blend of ordinary loops [proved]. This round attacked the last case: three terms with **four or more** different dials (H.17.16).
  - *Three simple rules [proved].* (1) If two dials differ only by a single flip pattern, they are really the same dial. (2) Two neighbouring corners whose dials differ must differ "evenly" in that

corner's direction. (3) For every three dials, the a-to-c difference has to be built from the a-to-b and b-to-c differences (the rule from note 82).

- *An exact search of the most "cancellation-prone" family [proved for that family].* Here all dials are built from the same three flip patterns, a so-called Klein group, which gives the most room for cancellations. Using the symmetries, there are 21 such families. The search covered all 21 families with four dials each: 1,862,595 combinations, and none of them gives a machine that escapes the classical loops [proved for this family]. A random check of 1,200 combinations found something stronger: the rules force every one of them to be trivial (one agent simply acts last).
- *Random search [numerical].* Among 39,206 random admissible three-term machines, 5,800 were "hard" (no agent is trivially first or last). Almost all of them collapse to a single dial; ten have two dials and none has three or more. None beats the classical loops.
- *What stays open.* We found no gluing argument that reduces four dials to three, and the full search is far too big for one machine (roughly 10^9 cases). So "four terms is the minimum" is **still open** in general. No three-term counterexample is known.
- *Update (notes 85, 106, 108, 112; H.17.17, H.26.1).* Settled since: there are at most four dials, and the four-dial search finished clean. So four terms is the minimum, proved in the same model.

84. Plain-language note: running a time loop on an ordinary clock, and the four-dial search continues (Oct 24 2026)

- **The question.** Can our loop machines exist in ordinary spacetime, with no wormholes? Earlier notes said no if each agent sits at one place and one time: that is the Vilasini–Renner no-go [cited]. This note follows the other way out.
- **Time-delocalised systems [cited idea, Oreshkov 2019; Wechs–Branciard–Oreshkov 2023].** Picture an ordinary lab that runs a fixed sequence of gates, one after another. Normally, "Alice's input" is the wire at one particular moment. But you can also bundle several wires from different moments and split the bundle differently. Then "Alice's input" can be "the wire at time 1 if a control bit is 0, the wire at time 2 if it is 1", in superposition. A system defined this way is *time-delocalised*. Described in these bundled systems, the ordinary step-by-step lab looks exactly like a loop.
  - Wechs, Branciard and Oreshkov proved that every three-agent loop with a reversible ("unitary") completion can be run this way [cited]. Both the Switchboard (H.12) and the rational counterexample (H.13.2) are such loops, so both can be run on an ordinary clock in this sense (H.18.3).
- **An explicit recipe for H.13.2 (H.18.4) [proved, checked numerically].** We already knew that H.13.2 is the time-reversed Lugano loop seen in a rotated basis (note 80, H.15.7e). Wechs–Branciard–Oreshkov wrote out a step-by-step lab for the Lugano loop. We reversed that recipe and added the basis rotations. Running every step backwards gives another ordinary lab, and the rotations become part of what each agent does. The computer confirms that the result reproduces H.13.2 exactly, to about 10^{-15}, for random agent actions and for the classical actions that beat the causal inequality.
  - In this lab, Alice and Bob each act once, but who goes first is decided by a quantum control bit. Charlie's action appears at two different moments, once in a "do, flip, undo" form. Wechs–Branciard–Oreshkov stress that on the bundled systems this still counts as Charlie acting once.
  - This does not contradict the no-go: there, agents sit at one time, and here they do not. It also does not contradict our earlier result that no one-use-per-agent model works (H.15.7b),

because this lab is not one-use at the level of individual moments.

- **What we did not do.** We have no explicit recipe for the Switchboard. Its existence follows from the theorem, but the Switchboard is not a rotated classical loop, so the shortcut fails (H.18.6). Whether a recipe with only classical signals exists for H.13.2 is open too.
- **The four-dial search (H.17.16b).**
  - We rewrote the search so that it builds cases one at a time instead of all at once, which used to run out of memory. It also skips cases that are mirror images under the symmetries.
  - With five dials in one family, half of the batches (52 of 104, about 417,000 cases) are finished and clean: no suspects, nothing left unresolved [proved for those batches]. The rest runs overnight.
    - *Update (note 112, H.17.16c).* All 104 batches finished clean (38,135,448 cases), and the coverage was audited.
  - A random sample of four-dial cases across families was also clean [numerical]. The full cross-family search would take about 10^5 processor-hours, far beyond this machine.
  - A small counting fact: with five dials on eight corners, at least two dials sit on a single corner [proved].
  - "Four terms is the minimum" is **still open**. *Update (H.26.1):* now proved in this model.

85. Plain-language note: every dial is stiff in one direction, so there are at most four dials (Oct 24 2026)

- **The question (from note 83).** Is four terms the fewest a loop machine needs to beat the classical loops? What was left: three-term machines with **four or more** different dials (classes). We looked for a "gluing" argument that would cut a big machine into pieces with fewer dials. We did not find one. We found something simpler.
- **Every dial is stiff in some direction [proved] (Lemma H, H.17.17).** Each three-term dial has a loop bit on which its flip patterns all act differently. Call that bit's direction on the cube the dial's *stiff* direction. The rules of note 83 then force the following: if a corner carries a dial, the neighbouring corner along the stiff direction carries the **same** dial. The proof is a short count. A different dial there would need more than three terms.
- **So dials come in pairs, and there are at most four [proved] (Theorem H.17.17).** Each dial covers an even number of corners, at least two. The cube has eight corners, so there are at most four dials. Five or six dials are impossible, so the five-dial search of note 84 was checking machines that cannot exist. It found none, as it had to.
- **Checked by computer proof [Lean ✓].** The stiffness lemma and the at-most-four-dials count are also checked line by line by the Lean proof assistant (lean/PFUL/LemmaH.lean). Lean takes a few facts from earlier notes as given: what three-term dials look like, and rule (2) of note 83 in the form the proof uses. Everything from there to "at most four dials, two corners each" is machine-checked.
- **Four dials is now a finite search.** With four dials, each covers exactly two neighbouring corners. Up to symmetry there are only **2,722** ways to place and label them. A computer search checks every way exactly [proved for the cases finished so far]. As a control, the same program without the final test finds ordinary "hard" machines easily, so it is not searching an empty space [numerical].
- **Status.** At the time of writing, a few hundred of the 2,722 cases are finished, and all are clean: no suspect and nothing left unresolved. The rest runs in the background for a few more hours on this machine. If it finishes clean, "four terms is the minimum" is proved for this model of three-term machines. Until then it is **still open**, but what is left is now a running search, not a missing idea.
  - *Update (notes 106 and 108; H.26.1, H.26.1a).* The search finished: all 2,722 cases clean

(2,896,784 sub-cases, no suspect, nothing unresolved), and an independent audit confirmed the coverage. "Four terms is the minimum" is proved in this model.

- **What did not work [numerical].** We checked whether the tests that separate quantum from classical behaviour (hull facets) look only at a few corners at a time. They do not: the ones the four-term counterexample violates all involve at least four corners. That is why the argument has to use the whole cube, as the stiffness argument does.

86. Plain-language note: a glossary for the technical note (Oct 24 2026)

- **What was added.** The technical note now ends with a glossary, Appendix K, just before the reference list. Each entry gives a precise definition, the notation, the section where the term is defined, and the source.
- **Why it helps.** Some words in the note are borrowed (process function, purification, causal inequality). Others are the note's own (board, Switchboard, rigid direction, tuned). A few are used in more than one sense, most of all "class" and "orbit". The glossary says which is which.
- **One trap it flags.** "Tuned" here means a pair of dials whose flip patterns overlap, so terms can cancel. It is not the "fine-tuned" of Baumeler and Wolf, which means a chancy loop outside the hull of deterministic ones.
- **Status.** The glossary adds no new results. Where it and the body of the note differ, the body is authoritative.

87. Plain-language note: the counterexample in the standard language, and infinitely large side systems (Oct 24 2026)

- **What was missing.** The computer-checked proof of the main result (H.12) used its own short definitions of "valid", "pure" and "loop function". A careful reader had to trust that these match the standard ones, which are written with *process matrices*: big matrices that encode a whole time-loop machine. The check also allowed each agent only a finite side system (ancilla).
- **The standard language, now checked [Lean ✓].** We wrote the standard definitions into Lean (lean/PFUL/ProcessMatrix.lean). A process matrix is valid if plugging in any allowed local operations, with any finite side systems, gives a proper channel. It is pure if plugging in reversible operations gives a reversible whole. We then proved that the note's own definitions say exactly the same thing. The main theorem is restated using only the standard definitions (`purification_conjecture_false_choi` L-092). One small bonus is that, for ordinary classical processes, "valid against coin flips" turns out to mean "valid against every quantum operation with side systems".
- **Infinitely large side systems [Lean ✓].** What should "pure" mean if an agent's side system is infinite-dimensional, like a particle's position? The natural answer: for every such side system and every reversible local operation, the whole loop is still reversible. Lean now proves this for the Switchboard for side systems of *any* size, countable or not (`isPureHilbert_Vsb` L-099). It also proves that the Switchboard stays valid when agents use any well-behaved noisy operation with countably infinite side systems (`valid_normalCPTP_Vsb` L-101).
- **Finite is enough [proved on paper; now Lean ✓].** Any loop machine that is pure for all finite side systems is automatically pure for infinite ones too. The reason is that each check only ever looks at finitely many directions of the side system at once. This was on paper only when this note was written; it is now checked in Lean for every three-party binary loop machine and side systems of any size (`PFUL.Counterexample.isPureHilbert_of_isPure` L-191). The Switchboard does not need it, because it is checked directly.
- **What is still open.** A version of the standard matrix pairing that works directly with infinite

side systems. There, the usual matrices are too large to have a trace, and Lean uses an equivalent "Kraus" description instead. Two more items are also open: the paper-only step above, and noisy operations with uncountably large side systems.

88. Plain-language note: closing the formal gaps (Oct 24 2026)

- **What was open (note 87).** Four technical items were left after the last round: (a) the standard matrix pairing ("link product") with infinitely large side systems; (b) a computer-checked proof that "pure for finite side systems" implies "pure for infinite ones"; (c) the standard fact that pure process matrices are exactly those built from a single reversible operation (Araújo et al., Theorem 2); and (d) noisy local operations on side systems too large to be listed one by one (uncountable).
- **(b) Finite is enough, now checked [Lean ✓].** For every loop machine, being pure for all finite side systems is the same as being pure for side systems of any size. The key step squeezes any reversible operation on an infinite side system into a finite one that looks the same on the few directions a given check uses.
- **(c) Pure means "one reversible operation" [Lean ✓].** A process matrix is pure exactly when it is valid and is the matrix of one reversible operation. This holds when each agent's input and output have the same size, and, for the converse, when the source and sink have the same size. Araújo et al. state it more generally; that general form is still only cited.
- **(d) Any size of side system for noisy operations [Lean ✓].** The earlier proof needed side systems that can be listed one by one. That step is now replaced by a general existence theorem, so the size no longer matters.
- **(a) The pairing with infinite side systems [Lean ✓, in matrix-entry form].** Validity and purity can now be defined directly with side systems of any size, and they agree with the finite definitions. The counterexample holds in this form too. The pairing is done entry by entry. It is not a statement about the infinite "Choi operators" themselves, which are too large to have a trace.
- **Qubit times side system [Lean ✓].** Each agent's space "qubit ⊗ side system" is now the real tensor product, not just a space with the right kind of basis. The joint space of all three side systems is still modelled by its basis. The reason is that Lean's library has no completed tensor product of infinite-dimensional spaces.
- **Still open.** A pairing stated for the operators themselves; Theorem 2 for unequal sizes; and the old Round 9 item, that positive trace-preserving maps are automatically bounded. That last item needs a spectral theorem that Lean's library lacks. None of these affects the main result.

89. Plain-language note: the second counterexample on an ordinary clock (Oct 24 2026)

- **What was open (notes 84–85).** The simpler counterexample (H.13.2) already had an explicit "ordinary clock" circuit: a normal, loop-free quantum circuit in which each agent still acts exactly once, on a system spread over several times. The first counterexample (H.12) had only an existence theorem from the literature.
- **A hidden family resemblance [proved].** H.12 turns out to be built the same way as H.13.2. H.13.2 is the well-known three-party "Lugano" loop run backwards in time and viewed in a rotated basis. H.12 is the same, except that one agent's bit flip is replaced by a "half flip" (the square root of a flip): a quantum gate that, applied twice, flips the bit. This was checked exactly, with whole-number arithmetic.
- **An explicit circuit [checked numerically to about 10^{-15}].** So H.12 also runs on an ordinary loop-free circuit, the same one as H.13.2 with one gate changed from "flip" to "half flip". We also ran the general recipe from the literature step by step and got a second, messier circuit that agrees

to about 10^{-14}.

- **A valid wiring diagram [proved, using a cited theorem].** The "routed graph" picture of Vanrietvelde and co-authors, which certifies that a loop is logically consistent, covers H.12 too. Their Lugano diagram works if one gate is filled in with the half flip, and they already allow such fillings.
- **What this corrects.** An earlier note said the H.13.2 construction does not carry over to H.12. It does, once the flip may be a genuinely quantum gate.

90. Plain-language note: a path between the counterexamples, classical stand-ins, and continuous dials (Oct 24 2026)

- **A dial from ordinary to strange [checked numerically].** The two counterexamples sit on one continuous dial. The dial sets the gate in one agent's place: "do nothing" at one end, "half flip" (H.12) in the middle, "full flip" (H.13.2) at the other end. Every setting gives a legitimate pure process. At "do nothing" the statistics are an ordinary mixture of consistent histories. Every other setting we checked (in steps of 1/24 of the range) is not, and the violation grows steadily along the dial. Some of the tests of "ordinariness" (the "never-crossed" facets) are never failed anywhere on the dial. That fits the suspicion that those tests hold for all pure processes, but it does not prove it.
- **Can a classical machine fake it? It depends on the rules [proved].** If each agent's box may be run twice, once for each possible input, with a referee deciding afterwards which run counts, then yes: any consistent classical process can be faked, the counterexamples included. That is easy and almost certainly folklore, and it says nothing about purity. If instead each agent acts exactly once and every step is reversible and classical, then no: such a machine only produces mixtures of consistent histories, which the counterexamples are not. Rules in between remain open.
- **Continuous dials and Fourier transforms [proved].** For continuous outputs (real numbers rather than bits), it was open how to close a non-causal loop when the agents do something like a Fourier transform. If the loop only reads which of finitely many "cells" each output lies in, as in the Lugano loop with thresholds, the answer is a finite sum of ordinary operators, and it is unitary. Fine-grid approximations converge to it under the same assumption as in the causal case. Loops that read the outputs in a genuinely continuous way remain open.

91. Plain-language note: the score gap for the first counterexample shrinks (Oct 24 2026)

- **The question (note 82).** How badly can the first counterexample (H.12) beat a "causal" score? The best strategy found scores -0.7058. The best proved floor was -1.5856, so the true answer lay somewhere in between.
- **A stronger floor [proved, exact integer arithmetic].** The old floor treated agent A's two possible measurements as unrelated. A stricter test makes both of A's measurements live side by side, on two copies of A's system, as any real strategy allows. With that requirement no strategy can score below -1.2772. The certificate is checked with whole numbers only, so no floating-point trust is needed.
- **Searching from above [checked numerically].** 120 more random searches did not beat -0.7058.
- **Where it stands.** The true best score lies between -1.2772 and -0.7058, both ends proved. The gap is about a third smaller than before, but it is still open. The method itself is a standard idea; only the numbers are new.

92. Plain-language note: how many times must a classical machine run each box? (Oct 24 2026)

- **The question (note 90).** A classical machine can fake either counterexample if it may run every agent's box once for each possible input and decide afterwards which run counts. It cannot if every box runs exactly once. What lies in between?
- **Counting runs settles it [proved, using a 2012 result of Oreshkov, Costa and Brukner].** With three agents and yes/no inputs, running *one* agent's box twice and the other two once each is enough, for every consistent classical process. The trick: once the machine knows how that one agent reacts to both possible inputs, the other two face an ordinary two-party situation, and two-party classical situations always have a definite (possibly random) order. Three runs, one per agent, are never enough for the counterexamples, by the earlier lemma. So the exact budget is four runs.
- **A curious match.** The quantum circuits that run the counterexamples on an ordinary clock (notes 84 and 89) also use one agent twice and the others once. The classical machine has to throw one of its runs away; the quantum circuit throws nothing away.
- **Still open.** A classical machine that runs every box once, on "spread-out" classical variables, and never throws anything away. We do not yet have a precise definition of that.

93. Plain-language note: the H.12 score range narrows again (Oct 24 2026)

- **Where we were (note 91).** The best score y_A that H.12 can reach was known to lie between -1.2772 (a proved floor) and -0.7058 (the best strategy found).
- **A better floor [proved, exact arithmetic].** The floor came from giving one agent's two measurements a common home. Doing this for agent B instead of agent A gives a higher floor, -1.20330, again checked with exact integers. Doing it for agent C gives a weaker floor (about -1.30), so it was not certified.
- **No better strategy [numerical].** Restricting all measurements to 13 symmetric directions and searching 3,000 combinations, then polishing, found nothing below -0.7058.
- **So** the true value lies in $[-1.2033, -0.7058]$. It is still open.

94. Plain-language note: ten "never-crossed" constraints fall (Oct 24 2026)

- **The list (notes on H.13–H.17).** Among the inequalities that fence in all consistent classical time-travel processes, 22 families had never been broken by any quantum loop we tried. We wondered whether they bind every reversible quantum loop.
- **Ten of them do not [proved, exact arithmetic].** Take the time-reversed Lugano loop of Guérin and Brukner (2018), and let each agent read and write its qubit in a tilted basis: one tilt for what it reads and an independent one for what it writes. With the right tilts the resulting classical statistics break ten of the 22 families. We checked this with exact fractions: the loop is exactly reversible, the statistics are exactly a valid classical process, and an inequality value is exactly negative.
- **Why it was missed.** Earlier searches only tilted what agents read, or used special tilts. Tilting what they write is what breaks the pattern. The loop itself is well known; what changes is only how it is read.
- **Two more [proved, exact arithmetic].** Letting the three gates inside the Lugano loop be general rotations instead of bit flips breaks two further families (53 and 79). This loop is consistent by the same routing argument as before (Vanrietvelde et al.).
- **Still open.** Ten families remain unbroken.

95. Plain-language note: a missing tool was not missing (Oct 24 2026)

- **The claim (Round 13).** One remaining gap in the computer-checked proofs was said to need a theorem (diagonalising compact symmetric operators) that the Lean maths library lacks.
- **The fact [checked in the library].** The library has had it since early 2026, and our own Lean files already use it. The gap is still not closed, but it is now ordinary work rather than a missing foundation.

96. Plain-language note: the last gap in the machine-checked infinite-dimensional story is closed (Oct 24 2026)

- **The gap (Rounds 8–15).** The computer-checked theorems about channels on infinite-dimensional systems assumed that a channel does not blow up the size of what it acts on ("boundedness"). On paper this is automatic for any map that keeps positive things positive and preserves total probability, but it had not been checked by machine.
- **Now it is [proved, Lean].** Split any self-adjoint operator into a positive part and a negative part. The map preserves the size of each, so it can at most add them up; for a general operator, split it into two self-adjoint pieces and lose at most a factor of 2. The library's existing "functions of operators" machinery gives the split directly, so the diagonalisation theorem discussed in note 95 was not even needed.
- **What it buys.** Every channel on any system, of any size, now counts in the machine-checked results, with no extra hypothesis: in particular the H.12 Switchboard is consistent against all of them.
- **Not new mathematics.** The fact itself is classical (Russo and Dye, 1966, give the sharp factor 1); only the machine check is new.

97. Plain-language note: the twelve broken families are broken everywhere; ten remain (Oct 24 2026)

- **The question (note 94).** A "family" of inequalities is all the copies of one inequality obtained by relabelling the agents and flipping their bits. We had broken one member of each of twelve families. Is every member broken?
- **Yes [proved, exhaustive check].** The relabellings that define the families are exactly agent swaps and bit flips, and each of these turns a consistent reversible loop into another one (flip a wire with a NOT gate, or rename the agents). So breaking one member breaks them all.
- **Why the old searches could not reach the remaining ten [proved].** Every "XOR-type" classical loop, where each agent receives the source bit flipped by a function of the others' outputs, is either causal or a relabelled Lugano loop. So the tilted Lugano family already covers that whole class.
- **New searches [numerical].** More random tilted Lugano loops, 200 reversible classical loops of a more general type (none of which turned out to be consistent quantum loops), and an unrestricted search over all three-qubit loops (which did not manage to stay consistent) found nothing.
- **Still open.** Ten families (26, 28, 30, 31, 54, 57, 66, 69, 75, 81).

98. Plain-language note: reversible loops with continuous dials, one step further (Oct 24 2026)

- **The question (Appendix F).** If the agents' dials are continuous, for example real numbers, and the loop is not causally ordered, does every choice of local reversible operation (including

"wave-like" ones such as a Fourier transform) still give one well-defined reversible whole? Earlier rounds proved yes when the loop only reacts to which of *finitely many* zones each dial is in.

- **New [proved, paper].** Yes also when it reacts to which of *countably many* zones each dial is in, even if the loop's responses grow without bound. The proof is shorter than the old one. Testing the whole on independent states of the agents turns the question into an ordinary probability statement: "a consistent loop has exactly one history". A standard algebraic trick then extends the answer to all states.
- **Example [proved; checked numerically].** A Lugano-type loop on the integer parts of three real dials, with ever larger jumps. It is not causally ordered and does not fit the finitely-many-zones case. A computer test with 3,000 random choices always found exactly one consistent history.
- **Still open.** Loops that respond continuously, or in uncountably many ways, to the dials.

99. Plain-language note: a better floor for the first counterexample (Oct 24 2026)

- **Where we were (note 92).** The best score y_A that H.12 can reach was known to lie between -1.2033 (a proved floor) and -0.7058 (the best strategy found).
- **A better floor [proved, exact arithmetic].** Earlier floors came from giving one agent's two measurements a common home, first for agent A and then for agent B. Now both are done at once, with the requirement that the two pictures agree on every observable statistic. The floor rises to -1.16564, and a computer certificate in exact integer arithmetic confirms it.
- **So** the true value lies in $[-1.1656, -0.7058]$. It is still open.

100. Plain-language note: continuous dependence in the infinite-dimensional lift (Oct 24 2026)

- **Where we were (note 98).** For a process function on continuous outputs whose dependence on the outputs can be described with countably many cells, the closed loop with any local unitaries is a well-defined unitary.
- **New [proved, paper].** Suppose the dependence is continuous, but can be approximated uniformly by such cell-based functions. Then the closed loops of the approximations converge, the limit does not depend on the approximation chosen, and it preserves lengths (an isometry). Example: the Lugano pattern, where each translation amount is any smoothly varying function of the other outputs.
- **Still open.** We have not shown that the limit is reversible (unitary). That would follow if the reversed loops did not leak probability towards ever higher momenta. We have no example where they do. Process functions that cannot be approximated this way are also still open.

101. Plain-language note: why the ten stubborn orbits resist one more family (Oct 24 2026)

- **Question.** Can boards beyond the "XOR" kind cross any of the ten facet orbits that no strategy has crossed so far?
- **Test [numerical].** We let each party apply a qubit rotation chosen by the other parties' classical outputs. Almost every way of choosing makes the process inconsistent. The only consistent pattern we found is the Lugano one: one rotation when the party is "activated", another otherwise. Those boards were already searched in Round 16, so this family brings nothing new.
- **So** the ten orbits are still open. Neither a crossing nor a proof was found.

102. Plain-language note: the best strategy for the first counterexample looks final (Oct 24 2026)

- **Search [numerical].** We ran about 6,000 fresh optimisations of the measurements in H.12, some general and some restricted to the simplest (rank-one) measurements. Every round found the same best score, -0.70580, and nothing lower.
- **So** the best strategy found in Round 14 is very likely optimal. The proved range is still $[-1.1656, -0.7058]$, because a search can only show what is achievable, not what is impossible.

103. Plain-language note: the continuous Lugano loop is reversible (Oct 24 2026)

- **Where we were (note 100).** With continuous outputs and smoothly varying shifts, the closed loop of the Lugano-type example was known to preserve lengths, but not to be reversible.
- **New [proved, paper].** It is reversible (unitary), and the shifts can even be arbitrary measurable functions, as long as one party at a time is "activated" and only the activated party's shift varies. When we check that the loop preserves lengths, the shifts cancel out completely, so their form does not matter.
- **Still open.** Process functions in which several parties' shifts vary at once.

104. Plain-language note: the best H.12 strategy is a certified local optimum (Oct 24 2026)

- **Shape of the strategy.** Each party simply announces its own setting as its output, and measures its qubit along one direction per setting.
- **Certified [proved, interval arithmetic].** A computer-assisted proof shows that an exact optimum of this kind exists, with value $-0.705802238872487294515\ldots$. Every small change of the six measurement directions makes the score worse. Numerically, no single party can do better with any measurement, given what the others do.
- **Still open.** Whether some completely different strategy does better. The value is not a simple closed-form number.

105. Plain-language note: what a proof for the ten stubborn orbits would need (Oct 24 2026)

- **Checked [proved, exact].** Each of the ten remaining inequalities fails on some consistent classical process, by exactly 1/6 of its range. So no argument that uses only "the process is consistent" can prove them. A proof would have to use that the process is pure (reversible), and in a way that convex methods such as semidefinite programs cannot capture directly.
- **Tested [numerical].** Adding an entangling gate to the Lugano-type processes, even just phases, always makes them inconsistent. So the natural one-qubit-per-wire family of candidates is already exhausted by earlier searches.
- **So** the ten orbits are still open.

106. Plain-language note: four terms really is the minimum (Oct 24 2026)

- **Where we were.** Boards with at most three Pauli terms per outcome had been shown to be classical mixtures in every case except boards with four distinct "classes", and those had been reduced to a finite search over 2,722 shapes.

- **New [proved, exhaustive computer search].** That search has finished. All 2,722 shapes were checked, with no counterexample and no case left undecided. So in our model of how the terms can combine, every board with at most three terms per outcome is a mixture of classical consistent histories. Four terms are needed, and four are enough (H.13.2).
- **Caveat.** The search uses a slightly generous description of which term patterns can occur. That can only add cases, so a clean result is sound for the model. The enumeration has not been checked in Lean.

107. Plain-language note: a slightly better floor for the first counterexample (Oct 24 2026)

- **New [proved, exact computer certificate].** Combining all three "copied party" relaxations, not just two, shows that the score of the first counterexample's causal-inequality test can never go below -1.1586 (before: -1.1656).
- **So** the true value lies in $[-1.1586, -0.7058]$. The gain is small, so adding more copies of the same kind is unlikely to close the gap.

108. Plain-language note: double-checking the "four terms" search (Oct 24 2026)

- **Audit [proved, exhaustive check].** An independent script recounted the shapes and matched them against the search's record. There are 21,906 labelled shapes, and up to symmetry exactly 2,722. Every one of the 2,722 was searched, with no counterexample and nothing left undecided. The record also holds 167 extra shapes from an early run that went through the unsymmetrised list; they are clean too.
- **So** note 106 stands: the counts (2,722 shapes; 2,896,784 sub-cases) are correct.

109. Plain-language note: the best H.12 strategy is the best of its kind (Oct 24 2026)

- **Where we were (note 104).** The best strategy was certified as a local optimum: small changes make it worse.
- **New [proved, exact certificates for all 64 output choices].** Consider all strategies of the same kind: each party picks its output from its setting, and measures its qubit along one direction per setting, possibly with noise but without bias. A sum-of-squares certificate proves that none of them beats -0.70607, and -0.70580 is achieved. Numerically the gap closes completely, and the runner-up of this kind is the other local optimum we had seen, -0.6597.
- **Still open.** Strategies of a different kind, such as outputs that depend on the measurement result, are not covered. Over all strategies the range is still $[-1.1586, -0.7058]$.

110. Plain-language note: how a computer can prove "there is no counterexample" (Oct 25 2026)

- **The claim (note 106).** Every board with at most three Pauli terms per outcome is a classical mixture, at least in our model of how the terms can combine. A proof has to rule out every possible board, and there are trillions of them.
- **Step 1: make the list finite and small.** Earlier lemmas show that a counterexample would need exactly four "classes", each sitting on one edge of the cube of outcomes. That leaves 21,906 possible shapes. Many of these are the same shape seen from a different angle: rename the parties, flip which outcome is called 0, or rotate a qubit by a fixed gate. Those changes cannot turn a classical

mixture into a non-mixture, so it is enough to check one shape from each family (the *canonical representative*). That leaves 2,722 shapes.

- **Step 2: check each shape completely.** For each shape the computer tries every way the terms can combine (a *job*), and every choice of the seven remaining Pauli labels. It skips only choices that provably cannot work. A job is *UNRESOLVED* if it runs out of its step allowance before finishing; none did. A *candidate* would be a board that passes every test a counterexample must pass; none appeared, across 2,896,784 jobs.
- **Step 3: check the checking.** A separate *audit* recounted the 2,722 families, confirmed that each had been searched, and caught a bookkeeping trap: the record files number shapes by their position in the long list of 21,906, not by family. All the arithmetic is exact (bits and yes/no tables, no rounding), and the list of shapes regenerates byte for byte.
- **A worked example (shape 3424, H.26.1a(e)).** Three classes each act on one qubit (A, C and B), and the fourth is the identity. Only one job arises. It has exactly 64 valid boards. All are "hard", so the easy arguments do not apply, but every one passes the identity-class test. For the simplest board, every pair of outcomes from different classes changes some party whose qubit neither class touches, so the two outcomes' terms can never conflict. The computer confirmed the test for all 64, which makes each board a classical mixture. The search really did meet hard boards here and dismissed each for a reason.
- **What it does not say.** The result depends on the model (each pair of classes gets every term pattern it could have), and on the code being right, which has been cross-checked but not formally verified. See H.26.1a(g).

111. Plain-language note: the ten stubborn inequalities, half-closed (Oct 25 2026)

- **The question (H.27).** Ten families of inequalities on classical time-travel statistics have never been broken by any quantum loop we know. Either some quantum loop breaks them, or there is a proof that none can. Round 20 tried both directions.
- **What is now proved.** Many quantum loops work like this: each party's lab holds a quantum device, and a classical "consistent rule" (a process function) decides which knob setting each device gets, based on what the others sent. Whatever happens inside the devices, in any dimension and with any gates, the statistics are those of a shared quantum system measured with settings chosen by that rule. We proved, with exact fractions and no rounding, that no such arrangement breaks any of the ten inequalities. That holds even if the devices are allowed super-quantum correlations, as long as they cannot be used to signal. A computer listed every possible wiring rule (20 that matter) and produced, for each rule and each of the ten families, an exact certificate.
- **What was searched.** The loops that broke the other twelve families run the same construction backwards in time. We searched that reversed family, now with larger quantum systems than before. On the twelve families already known to be breakable, the search found the known violations, and on some it found larger ones. On the ten stubborn families: nothing. The best value on each of the ten was exactly the borderline, never past it. This search covered 8 of the 20 wiring rules before it was stopped.
- **What it does not say.** A search that finds nothing is not a proof: in the test runs, hitting a known violation sometimes took about 300 random starts. The ten families are still open for quantum loops in general. What is new is that any loop breaking them must use backward-running coherence that the forward construction lacks.

112. Plain-language note: the five-class search finished, and found nothing, as it should (Oct 25 2026)

- **The question (H.17.16c).** Can a three-party quantum loop beat the classical ones while using at most three Pauli terms per outcome? One way to try is to give the board five different "classes" (kinds of local unitary), all drawn from one six-member family built around a Klein group. A computer checked every way of doing this, up to symmetry.
- **What finished.** The run took about 37 hours on four cores and checked 104 blocks, 38,135,448 cases in all. It found no candidate, and no case was left undecided. A separate audit script recounted every block from scratch with a different method and found exactly the same numbers, with no block missing or repeated. The arithmetic is all whole-number bit operations, so rounding plays no part.
- **Why "nothing" was expected.** An earlier pen-and-paper result (the class bound, also checked in Lean) says a board of this kind can have at most four classes. So five-class boards cannot exist, and the search was asking about an empty set. The four-class case, which is the one that matters, was finished in Round 19.
- **What it adds.** The run is an independent cross-check. It used a different program and a different way of cutting up the problem, and it did not assume the class bound. Had it found a five-class board, something in the theory or the model would have been wrong. It did not.
- **What it does not say.** The six-class version was not run (it is estimated at four times as many cases or more), though the class bound rules it out too. The search programs themselves are not formally verified, and the result holds in the same simplified "one support per pair of classes" model as the earlier searches.

113. Plain-language note: three of the ten stubborn inequality families are broken; seven remain (Oct 26 2026)

- **The question.** Twenty-two families of inequalities (facet orbits) had never been broken by any quantum loop we built. Round 15 broke twelve. The other ten (orbits 26, 28, 30, 31, 54, 57, 66, 69, 75, 81) looked like they might hold for every reversible quantum loop.
- **What is new (H.28).** Three of the ten, orbits 26, 30 and 31, are broken. The loops that break them are very simple. Each party gets its own qubit straight from the past. Whatever bit it sends on to the future picks up a phase, and that phase depends on its own bit and on what the other two received, routed by one of the consistent classical wiring rules. These "phase boards" are a special case of the time-reversed construction from Round 20, so they are legitimate quantum processes. The famous H.12 example turns out to be one of them, with phases fixed at quarter and half turns. Letting the phases take any value is what breaks the three new families.
- **How sure.** Each violation was converted to exact fractions and checked by two separately written programs. Both confirm that the loop is exactly a unitary, that it passes the exact validity test for quantum processes, that every outcome probability is non-negative, and that the inequality is broken. Orbit 26 is broken only barely, by about 2.4 parts in 100,000, but exactly. Orbits 30 and 31 are broken by about 0.9%. Relabelling parties, inputs and outputs carries a broken inequality to every inequality in its family, and we checked this directly: all 192 inequalities in each of the three families are broken.
- **The other seven (28, 54, 57, 66, 69, 75, 81).** Every search came back empty. That covers phase boards on all wiring rules, the general time-reversed family on all 20 rules, random hops around the best points, and a small test with outcome-dependent preparations and mixed forward/backward wiring. The best value was always exactly the borderline. Forward-wired loops are already proved

never to break them.

- **What it does not say.** Empty searches are not proofs. In fact the general search, which contains the phase boards, missed orbits 30 and 31 entirely, so a better-aimed family can succeed where a broad search fails. Whether the seven families hold for all quantum loops is still open. The tally is now 15 of 22 broken and 7 open.

114. Plain-language note: why the broad search missed, and a full map of phase loops; the seven still stand (Oct 26 2026)

- **The question.** Round 21 left seven families of inequalities (orbits 28, 54, 57, 66, 69, 75, 81) that no reversible quantum loop had broken. Round 22 tried to break them with wider families of loops, and asked why the broad search of Round 21 had missed the breaks that the simple phase boards found.
- **Why the broad search missed (H.29.2).** In the time-reversed family, everything that matters about each party is a small table of overlaps between unit vectors. The inequality depends on each table in a straight-line way, and the best value of a straight-line function over such a set lies at its corners. For parties with at most three cases, the corners are exactly the phase boards. A broad search wanders in the middle of the set, and a phase search sits on the corners. So for most wiring rules the phase boards are the whole story for that family. For the six rules where one party has four cases, the corners are slightly richer, and we searched those too. Nothing broke.
- **A rule for a wider family (H.29.1).** For loops whose future simply records what the agents sent, we derived an exact test of when the loop is a legitimate process. In words, an agent's output can never reach its own input. The test agrees with the standard one everywhere we checked.
- **A full map of phase loops (H.29.3).** When the loop only adds phases, the test splits into simple either/or conditions: for each pair of agents and each setting of the third, one of the two must not influence the other. Listing all ways of satisfying these gives 34 families of legitimate phase loops. Eight are the phase boards that broke orbits 26, 30 and 31. Eight are new: in them, two agents' outputs jointly steer a phase on the third agent's input. Eighteen mix the two kinds.
- **Searching the map (H.29.4).** Inside each family the loop is legitimate by construction, so the search needs no penalty. It finds the known breaks of orbits 26 and 30 again, at the exact known values, and only in the phase-board families. On the seven open orbits it finds nothing in any of the 34 families. Two older-style searches, which enforce legitimacy only through a penalty, could not even find the known break of orbit 30, so their empty results count for little. We report that plainly.
- **What it does not say.** Empty searches are not proofs, and the map covers phase loops on qubits only. The tally is unchanged: 15 of the 22 families broken, 7 open. A certified bound that uses purity, which is what would settle the question in the other direction, was not attempted this round.

115. Plain-language note: a first proof for the seven, away from the Lugano loops (Oct 26 2026)

- **The question.** Seven families of inequalities (orbits 28, 54, 57, 66, 69, 75, 81) have never been broken by a reversible quantum loop. Round 23 tried to *prove* that they cannot be, using the fact that the loops are pure.
- **The idea (H.30.1).** For a given family of loops, write the outcome probabilities as a sum over pairs of hidden states. Each term factors into pieces, and each piece can only take certain shapes: a measurement in a rotated frame either has a constant part or does not, and a phase either cancels

or does not. So every probability table the family can produce lies in a fixed, fairly small linear space, of dimension 20–24 out of the 38 available. We computed that space exactly.

- **The proof (H.30.2).** Inside that smaller space, a linear program can check whether an inequality can fail at all. For every phase loop outside the eight Lugano-type families, and for the time-reversed loops on the twelve rules where one party acts first, it cannot. All 1,536 inequalities of the ten families that were open before Round 21 hold there, the seven included. Two separately written programs prove each case in exact fractions. A float check suggests something stronger: these loops only ever produce mixtures of classical process functions.
- **The map is now complete (H.30.3).** Last round's list of 34 families of legitimate phase loops relied partly on random spot checks. It is now proved exactly. The two leftover cases were loops in which each agent steers the next one around a cycle, and they turn out to be legitimate only where one link of the cycle is switched off, which puts them inside the known families.
- **Where the question now lives.** On the Lugano-type loops the probability tables fill the whole space, so this argument says nothing there. That is also exactly where every known break of any of these inequalities sits. Proving the seven there would need the finer, quadratic structure of those loops, and that was not attempted.
- **A search that failed its test (H.30.4).** We also started searches at the worst classical-looking point for each inequality and pushed them toward legitimate loops. They did not re-find the two known breaks, so their empty results on the seven count for little.
- **The tally.** Unchanged: 15 of the 22 families broken, 7 open. But the seven are now proved to hold everywhere outside the Lugano-type loops that we have studied.

116. Plain-language note: two of the seven are now proved on the Lugano loops too (Oct 26 2026)

- **The question.** After Round 23, the seven unbroken families of inequalities (orbits 28, 54, 57, 66, 69, 75, 81) were proved everywhere we had looked except on the Lugano-type loops. On those loops a linear argument cannot work, because their probability tables fill the whole space.
- **Sixteen cases are really one (H.31.1).** There are eight Lugano-type phase families and eight Lugano-type routing rules. Relabelling a hidden bit, re-choosing a measurement frame, or absorbing a phase into the starting state turns every one of them into the same family: phase loops on one rule, described by nine angles. The same symmetries show that checking one inequality per family is enough.
- **Using purity (H.31.2).** The loops are pure quantum processes. So their probability tables come from a positive semidefinite "lifted" matrix, and its entries repeat in a fixed pattern that the phases dictate. Asking only for these properties gives a semidefinite program, which bounds how far any inequality can be broken. We first tested it on the three families that *are* broken on these loops. It correctly failed to prove them, and its bounds sat below the known breaks.
- **The proof (H.31.3).** For orbits 54 and 66 the program's answer is exactly zero: they cannot be broken. We turned the floating-point answer into an exact certificate in fractions. The step that makes this possible is that the certificate's null directions turn out to be simple rational vectors. Two separately written programs check it. Together with Rounds 20 and 23, orbits 54 and 66 now hold on every family of loops studied in the paper. That is a statement about those families, not about every pure process. On all pure processes, the count is still 15 of 22 broken and 7 open.
- **Close but not proved (H.31.4).** For orbit 57 the program also says zero, to ten decimal places. But its certificate would not round to exact fractions, so this one stays numerical. Orbit 75 is like 57, not like the rest. The program's answer is near zero (about $-3\cdot10^{-7}$), but the solver flagged it as inaccurate, so it is not proved either. Round 25's last attempt on it also came back inaccurate and

told us nothing, and 75 probably marks a weakness of the program rather than a near-proof (note 117). For orbits 28, 69 and 81 the program is too weak to decide. (Note 118: a different program now limits any break of 57 on these families to $2.6 \cdot 10^{-8}$.)

- **A search that passed its test (H.31.5).** We also searched the nine-angle family directly. Every angle setting is a legitimate loop, so no penalty tricks are needed. This search re-finds all three known breaks exactly, including the rare one (orbit 26, found by about one start in 2,000). On the seven it finds no break.
- **The tally.** Unchanged on all pure processes: 15 of 22 broken, 7 open. But on the families studied, only five (28, 57, 69, 75, 81) are still open, and all five sit on one nine-angle family.

117. Plain-language note: a sturdier proof for orbit 66, two near-misses, and a second search (Oct 27 2026)

- **Where we were.** After Round 24, five of the seven unbroken families of inequalities (orbits 28, 57, 69, 75, 81) were still open on one nine-angle family of Lugano-type loops. Orbits 54 and 66 were already proved there.
- **Where a proof must be tight (H.32.1).** If an inequality is exactly tight at some loops, any exact proof must "vanish" in the directions those loops point to. Those directions can be read directly from the loops, with no solver.
- **A sturdier proof for 66 (H.32.2).** If we build these directions in from the start, orbit 66 has a certificate with lots of room to spare, and two independent programs check it in exact fractions. This is not a new result: 66 was already proved. But it is a second, independent proof, and it is much less fragile.
- **57 and 75 don't give way (H.32.3–H.32.4).** For orbit 57 the same method gets to within about 10^{-7} of zero on every one of the 190 facets we could test (out of 192), but never clearly above zero. Its proof, if one exists, seems to need irrational numbers. For orbit 75 the margin is slightly negative, which suggests our relaxation is just too weak for it. We stopped here, by design, and both stay numerical.
- **A second search (H.32.5).** We started from a fixed grid of angle settings and slid downhill from the best points in every cell. This search re-finds all three known breaks, including the rare one on orbit 26. It finds no break of the five. That test is what makes the "no break" meaningful: a search that could not find the known breaks would prove nothing by finding none.
- **The tally.** Unchanged: 15 of 22 broken on all pure processes, 7 open. On the families studied, five are open (28, 57, 69, 75, 81).

118. Plain-language note: a near-proof for orbit 57 (Oct 27 2026)

- **What we tried (H.33.1).** We rewrote the inequality as "this expression is never negative", with the loop's settings as complex variables, and looked for a proof that writes the expression as a sum of squares. Such a proof cannot be wrong in the direction that matters: if the squares add up, the inequality holds.
- **The test (H.33.2).** The method proves orbits 54 and 66, which were already proved. It does not "prove" orbit 30, which is known to be broken: it reports about -0.019 there, below the true depth of the break. A method that proved a broken inequality would be wrong. This one passes that test.
- **Orbit 57 (H.33.3).** For orbit 57 the method's answer is zero to about nine decimal places, just as for the two proved orbits. We could not make it an exact proof of "never negative", because the solution has no clean structure to round to. What we could prove exactly, checked by two separately written programs in fractions, is that on every family of loops studied, orbit 57 can be

broken by at most $2.6{\cdot}10^{-8}$. The only known break on this family, orbit 26's, is a thousand times deeper. So 57 is either true there or broken by a hair. It is still open.

- **The other four (H.33.4).** On 28, 69 and 75 the solver did not settle within its time limit, and 81 was not run. Nothing new.
- **The tally.** Unchanged: 15 of 22 broken on all pure processes, 7 open. On the families studied, five are open (28, 57, 69, 75, 81), and 57 is now pinned to within $2.6{\cdot}10^{-8}$ of holding.

119. Plain-language note: orbit 57 proved on every family studied (Oct 27 2026)

- **What changed (H.34).** Round 26 got within $2.6{\cdot}10^{-8}$ of a proof for orbit 57 but could not close the gap. Round 27 closes it: the expression is now written exactly as a sum of squares, with fractions throughout, so it is never negative on the loops of the reduced family.
- **How (H.34.1–H.34.4).** First, a symmetry of the problem splits each of the big matrices into small pieces of size at most 22. Second, the places where orbit 57's expression is exactly zero turn out to form an explicit family, which tells us exactly which directions every proof must leave out. Third, one more such direction was hiding in twelve of the pieces. Once those are removed too, the solver finds a proof with room to spare (margin $1.2{\cdot}10^{-3}$), and room to spare is what lets a floating-point answer be rounded to an exact one.
- **The check (H.34.6).** Two separately written programs check the rounded proof in exact fractions: every coefficient matches, and every piece is positive definite. Both programs reject the proof if the claimed bound is raised by 10^{-6}, or if a single entry is changed by 10^{-12}.
- **What it means.** Like 54 and 66, orbit 57 now holds on every family of pure processes studied here. It is still open on pure processes in general.
- **Julia.** Julia and its Clarabel solver were installed for high-precision solving. On the full problem that solver was too slow here: about 8 steps in 30 minutes, against 80 seconds for the Rust version of the same solver. In the end high precision was not needed. The proof was found with ordinary double precision on the smaller reduced problem, then made exact with fractions in Python. Julia played no part in the final proof.
- **The tally.** On all pure processes it is unchanged: 15 of 22 broken, 7 open. On the families studied, four are open (28, 69, 75, 81).

120. Plain-language note: orbit 75 proved on every family studied (Oct 27 2026)

- **What changed (H.35).** Orbit 75 now has the same status as 54, 57 and 66: it holds on every family of pure processes studied here. It is still open on pure processes in general.
- **How (H.35.1–H.35.3).** Orbit 75's expression is zero at exactly the same places as orbit 57's. The Round 27 trick for 57 (removing one more hidden direction) did not work here. But the solver's answer turned out to be very simple: of more than a thousand possible directions it uses only 12, each with small whole-number coefficients. Taking those 12 at face value writes the expression exactly as a sum of 12 squares, with weights 1/6 and 1/3. No correction step was needed.
- **The check (H.35.4).** The two exact checkers from Round 27, unchanged, accept the proof. They reject it if the bound is raised by 10^{-6}, if one weight is changed by 10^{-12}, or if one weight is made negative. A separate floating-point test compares the sum of squares with the original quantity at 2,000 random points and agrees to about 10^{-15}. A rerun from a different random seed gives exactly the same proof.
- **The other three (H.35.5).** For 28, 69 and 81 the same method gives a strictly negative bound, so it cannot prove them as it stands. For 28 and 81 the shortfall is large, about 0.01 to 0.03. For 69 it shrinks to about $3{\cdot}10^{-7}$ when more terms are allowed, so 69 might be provable with a more

accurate solver. Searches with 4,600 random starts per orbit found no point that breaks any of the three.

- **The tally.** On all pure processes it is unchanged: 15 of 22 broken, 7 open. On the families studied, three are open (28, 69, 81).

121. Plain-language note: orbits 69 and 81 proved on every family studied (Oct 27 2026)

- **What changed (H.36).** Orbits 69 and 81 now have the same status as 54, 57, 66 and 75: they hold on every family of pure processes studied here. They are still open on pure processes in general. Of the seven open orbits, only 28 is still open on the studied families.
- **How (H.36.1).** Before, the method tried to write the facet expression directly as a sum of squares, and for 69 and 81 that fell short. This round multiplies the expression first by $|\xi_K|^2$, a quantity that equals 1 everywhere it matters, so the bound is not changed. The product has more room to be written as a sum of squares. This is a standard trick and not new. It closes the gap for both orbits: from -0.0292 (81) and $-5{\cdot}10^{-5}$ (69) to zero within solver accuracy.
- **Controls (H.36.2).** The strengthened method still fails, as it must, on orbit 30, which is known to be broken. On 57 and 75, which are already proved, it gives zero as expected.
- **The check (H.36.3).** As for 57, the zero set of each expression fixes which directions the proof must leave out. On what remains the proof has room to spare, with a margin of about 10^{-3}, so it can be rounded to exact fractions. Two separately written checkers, each building the expression in its own way, accept both proofs. Both reject them if the bound is raised by 10^{-6}, if one entry is changed by 10^{-12}, or if one entry is negated. Reruns from a different random seed give different proofs, and both checkers accept those too.
- **Orbit 28 (H.36.4).** The new method improves its bound from -0.023 to -0.008 but does not reach zero. Two more variants did not help. Searches have never found a point that breaks it.
- **The tally.** On all pure processes it is unchanged: 15 of 22 broken, 7 open. On the families studied, only 28 is open.

122. Plain-language note: orbit 28, still open; the corner is not the problem (Oct 28 2026)

- **What was tried (H.37.1–H.37.2).** The proof method was built around the places where orbit 28's expression is exactly zero, by giving the proof extra terms that vanish there. That did not help. In fact, these extra terms can never beat simply raising the degree of the proof (Remark H.37.1). The best bound stays at -0.00065, short of the 0 a proof needs.
- **Where the gap is (H.37.2–H.37.3).** Cutting out strips around the zero set barely changes the gap. So the trouble is not at the zeros themselves, but in a wide region where the expression is positive but tiny (around 10^{-7}). That also rules out a brute-force interval proof, which would need about 10^{21} boxes.
- **The corner (H.37.4).** Where two of the zero surfaces cross, the leading term of the expression was computed exactly, and it is proved never to be negative. It vanishes only along the directions of the known zero surfaces.
- **But it does not decide anything (H.37.5).** Orbit 30, which is known to be broken, has exactly the same leading corner behaviour. So the corner is not where 28 and 30 differ. The difference lies along the zero surfaces, away from the corner, in a quantity that is zero at worst on 28 and negative on 30.
- **Status.** Orbit 28 remains open, both on the families studied and on pure processes in general. The tally is unchanged: on all pure processes, 15 of 22 broken and 7 open; on the families studied, only

28 is open.

123. Plain-language note: orbit 28, half of the deciding quantity proved (Oct 28 2026)

- **What was attacked (H.38.1).** Round 32 found that orbits 28 and 30 differ through one quantity, g. It measures how fast the expression grows as one moves off the surface {sin t_0 = 0}, where it is zero. That surface has two sheets, t_0 = 0 and t_0 = π. On each, g was computed exactly as a polynomial in the five remaining variables.
- **Proved on one sheet (H.38.2).** On the sheet t_0 = π, g is exactly a sum of three squares, so it is never negative. This holds for 28 and also for 30. So this sheet is not where the two orbits differ.
- **Numerical on the other (H.38.4–H.38.5).** On the sheet t_0 = 0, a sum-of-squares search gives zero to solver accuracy for 28 ($-1\cdot10^{-8}$), and -1.46 for 30, which is known to be broken. A 40-digit check confirms that 30's g really is -0.042 there. Turning the numerical result for 28 into an exact proof needs the zero set of the problem pinned down completely. About a quarter of the directions it forces are still unexplained, so no exact proof was obtained this round.
- **Why even a proof of g $\geq$ 0 would not be enough (H.38.6).** Where g $>$ 0, the expression is nonnegative near the surface. This is proved. But g vanishes on several curves and surfaces, and there higher-order terms decide. The nearby region where nonnegativity is guaranteed also shrinks to nothing as g approaches 0. And far from the surface the expression is positive but tiny, which is the region where previous rounds got stuck.
- **A better bound (H.38.7).** The plain method at a higher degree improves 28's bound from -0.00065 to -0.00053, and to -0.00047 in the variant that splits the domain by symmetry. That is still short of 0.
- **Status.** Orbit 28 remains open, both on the families studied and on pure processes in general. The tally is unchanged: on all pure processes, 15 of 22 broken and 7 open; on the families studied, only 28 is open.

124. Plain-language note: an audit, a recheck, and two more converses (Oct 31 2026)

The machine-checked proofs were audited. Every one of the 779 theorems in the main Lean library rests only on Lean's three standard axioms: no gaps, no shortcuts. Eleven theorems in older, separate files use a shortcut called `native_decide`, which trusts the compiler rather than the proof checker. Nothing in the main results depends on them. The half of Theorem 1.2 that says "only consistent loops are unitary" is now machine-checked too (update below).

The counterexample was rechecked from scratch, in exact arithmetic with a different tool (sympy), starting from the table printed in the paper. Everything came out as claimed. The inequality value is exactly $1/4 - (3/16)\sqrt{2}$. A new direct test plugged exactly unitary local operations into the machine, with and without an extra qubit per party, and got an exactly unitary result every time (5 of 5). The two "distance" numbers in the paper, 0.0076 and 0.0607, measure the same gap: one is the other divided by 8.

Ancillas are not needed for the converse (proved on paper). If every party's input and output have the same finite size, a loop that stays unitary for every choice of local unitaries, with no extra systems, must be a process function. In fact one randomly chosen set of local unitaries already tells you, with certainty. The key fact is that two different histories must always disagree somewhere their inputs agree. A computer check over all 16.8 million three-party binary loops found no exception.

Infinite alphabets (proved, and now machine-checked, with ancillas). The "copy gadget" argument carries over to countably infinite alphabets: an inconsistent loop gives a column whose length is its

number of fixed points, which is not 1 (it can even be infinite). As of Nov 1 the proof checker confirms this too (`PFUL.Countable.unitary_l2_iff_pf` L-069). It covers finitely many parties, alphabets you can list one by one (finite or infinite), outcomes you can add and subtract, and the copy trick's extra systems. It shows more than was asked: an inconsistent loop cannot even be a length-preserving map, let alone a unitary. Alphabets that form a continuum are not covered. Details: lift.md Appendix L.

Update, Nov 1: the infinite case without extra systems is settled too. It works out the same way: for a countable alphabet with addition, the same number of inputs as outputs, and finitely many parties, a loop that stays unitary for every choice of local unitaries, with no extra systems, has to be a process function. The proof checker confirms it (`PFUL.Countable.unitary_l2_noAncilla_iff_pf` L-177). The argument comes in two gentle steps. First, if a loop were inconsistent, you could pick local operations that give it away: mostly simple reshufflings, plus a small rotation by the angles of a 3-4-5 triangle wherever two histories agree on an output but not on an input. So a loop that passes every test has the "two histories always disagree somewhere their inputs agree" property. Second, that property alone already guarantees exactly one consistent history, which you can see by bringing the parties in one at a time; no counting is needed, so infinite alphabets are no obstacle. Alphabets that form a continuum are still not covered. *(An independent read-only check on 2026-11-01 found that the definitions in the Lean statements match the paper's.)*

Update, later the same day: both converses are now machine-checked. The proof checker now confirms both "only consistent loops are unitary" results: the one with extra systems (the copy trick, Theorem 1.2) and the one without (Theorem N.1). The only assumptions are finite alphabets with the usual arithmetic on them. The no-extra-systems proof no longer needs a random choice of local unitaries. When a loop is inconsistent, a concrete choice exposes it: a simple reflection, built from the vector 2e_p + e_{p+δ}, at parties whose outputs agree but whose inputs differ, and a reshuffling (permutation) everywhere else.

125. Plain-language note: alphabets of any size (Nov 2 2026)

The results no longer need alphabets you can list. Earlier, the infinite case covered alphabets you can count off one by one. As of Nov 2 the proof checker confirms the same results for discrete alphabets of any size at all, even ones as large as the real numbers (taken as a plain set of labels, not as a continuum). This holds for both the "consistent ⇔ unitary" theorem with the copy trick and the version with no extra systems (`PFUL.General`, lift.md Appendix L.4(c)).

Why the countable trick works. A sum of probabilities that adds up to 1 can only have countably many nonzero terms. So start from one history, add every outcome that has a nonzero chance of happening next, then everything reachable from those, and so on. After countably many rounds you have a countable "closed box": once you are inside it, nothing with a nonzero chance takes you out. Inside the box the loop is still consistent, since a strategy that only answers within the box has its unique consistent history in the box. So the countable theorem already proved applies there. Every finite set of terms fits inside some such box, and that pins the full sum at exactly 1.

Why each party needs at least one possible output. "Two histories always disagree somewhere their inputs agree" says nothing at all if there are no histories. A party with no possible outputs makes that property hold trivially, yet no consistent history exists. So the rule "this property ⇔ exactly one consistent history" needs every output alphabet to be nonempty, and the proof checker confirms the counterexample.

Still open. The results cover finitely many parties and still use addition on the alphabets. Alphabets

that form a genuine continuum (positions on a line) need a different formulation, because single points carry no weight there. Infinitely many parties were also open then; Section 126 settles the well-behaved case and gives a counterexample under an extra axiom.

126. Plain-language note: swapping out addition, and infinitely many parties (Nov 2 2026)

This note covers three things: a weaker rule that can replace addition, a proof for infinitely many parties when the loop is well behaved, and a counterexample when it is not. Each step says exactly what is proved, how, and by whom. Details are in lift.md L.4(d) and L.4(e) and in paper/notes/2026-11-02-general-alphabets.md.

(a) Addition is not needed: any "free shuffle" will do

What the addition was for. In the induced loop each party's local operation is read at "input plus offset", w_k(f) + e_k. The plus sign was never really about arithmetic. Its only job was to let the input *move* the offset: each possible input sends the fixed offset e_k to some label, and different inputs send it to different labels. So the plus can be replaced by a family of shuffles: for each input a, a rearrangement σ_a of the input labels. The loop then reads the operation at σ_a(e_k) instead of a + e_k. Ordinary addition is the case σ_a = "add a".

The one property that matters: freeness. Call the family *free* if two different inputs never send the same label to the same place: σ_a(c) = σ_a′(c) only when a = a′. Picture a little sorting office. Every input is a clerk, and every clerk has a rule for re-shelving any parcel. Freeness says two different clerks never put the same parcel on the same shelf. So by looking where a parcel ended up, you can always tell which clerk handled it.

A tiny worked example. Take three labels {0, 1, 2} and these three shuffles, none of which is "add a": - σ_0 swaps 1 and 2 and leaves 0 alone; - σ_1 swaps 0 and 1 and leaves 2 alone; - σ_2 swaps 0 and 2 and leaves 1 alone.

Follow parcel 0: the three clerks put it on shelves 0, 1, 2. Parcel 1 goes to 2, 0, 1, and parcel 2 goes to 1, 2, 0. Every parcel lands on three different shelves, so the family is free. (In table form: the shuffles make a Latin square, a grid where each row and each column uses every symbol once.) None of the shuffles is "add a", and that does not matter.

What the proof checker confirms (`lean/PFUL/BijectionLaw.lean`, standard axioms only, finitely many parties, discrete alphabets of any size): - *Consistent ⇒ unitary* needs **no condition at all** on the shuffles. Each one is a rearrangement, and that alone is enough (`induced_unitary_l2_σ` L-016). - *Unitary for every choice of local operations ⇒ consistent* (with the copy trick) needs the family to be **free** and every output alphabet to be **nonempty**. The outputs need no arithmetic at all, because the copy trick only swaps two labels (`unitary_l2_iff_pf_σ` L-017). - *The same with no extra systems* needs freeness, the same number of outputs as inputs (O_k ≃ I_k), and nonempty outputs (`unitary_l2_noAncilla_iff_pf_σ` L-018). Nonempty is genuinely needed: with no outputs the space of states is trivial and the claim fails.

The earlier plan was to require the shuffles to form a "simply transitive" action, meaning any label can be moved to any other by exactly one input. That turns out to be more than needed. Freeness alone suffices; "any label can reach any other" is never used.

Why freeness cannot simply be dropped. Take the laziest family: every input shuffles nothing, so σ_a is the identity for every a. Then the loop reads every party's operation at the same fixed offset, whatever the input. The loop's routing w has dropped out entirely, so the result is unitary for *every* w, consistent or not. The converse fails completely: unitarity no longer tells you anything about consistency. In the sorting-office picture, every clerk leaves every parcel where it was, so you can never tell who handled it. (This is a short argument on paper; it is not in the proof checker.)

(b) Infinitely many parties, the well-behaved case: proved

The question. So far every result has had finitely many parties. Now imagine infinitely many, each with finitely many possible outputs (say yes/no). Recall the key property, *exclusivity*: any two different histories disagree at some party whose input is the same in both. With finitely many parties, exclusivity guarantees exactly one consistent history. Does it still with infinitely many?

A story: the infinite row of clerks. Picture an endless row of clerks, each writing yes or no on a card. What each clerk is told (their input) depends on what the others wrote. A *consistent history* is a set of cards where every clerk's card agrees with what their rule says, given what they were told.

What "continuous" means here. Call a loop *continuous* if every clerk's input is decided by finitely many other cards. Each clerk may need a different finite set, and the sets can be as large as you like, but for each clerk there is some finite set of cards that settles it. (In general a clerk's input may only need to be known to a given precision, but here inputs are discrete labels, so "to any precision" just means "exactly".) Nothing far away can change what a given clerk is told. Mathematicians say "continuous in the product topology"; this is that phrase in plain language.

Step 1: freeze everyone outside a finite group. Pick any finite group T of clerks, and freeze every clerk outside T with some fixed card t. Now look only at the clerks in T: given their cards u, compute what the loop tells each of them, with the rest frozen at t. This is a loop with finitely many parties.

Step 2: the frozen loop is still exclusive. Take two different card-sets u and u′ for the group T, with everyone else frozen at t in both. By exclusivity of the whole loop, the two full histories disagree at some clerk whose input is the same in both. That clerk must be *inside* T, because outside T both histories hold the same frozen card t. So the frozen loop on T is exclusive too.

Step 3: the finite theorem applies. For finitely many parties, exclusivity gives exactly one consistent history (`exclusive_iff_pf` L-126). So for any finite group T there are cards that make every clerk *in T* consistent.

Step 4: compactness glues the pieces together. For each clerk k, let C_k be the set of full histories in which clerk k is consistent. Step 3 says that any finitely many of these sets have a history in common. We want a history in *all* of them at once. - Because the loop is continuous, each C_k is decided by finitely many cards. That makes C_k *closed*, meaning it contains its own limits. - The space of all histories (one card per clerk, each from a finite alphabet) is *compact*. That is Tychonoff's theorem. - In a compact space, closed sets that always meet finitely many at a time all meet together. This is the *finite intersection property*.

Picture an endless series of shrinking closed boxes, each sitting inside the last and none of them empty. In a compact space they cannot shrink to nothing, so there is a point in all of them. That point is a history in which every clerk is consistent.

Step 5: there is only one. If there were two different consistent histories, exclusivity would give a clerk whose input is the same in both but whose cards differ. That is impossible when the clerk is consistent

in both, because the same input gives the same card. So the consistent history is unique.

What the proof checker confirms (`lean/PFUL/InfiniteCompact.lean`, `pf_of_exclusive_continuous` L-134, standard axioms only): - any set of parties, of any size; - finitely many possible outputs per party, at least one; - a continuous loop; - exclusive ⇒ every choice of local actions has exactly one consistent history.

Continuity is used in exactly one place: to make the sets C_k closed.

(c) The badly-behaved case: it can fail (a consistency result, now machine-checked)

First, a simplification. It is enough to find a single map F, sending each history to another history, that has no *fixed point* (no history s with F(s) = s) and is exclusive: any two different histories s, s′ disagree at some clerk k where F(s) and F(s′) agree. Given such an F, take it as the loop and let every clerk simply copy what they are told. Then the loop is exclusive and has no consistent history. The proof checker confirms this reduction (`exists_exclusive_not_pf_iff` L-146). We use countably many clerks, numbered 1, 2, 3, …, each answering yes/no, so a history is an infinite yes/no sequence.

Idea 1: decide by the long run only. Call two sequences *neighbours* if they differ in only finitely many places. Neighbours fall into classes; each class is "a long-run pattern". Let F give the *same* answer to every sequence in a class: F(s) = the chosen representative of g([s]), where [s] is the class of s and g is a rule on classes, still to be built.

Why pairs inside one class come for free. If s and s′ are neighbours, then F(s) = F(s′), so F agrees with itself at every clerk. Since s ≠ s′, they differ somewhere, and there F's two outputs agree. Exclusivity holds automatically.

The two conditions on g. Everything then reduces to two conditions on the class-rule g: 1. **No fixed point:** g(x) ≠ x for every class x. Then F(s) never lands in the class of s, so F(s) ≠ s. 2. **Different classes:** for different classes x ≠ y, the set where x and y differ must contain *infinitely many* places where g(x) and g(y) agree. "Infinitely many" is what is needed: neighbours can move any finite set of differences around, and the agreement must survive every such move.

Building g one class at a time. There are $\mathfrak{c}$ classes ($\mathfrak{c}$ is the size of the continuum). List them as a transfinite sequence x_0, x_1, …, x_α, … of length $\mathfrak{c}$, and choose g(x_α) in order. At step α, each earlier choice, plus the "no fixed point" rule, *forbids* one set of candidates for g(x_α): the sequences that copy a specific forbidden pattern on a specific infinite set R, apart from finitely many places. Each forbidden set is *meager*, which means "thin" in a precise sense. It is a countable union of the sets "z matches the pattern on R, beyond place n". Each of those is closed and has no interior, because R beyond n is still infinite and so can always be wiggled. At step α there are fewer than $\mathfrak{c}$ forbidden sets ($|\alpha| + 1$ of them).

The extra axiom. The axiom cov(M) = $\mathfrak{c}$ says exactly that fewer than $\mathfrak{c}$ meager sets can never cover all sequences. So at every step a good choice is left over, and the recursion never gets stuck. The continuum hypothesis (CH) implies cov(M) = $\mathfrak{c}$, and so does Martin's axiom. Under CH there is also a hands-on choice: pick disjoint infinite pieces of the R's and deliberately break each forbidden pattern on its own piece.

Why it fools every finite check but still fails. Freeze all but finitely many clerks, and the finite theorem from (b) says the rest can be made consistent. This loop passes that test for every finite group and every frozen background. What breaks is the gluing: F jumps wildly with far-away cards (changing infinitely many cards can move a sequence into a different class and change F's answer), so the sets C_k

are not closed. The shrinking boxes of (b) have nothing to force a common point, and indeed there is none.

Status, stated carefully. - The paper proof was refereed by a Gödel likeness (verdict: correct; /workspace/leibniz-likeness/godel-check-E.md). As of Nov 2 the **proof checker confirms it too** (`lean/PFUL/CovMCounterexample.lean`, standard axioms only). There, $\mathrm{cov}(M) = \mathfrak{c}$ is an explicit assumption of the theorem, not a new axiom, and a second theorem derives that assumption from CH. The checker's version goes through the sequences one at a time in a suitable well-order: a sequence with an earlier neighbour copies that neighbour's answer, so F still gives one answer per class. - It is a **consistency result only**: if ordinary set theory (ZF) is consistent, so is ZFC together with "an exclusive loop with countably many yes/no parties and no consistent history". It does *not* by itself show that such a loop exists in ordinary set theory; (d) now does, by a different, explicit construction. - The opposite assumption, $\mathrm{cov}(M) < \mathfrak{c}$, is also consistent (for example in the random-real model), and there the construction says nothing.

(d) Settled in ordinary set theory (Nov 3)

- **ZFC, explicit, Borel** (Borel, in fact exactly Baire class 2 [proved, Lean: `PFUL.Infinite.zfcF_borel_measurable` L-247, `zfcF_coord_baireTwo` L-248, `zfcF_not_baireOne` L-249]). The "possible shortcut" anticipated here works on the sequences with finitely many "yes" answers. The loop: a sequence with finitely many yeses is shifted one place right and gets a "yes" put in front; a sequence with infinitely many yeses is sent to all "no". It is exclusive and has no consistent history (`PFUL.Infinite.exists_exclusive_not_pf` L-233, standard axioms only; Story 4 of Section 127). The construction of (c) is no longer needed; it remains correct, as an aside.
- **Why, in one line.** Write $G(s) = F(s) \oplus s$, the "conjugate" of Richard (arXiv:1302.6346). Exclusivity says: freeze any set of coordinates, and G, read only on the free coordinates (G(s)|_D), is one-to-one (reading all of G is not enough; `PFUL.Infinite.exclusive_iff_conj_injOn` L-236); a consistent history is a point where G is 0. With finitely many parties one-to-one forces onto (`PFUL.Infinite.isProcessFunction_of_exclusive` L-238); with infinitely many it does not (`PFUL.Infinite.zfcF_conj_injective_not_surjective` L-239).
- **Still open.** Which intermediate kinds of discontinuity (for example Baire class 1) already force a consistent history. Section 128 tells, in plain language, what has been settled around that question and what has not. Finite-prefix approximations force a consistent history when the exceptional set's finite differences accumulate only among themselves, and in particular when those differences are bounded. One-sided Σ^0_2 and Π^0_2 ledgers can still fail. Continuity off a closed perfect nowhere-dense set can still fail. For Baire class 1, restricting to maps that are constant off such a set is the same question as the unrestricted one. The countable-closed conjecture is now a theorem (Section 129). Section 130 rules out finite palettes of whole output lists, and output ranges trapped in one finite-difference class. An arbitrary countable exceptional set, even with Baire class 1, and the unrestricted Baire-class-1 question, remain open. No exclusive Baire-class-1 counterexample is known.

127. Four stories for the Nov 2 results

The results of Sections 125 and 126 are technical, and this section retells them as stories, in the style of Part I. Each story ends with what is actually proved and where. The stories are illustrations. If one seems to say more than the technical note, the note wins.

Story 1: the guest list that closes itself (the countable closure trick)

A host plans a party in a city with more people than anyone could ever list, uncountably many. The host's rule: invite one friend, then invite everyone who has a real chance of being brought along by someone already invited, then everyone *they* might bring, and so on.

It sounds as if the list must swallow the whole city. It does not. Each guest can bring along only countably many people with a nonzero chance, because probabilities that add up to 1 can have only countably many nonzero terms. Countably many rounds of countably many additions give a countable list. And the list is *closed*: nobody on it can bring anyone who is not already on it.

That is the whole trick of Section 125. A consistent loop on a huge alphabet behaves, inside such a closed list, exactly like a loop on a countable alphabet. The countable theorem already proved then applies inside the list. Since every finite set of guests fits inside some closed list (start the list with them), the full total is pinned at exactly 1.

What is proved. Lean ✓: `PFUL.General.tsum_prod_eq_one_anc_any` L-131 (the total is 1), used in `PFUL.General.induced_unitary_l2_any` L-130 (the lift is unitary for discrete alphabets of any size). The easier route through "infinite product measures" does not work here. An uncountable pile of zero-probability events can add up to more than zero, and the closed list avoids that problem.

Story 2: the cloakroom (the free shuffle)

A cloakroom has numbered hooks. Each attendant has their own rule for moving a coat from the hook where it was dropped to a hook at the back. The cloakroom is *honest* if, from where a coat ends up, you can always tell which attendant moved it. In other words, two different attendants never put the same coat on the same hook. That is freeness (Section 126(a)).

Ordinary addition is one honest cloakroom: attendant a moves the coat from hook c to hook c + a. The Latin-square cloakroom of Section 126(a) is another honest one, with no addition in sight. Now picture a *lazy* cloakroom where no attendant ever moves anything. Every coat stays on its hook, and you cannot tell who touched it.

Here is what the loop needs. Consistent loops come out unitary whatever the attendants do, honest or lazy, because any rearrangement of hooks loses nothing. The *converse* is the detective's direction: "unitary for every test ⇒ consistent". It needs honesty, because the detective reads the routing off where the coats ended up. In the lazy cloakroom the routing w leaves no trace at all, so every loop, consistent or not, passes every test.

What is proved. Lean ✓: `PFUL.General.induced_unitary_l2_σ` L-016 (no condition on the shuffles), `PFUL.General.unitary_l2_iff_pf_σ` L-017 and `PFUL.General.unitary_l2_noAncilla_iff_pf_σ` L-018 (honest, meaning free, shuffles and at least one possible output per party). The lazy-cloakroom failure is now Lean ✓ too: `PFUL.General.lazy_unitary_all` L-257, `PFUL.General.lazy_converse_fails` L-258 (a non-PF loop that passes every test when no attendant moves anything).

Story 3: the infinite village of gossips (compactness)

An endless village, each villager answering yes or no. What each villager hears depends on others' answers, but on only finitely many of them, though different villagers may listen to different neighbours. The village has the pairwise property: any two different sets of answers clash at some villager who hears the same thing in both.

The mayor wants one set of answers in which everybody is consistent. The mayor cannot check everyone at once, so the mayor checks committees. Take any finite committee, freeze everyone else, and the finite theorem says the committee can be made consistent. Every committee passes.

Does passing every committee test mean the whole village can be satisfied? Here the gossip's locality does the work. Because each villager listens to only finitely many others, "villager k is consistent" is a condition you can check on finitely many answers. Conditions of that kind, on a space of yes/no answers, have a magic property called compactness. If every finite batch of them can be met together, then all of them can. It is like a nest of boxes, each inside the last and none empty: something sits in all of them.

What is proved. Lean ✓: `PFUL.Infinite.pf_of_exclusive_continuous` L-134 (any number of parties, finitely many outputs each, continuous loop ⇒ exactly one consistent history), and `PFUL.Infinite.fixed_unique` L-147 (never two). This is Section 126(b), told shorter.

Story 4: Hilbert's hotel of time travellers (the counterexample in ordinary set theory)

A hotel has rooms 0, 1, 2, … forever. A *guest list* says which rooms are occupied: a yes/no answer for each room. Each night the loop hands back tomorrow's list as a function of tonight's, and a history is consistent if tomorrow's list equals tonight's.

The rule of this hotel: - If tonight's list is **finite**, everyone moves one room to the right (room n to room $n + 1$), and a new guest checks into room 0. - If tonight's list is **infinite**, everyone leaves: tomorrow the hotel is empty.

No list stays put. An infinite list becomes the empty list, which is finite, so it changed. A finite list that stayed put would need room 0 occupied (the new guest), then room 1 occupied (the guest from room 0 moved up), then room 2, and so on forever. That list is infinite, so it was not finite after all. Every possible night is a grandfather paradox.

Yet no two lists ever clash pairwise. Take two different lists. We must find a room where they differ tonight but the hotel's rule sends the same answer to that room tomorrow. - Both infinite: tomorrow both are empty, so any room where they differ works. - Both finite: take the first room m where they differ. Tomorrow room m holds either the new guest (if $m = 0$, in both) or whoever was in room $m - 1$, and the two lists agree there because m was the *first* difference. - One finite, one infinite: far beyond the finite list's last guest, the infinite list has an occupied room. Tonight the lists differ there; tomorrow both have it empty.

Why only an infinite hotel allows this. Record, for each list, the rooms where tomorrow's list differs from tonight's (the "conjugate", G). The pairwise property says G never sends two lists to the same record, even when you freeze some rooms and compare the records only on the rooms you let vary (one-to-one). A consistent night is a list whose record is "no change". In a finite hotel a one-to-one map of lists onto lists must hit everything, including "no change", so a consistent night exists. That is the finite theorem in one line. In an infinite hotel, one-to-one does not mean onto: this is exactly Hilbert's trick of making room for a new guest by shifting everyone along. Here the record "no change" is the room that is never filled.

Aside: the size-of-infinity version. Before this explicit hotel was found, the counterexample was built by a transfinite recursion that needed the extra axiom $\mathrm{cov}(M) = \mathfrak{c}$, which follows from the continuum hypothesis (Section 126(c); `PFUL.Infinite.exists_exclusive_not_pf_of_covM` L-119, `PFUL.Infinite.exists_exclusive_not_pf_of_CH` L-118). The worry that the answer might depend

on the size of infinity is now gone: ordinary set theory already builds the paradoxical loop, and it is even explicitly definable (Borel, in fact exactly Baire class 2 [proved, Lean: `PFUL.Infinite.zfcF_borel_measurable` L-247, `zfcF_coord_baireTwo` L-248, `zfcF_not_baireOne` L-249]).

What is proved. Lean ✓, standard axioms only, no extra hypothesis: `PFUL.Infinite.exists_exclusive_not_pf` L-233 (the hotel gives an exclusive loop with no consistent history), `PFUL.Infinite.zfcF_exclusive` L-234, `PFUL.Infinite.zfcF_ne` L-235, and the one-to-one-but-not-onto reading `PFUL.Infinite.zfcF_conj_injective_not_surjective` L-239. The conjugate idea and the finite criterion "unique fixed points everywhere iff the conjugate is a bijection on every subnetwork" are from Richard (arXiv:1302.6346), crediting Ruet [cited]. The physical moral is unchanged: realistic loops have finitely many agents, or agents who each hear only finitely much, and there the question is settled (Story 3).

128. Three stories for finite prefixes, one-sided ledgers, and a hidden perfect set

The additions to L.4(e) are technical, and this section retells them as stories, in the style of Section 127. Each story ends with what is actually proved and where. The stories are illustrations. If one seems to say more than the technical note, the note wins.

Story 1: the night that is almost consistent (finite prefixes)

Keep the hotel of Story 4, but change the rule. Off some set S of guest lists, tomorrow is always the same list c. On S the rule may do whatever it likes.

For each n, freeze every room from n onward so that it matches c, and ask only the first n rooms to be consistent with the rule. That is a finite problem. The pairwise property on a finite set of rooms gives a solution x_n: a list that agrees with c from room n on, and that is consistent on rooms 0 through n − 1. These are approximate fixed points. Each one is a night that is settled on a longer finite prefix and copied from c afterwards.

Suppose those approximate lists have a limit y, and y itself differs from c in only finitely many rooms, and from some room on the rule at y agrees with y. Then y is a genuine consistent history. Some late approximate list x_n agrees with y on the finite prefix that matters, and the pairwise property forces x_n and y to be the same list.

This argument goes through whenever the lists in S that differ finitely from c accumulate only at other such lists. It goes through in particular when those differences are bounded in length: nobody in S differs from c in more than K rooms. The obstruction is the other kind of accumulation. Finite lists can pile up on an infinite one. In the hotel of Story 4 the finite initial segments, rooms 0 through n − 1 occupied and the rest empty, accumulate at the list in which every room is occupied, and that limit is not a consistent night.

If S is countable and closed, every map that equals c off S is Baire class 1, whatever values it takes on S. A counterexample of this shape, had one existed, would have been a class-1 map using infinite accumulation: finitely supported points of S accumulating at an infinitely supported one. The bounded-support families that a finite computer search had left undecided are theorems. They have a fixed point. The search was evidence. The prefix argument is the proof. Section 129 shows that a countable closed S cannot carry that accumulation at all, so the countable-closed conjecture is a theorem. Countability does not by itself make S closed.

What is proved. Lean ✓: `PFUL.Infinite.exists_prefix_fixed` L-269, `PFUL.Infinite.fixed_of_prefix_agree` L-270, `PFUL.Infinite.exists_fixed_of_closure_finite_diff` L-271, `PFUL.Infinite.exists_fixed_of_bounded_diff` L-272, `PFUL.Infinite.baireOne_of_const_off_countable_closed` L-268, together with the bookkeeping `PFUL.Infinite.const_mem_of_fpf` L-265, `PFUL.Infinite.box_subset` L-266 and `PFUL.Infinite.finite_defect_of_countable` L-267. Standard axioms only.

Story 2: ledgers that only count one way (Σ^0_2 and Π^0_2)

A yes/no ledger for each room can be complicated in two one-sided ways. Σ^0_2 means each "yes" is a countable union of closed conditions: one of countably many closed reasons is enough to be sure. Π^0_2 means each "yes" is a countable intersection of open conditions: the yes survives every finite check.

The hotel of Story 4 is Σ^0_2 in every room. Each room's yes/no is a countable union of closed sets. Now flip every bit of tonight's list, run the hotel, and flip every bit of the answer. Call that the dual hotel. It is still pairwise consistent, in the sense of exclusivity, and it still has no night that stays put. Each room's "yes" is now Π^0_2.

So a ledger that is merely one-sided, only Σ^0_2 or only Π^0_2, can still be a grandfather paradox that passes the pairwise test. Baire class 1, for yes/no answers on this space of lists, lives inside the two-sided class Δ^0_2: the yes-sets that are both a countable union of closed sets and a countable intersection of open sets. That two-sided question is still open. It is one open question among others in the note, including the unrestricted Baire-class-1 question and the question about triangular maps in the technical note. The countable-closed conjecture of Story 1 is settled in Section 129. Settling the one-sided ledgers leaves those questions where they were.

What is proved. Lean ✓: `PFUL.Infinite.zfcF_coord_sigma02` L-273, `PFUL.Infinite.zfcD_ne` L-274, `PFUL.Infinite.zfcD_exclusive` L-275, `PFUL.Infinite.zfcD_coord_pi02` L-276. Standard axioms only.

Story 3: the even seats (a bad map hidden on a perfect dust)

Give every room a partner. Even rooms 0, 2, 4, … and odd rooms 1, 3, 5, …. A list is in the set P when every odd room is empty. On such a list, read the even rooms as a list of their own, hand that list to the old map, and write the old map's answer back on the even rooms. Every odd room of the answer is empty, even on P. If any odd room is occupied, ignore the old map and send everyone home: the answer is the constant empty list.

P is a thin perfect dust. It is closed, because a limit of lists with all odd rooms empty still has them empty. It has no interior: next to any list you can occupy one further odd room and step outside P, so P contains no solid block of lists. It is perfect: every list in P is a limit of other lists in P, by flipping one far even room. A closed set with empty interior is nowhere dense, and a perfect set has no isolated point. P is uncountable.

The padded map is the constant empty list on the open set of lists outside P, so it is continuous at every list outside P. Every actual discontinuity lies in P. This story does not say that every list in P is a discontinuity. Continuity off P puts the discontinuities inside P. Equality of the two sets would be a further theorem, and neither the Lean file nor the technical note asserts it.

Pad the hotel of Story 4 in this way. The result is still exclusive, still has no consistent night, and is still not Baire class 1. It is constant off P. So continuity off a closed perfect nowhere-dense set does not force

a consistent history. The prefix theorem of Story 1 really uses countability of the exceptional set. P is closed and uncountable, and the padded hotel, constant off P, has no fixed point.

There is a second fact, and it is an equivalence, not a shortcut. Padding preserves Baire class 1 in both directions. If the old map is a pointwise limit of continuous maps, so is the padded map. If the padded map is such a limit, so is the old map, by reading it back along the even rooms. Therefore an exclusive fixed-point-free Baire-class-1 map pads to another map that is exclusive, fixed-point-free, Baire class 1, and constant off P. The converse direction is the same equivalence read backwards. The question "does exclusive plus Baire class 1 force a fixed point, for maps constant off this perfect dust?" is the same question as "does exclusive plus Baire class 1 force a fixed point?" The geometric picture reduces the restricted question to the full open problem. It does not answer it. The countable-closed question of Story 1 is a different question. It is settled in Section 129. The padding argument does not settle it, and Section 129 does not settle the full Baire-class-1 question.

What is proved. Lean ✓, standard axioms only: `PFUL.Infinite.PerfectPad.isClosed_P` L-277, `PFUL.Infinite.PerfectPad.interior_P` L-278, `PFUL.Infinite.PerfectPad.perfect_P` L-279, `PFUL.Infinite.PerfectPad.pad_exclusive` L-280, `PFUL.Infinite.PerfectPad.pad_ne` L-281, `PFUL.Infinite.PerfectPad.pad_continuousAt` L-282, `PFUL.Infinite.PerfectPad.baireOne_pad_iff` L-283, `PFUL.Infinite.PerfectPad.exists_exclusive_fpf_const_off_perfect` L-284, `PFUL.Infinite.PerfectPad.reduction` L-285.

129. Two stories for countable lists, finite corners, and a dense dust

These stories retell the Nov 7 theorems that settle the countable-closed conjecture and its G-delta strengthening. They are illustrations. If a story seems to say more than the technical note, the note wins. Two pictures in the technical note, a countable discrete G-delta whose closure contains a perfect set, and an uncountable closed set that ties coordinates in pairs, are illustrations. They are not Lean declarations.

Story 1: the boxes a countable closed list cannot hold

Keep the hotel rule that, off a guest list S, always returns the same list c. Suppose S is countable and closed, and the rule is exclusive. Story 1 of Section 128 built approximate nights x_n, consistent on the first n rooms and equal to c afterwards. Each such night lies on S. Otherwise the rule would return c, the prefix equations would force the night to be c itself, and c would be a consistent history.

The pairwise rule drags more than the night itself onto S. From x_n, every list obtained by flipping only rooms where x_n already differs from c must also lie on S. That is a whole finite box. Now let the approximate nights approach a limit y. Because S is closed, the entire box between c and y lies on S, not merely the finite boxes along the way. A countable list cannot contain an infinite box: an infinite box holds as many lists as the whole hotel of yes/no answers. So y differs from c in only finitely many rooms. The prefix argument of Section 128 then makes y a consistent history, and exclusivity makes it the only one. It differs from c in finitely many rooms.

Countability does not make a list closed. The finite-support lists in Hilbert's hotel are countable, and they are not closed: they accumulate on the list in which every room is occupied. Closedness does not by itself force a consistent history either. The perfect dust of Section 128 is closed, and the padded hotel on it has no consistent night. That dust is uncountable, and it contains an infinite box, the even rooms.

What is proved. Lean ✓: `PFUL.Infinite.finite_diff_of_countable_box` L-286, `PFUL.Infinite.cluster_box_subset_of_prefix_fixed` L-287, `PFUL.Infinite.exists_fixed_of_const_off_countable_closed` L-288, `PFUL.Infinite.exists_unique_fixed_of_const_off_countable_closed` L-289,

PFUL.Infinite.fixed_finite_diff_of_const_off_countable L-290. Standard axioms only. This is the countable-closed conjecture, now a theorem. It is not a solution of the general Baire-class-1 question.

Story 2: finite corners, and why a dense dust cannot be countable

Drop closedness. Keep S countable, keep the rule exclusive and equal to c off S, and suppose there is still no consistent night. The approximate nights and their finite boxes still lie on S. Pass to a limit y. If y differed from c only finitely often, the prefix argument would build a consistent night. So y differs from c on infinitely many rooms.

A finite corner of y's box is a list t that differs from c in only finitely many rooms, and only in rooms where y itself differs from c. Some late approximate night agrees with y on those finitely many rooms. Reset every other room to c, and the result is exactly t, not a sequence creeping toward t. So every finite corner lies on S. No closedness is used for this step.

Those corners are dense in the infinite box. Match any list in the box on a longer finite set of rooms, and you have a corner nearby. If S were a countable intersection of open sets, a G-delta, then S cut with this box would be a countable dense G-delta inside a perfect compact space of yes/no answers. That is impossible. Each single list is closed and has empty interior, and a dense G-delta in this space cannot be covered by countably many such sets. So a countable G-delta exceptional set forces a unique consistent history.

G-delta here is a property of the whole exceptional set. It is not a property of the separate ledger for each room. Baire class 1, for a map constant off a countable set, is exactly the demand that each room's defect set be a G-delta. The union of those room-defects, the actual exceptional set, need not be a G-delta: a countable union of G-delta sets need not be G-delta. So exclusive plus Baire class 1 plus an arbitrary countable exceptional set is still open. The unrestricted Baire-class-1 question, with no "constant off a countable set" assumption, is still open. This story does not solve either one.

The strongest new structural fact is the finite-corner obstruction. Any hypothetical counterexample that is constant off a countable set, whether or not it is Baire class 1, must carry every finite corner of some infinite box. Closedness would then drag the whole limit box into S and contradict countability. G-delta regularity contradicts the corners being dense. Without one of those, the corners can sit in S while the rest of the box stays outside.

A closed exceptional set need not be countable. What the closed argument really used, twice, is that S contains no infinite coordinate box. Call such a set box-thin. A closed box-thin set, countable or not, still forces a consistent history. The perfect dust of Section 128 is not box-thin: its even rooms form an infinite box. An illustration of an uncountable closed box-thin set, tying later rooms in pairs, is in the technical note. That illustration is not a Lean declaration.

What is proved. Lean ✓, standard axioms only: PFUL.Infinite.finite_corners_subset_of_prefix_cluster L-291, PFUL.Infinite.exists_infinite_finite_corner_box_of_fpf_countable L-292, PFUL.Infinite.finite_diff_of_countable_isGδ_finite_corners L-293, PFUL.Infinite.exists_fixed_of_const_off_countable_isGδ L-294, PFUL.Infinite.not_isGδ_exceptional_of_fpf_countable L-295, PFUL.Infinite.isGδ_fiber_of_baireOne_bool L-296, PFUL.Infinite.isGδ_coordinate_defect_of_baireOne L-297, PFUL.Infinite.baireOne_of_const_off_countable_isGδ L-298, PFUL.Infinite.baireOne_iff_coordinate_defects_isGδ_of_const_off_countable L-299, PFUL.Infinite.exists_unique_fixed_of_const_off_countable_isGδ L-300, PFUL.Infinite.coordinateBoxThin_of_countable L-301, PFUL.Infinite.exists_

`fixed_of_const_off_closed_coordinateBoxThin` L-302, `PFUL.Infinite.exists_infinite_box_of_fpf_const_off_closed` L-303.

130. A finite palette of whole lists, and a witness that runs away

These stories retell a later Nov 7 tranche. They are illustrations. If a story seems to say more than the technical note, the note wins. The general question, exclusive and Baire class 1 implies a consistent history, stays open. Nothing here is a counterexample to it.

Story 1: the hotel that only ever prints finitely many guest lists

Each room still answers yes or no, so each room, looked at alone, has only two possible answers. That is not news, and it does not settle an infinite hotel. The new theorem is about the whole list at once. Suppose the rule, whatever tonight's list is, only ever prints one of finitely many complete guest lists tomorrow. A finite palette of whole lists, not a finite palette at each room.

Tie together the rooms that always receive the same answer on that finite palette. Two rooms are tied when, on every list the rule actually prints, they agree. There are only finitely many such patterns, because a pattern is a yes/no assignment to a finite menu. The infinite hotel collapses to a finite committee, one seat per pattern. The pairwise property survives the collapse, the finite theorem supplies a consistent night for the committee, and that night lifts to the whole hotel.

The same idea gives full consistency, not only one fixed history of the rule itself. The parties may receive messages from arbitrary alphabets. If the world's whole message-list still comes from a finite palette, then every way the parties answer those messages has exactly one consistent history. Arbitrary dependence on the input is allowed. Continuity is not used. A finite palette of whole outputs is simply not a route to a grandfather paradox.

A different output restriction also forces a consistent night, and it is not a restriction on which inputs are exceptional. Fix one baseline list c. If every list in the closure of the outputs differs from c in only finitely many rooms, there is a consistent night. In particular this holds when no output differs from c in more than K rooms, and when the set of outputs is closed and each output differs finitely from c, with no uniform K. That is a hypothesis on what the rule prints. It does not say that a Baire-class-1 rule prints only such lists.

What is proved. Lean ✓: `PFUL.Infinite.exists_fixed_of_finite_global_range` L-304, `PFUL.Infinite.isProcessFunction_of_exclusive_finite_global_range` L-305, `PFUL.Infinite.exists_fixed_of_rangeClosure_finite_diff` L-306, `PFUL.Infinite.exists_fixed_of_bounded_output_diff` L-307, `PFUL.Infinite.exists_fixed_of_closed_range_finite_diff` L-308. Standard axioms only.

Story 2: the witness that runs off the end of the hotel

One tempting route is to solve longer and longer finite prefixes and pass to the limit. For an exclusive rule there is a unique way, P_n(b), to fill the first n rooms consistently and leave the tail equal to a prescribed boundary b. These operators commute: doing m after n is the same as doing the longer of the two. If the rule is Baire class 1, each finite P_n is Baire class 1 as well. The proof looks at the whole finite table of answers of the continuous approximants. It does not pretend those approximants are exclusive.

If the P_n converge, the limit ignores any finite change in its input. A Baire-class-1 yes/no answer with that indifference is constant, so a Baire-class-1 limit would be a constant list. What is not proved is that

the P_n converge, or that a limit they do have stays Baire class 1. A constant limit would still not let you swap the order of limits at a discontinuity of P_m. Continuity of every P_m at a cluster point, in the whole space of lists, does produce a consistent night. Continuity only on the cluster set does not: the approximating lists can approach that set from outside it. For the Hilbert hotel and the empty boundary, the cluster set is the single all-yes list, every map restricted to that one point is continuous, and there is still no consistent night. That example is Baire class 2. It shows the gap in the argument. It is not an exclusive Baire-class-1 counterexample.

A second temptation is to take continuous exclusive approximations and pass to the limit. Cut Yablo's rule off at room N: from N onward the answer is yes, and below N a room says yes when every later room short of N says no. Each of these finite machines is continuous and exclusive on the whole hotel, not merely on finite faces. They converge to Yablo's map. That limit is Baire class 1 and has no consistent night, and it is not exclusive: the all-no list and the all-yes list have no witness. So global exclusivity of every approximant need not survive. This is not a counterexample to the open question, because the limit fails the pairwise test.

What does survive is the pairwise test for lists that differ in only finitely many rooms. Stabilize the approximants on that finite set and use one approximant's witness. The witnesses that escape are the ones for pairs that differ in infinitely many rooms: the witness can run off toward larger and larger room numbers and never settle.

A third shortcut also fails, and only that shortcut. For the two-room rule "answer the opposite of the other room's bit, and answer yes yourself," the point (yes, no) is as large as it can be among points that sit below their own image, and it is not a consistent night. The consistent night is (no, yes). So "maximal and below its image" does not become "equal to its image" from the pairwise property alone. The example is finite, continuous, and exclusive. It does not say that no semicontinuity theorem exists. It says this proposed proof does not.

What is proved. Lean ✓, standard axioms only: `PFUL.Infinite.prefix_solution_unique` L-309, `PFUL.Infinite.prefixNormalize_comp` L-310, `PFUL.Infinite.baireOne_prefixNormalize` L-311, `PFUL.Infinite.normalizer_limit_tail_invariant` L-312, `PFUL.Infinite.eq_of_baireOne_tail_invariant_bool` L-313, `PFUL.Infinite.normalizer_baireOne_limit_constant` L-314, `PFUL.Infinite.fixed_of_normalizer_cluster_continuity` L-315, `PFUL.Infinite.fixed_of_normalizer_self_limit_finite_defect` L-316, `PFUL.Infinite.finiteYablo_exclusive` L-317, `PFUL.Infinite.continuous_exclusive_limit_can_fail_exclusive` L-318, `PFUL.Infinite.finite_pair_exclusive_of_pointwise_limit` L-319, `PFUL.Infinite.yablo_finite_pair_exclusive` L-320, `PFUL.Infinite.maximal_postfixed_need_not_be_fixed` L-321.

Appendix: superseded readings (written assuming the conjecture)

These passages were written while the conjecture (every purifiable classical process is a mixture of process functions) was still open. Section 58h shows that it is false. They are kept as a record only, and nothing else in this guide relies on them.

From Section 64 (Can chance be fundamental inside a loop?). *The reading below was written assuming the conjecture and is superseded.* If the conjecture held, the note's reading would be this. Every reversible quantum loop with classical statistics is a coherent ignorance-mixture of deterministic consistent loops. Genuinely chancy loops like E_ex1 would be classically consistent but not reversibly realisable. **Reversibility, not consistency alone, would be the constraint that excludes basic chance from loops.** D.5 sharpens this: consistency under all quantum interventions holds for every

classical process, so it is precisely the reversibility requirement that does the excluding.

From Section 65 (Determinism hidden, or created on the fly?). Interpretation (written assuming the conjecture; superseded by Section 58h for loops outside the hull). If the conjecture were true, the two pictures cannot be told apart by any loop's classical statistics. Every quantum-reversible chancy loop has the statistics of a shuffled deck. But the *mechanism* may still be on-the-fly. The ignorance reading is safe as a claim about W. It is not safe as a claim about a hidden label. This is the note's own caveat. That it echoes contextuality in quantum foundations (Kochen and Specker) is my comparison, not a result.